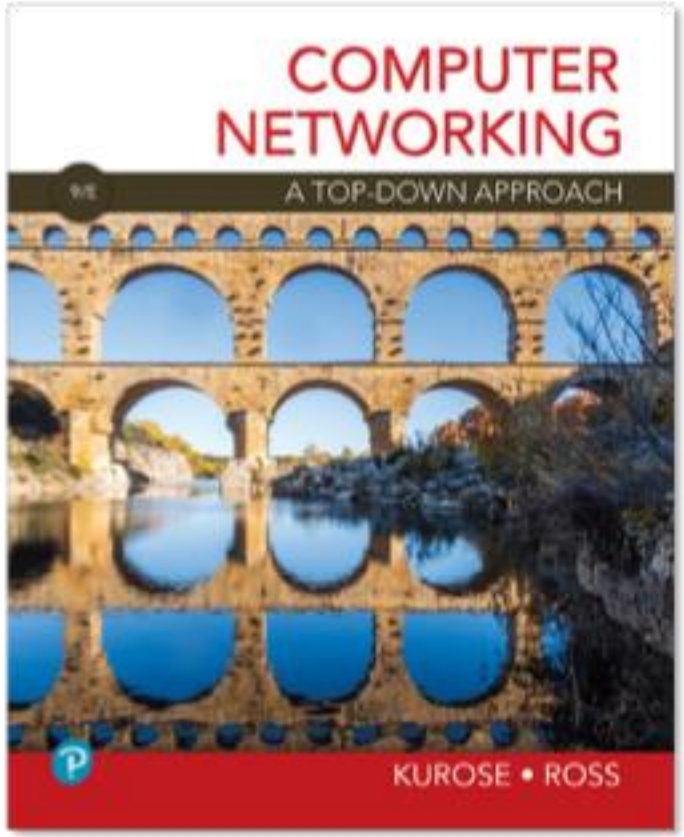
Chapter 5
Network Layer: Control Plane

第00讲-课程简介_哔哩哔哩_bilibili

https://www.bilibili.com/video/BV1Ut411d7RE/?p=1&vd_source=5fbc4920b6c246ce2d9126818c78b17b







Computer Networking: A Top-Down Approach

9th edition

Jim Kurose, Keith Ross

Pearson, 2025

Network layer control plane: our goals

- understand principles behind network control plane:
  - traditional routing algorithms
  - SDN controllers
  - network management, configuration
- instantiation, implementation in the Internet:
  - OSPF, BGP
  - OpenFlow, ODL and ONOS controllers
  - Internet Control Message Protocol: ICMP
  - SNMP, YANG/NETCONF

Network layer: "control plane" roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing: RIP, OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
  - SNMP
  - NETCONF/YANG

Network-layer functions

- forwarding: move packets from router's input to appropriate router output

data plane

- routing: determine route taken by packets from source to destination

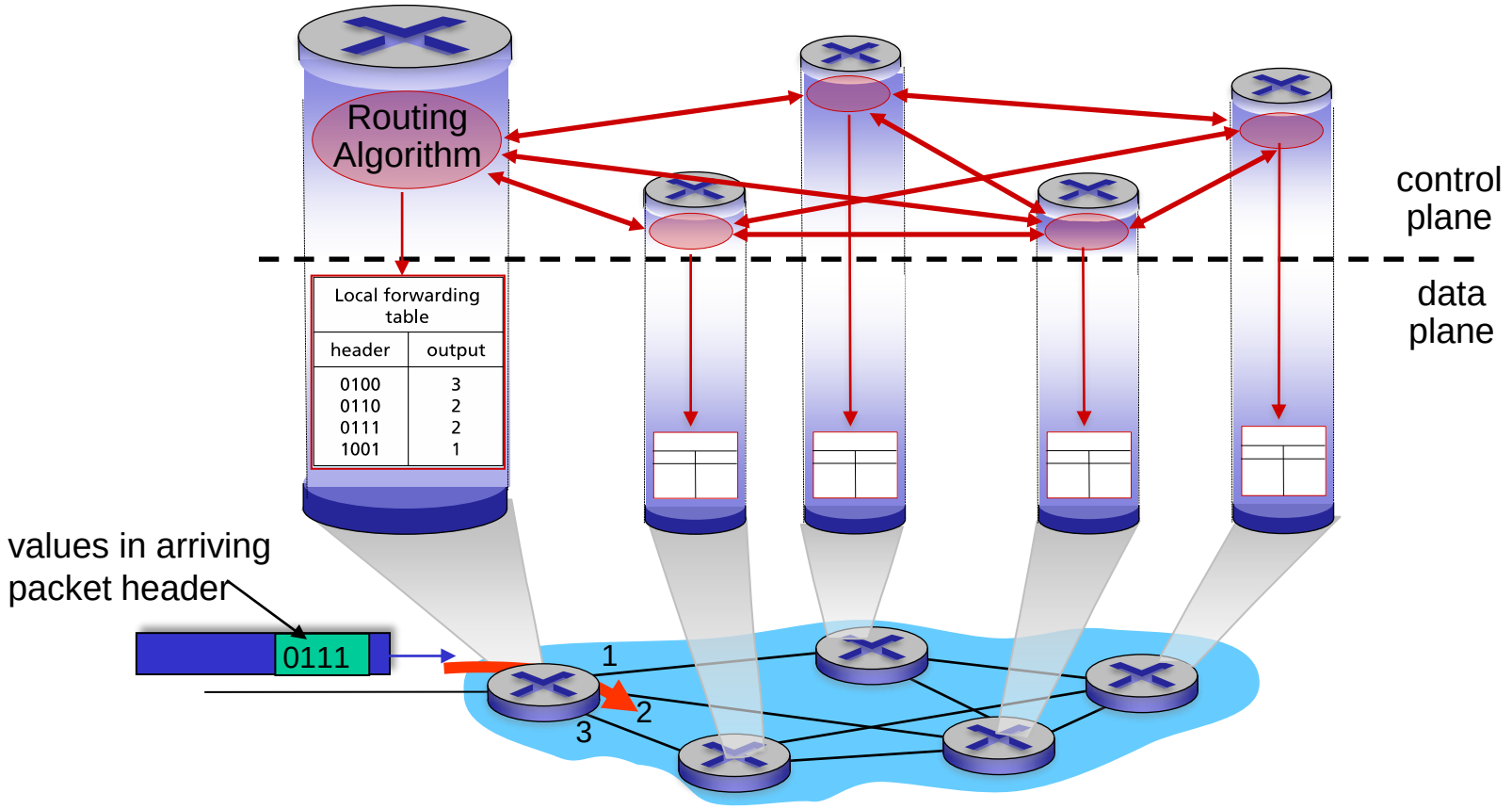
control plane

Two approaches to structuring network control plane:

- per-router control (traditional)
- logically centralized control (software defined networking)

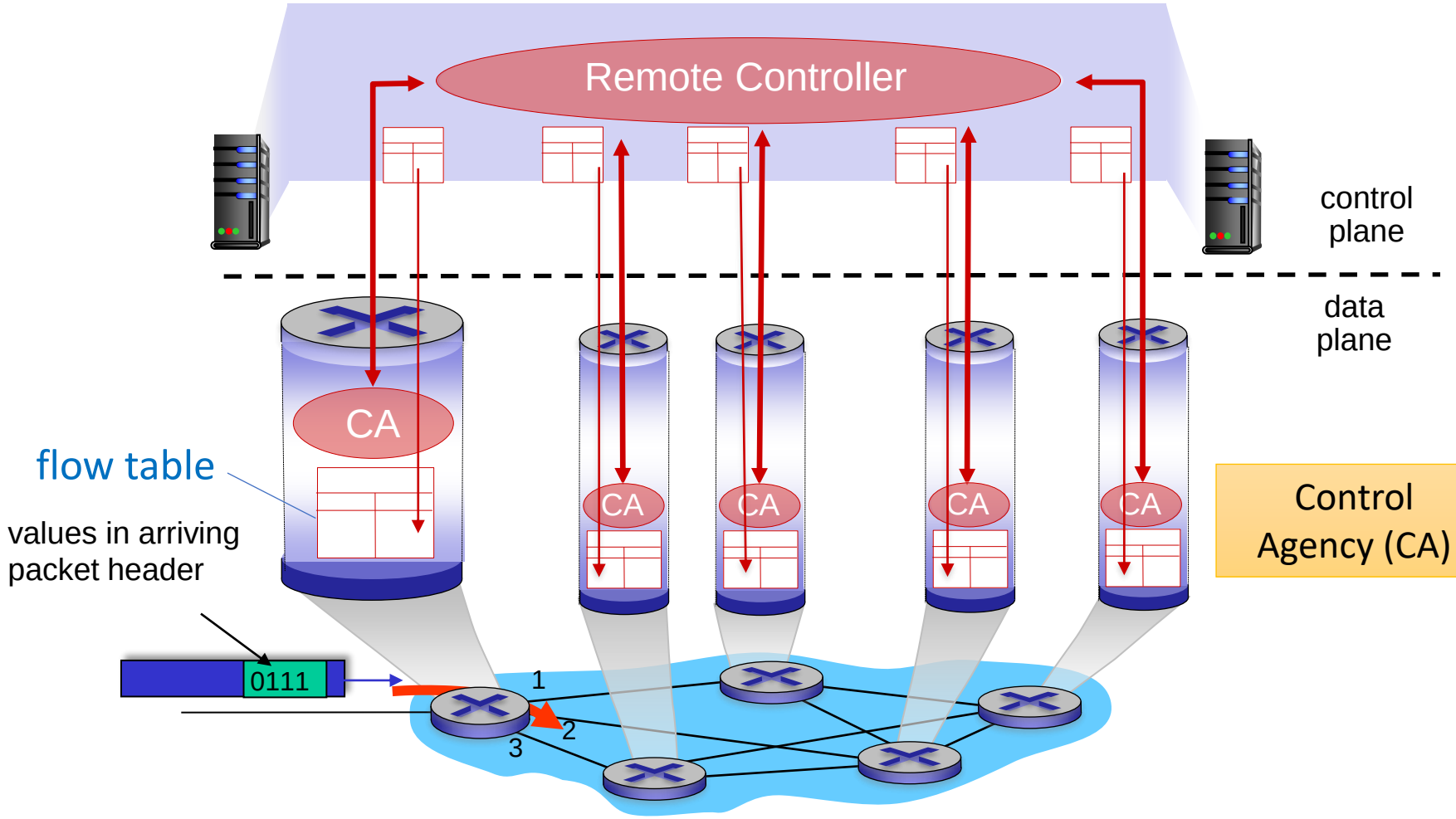
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane



Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers

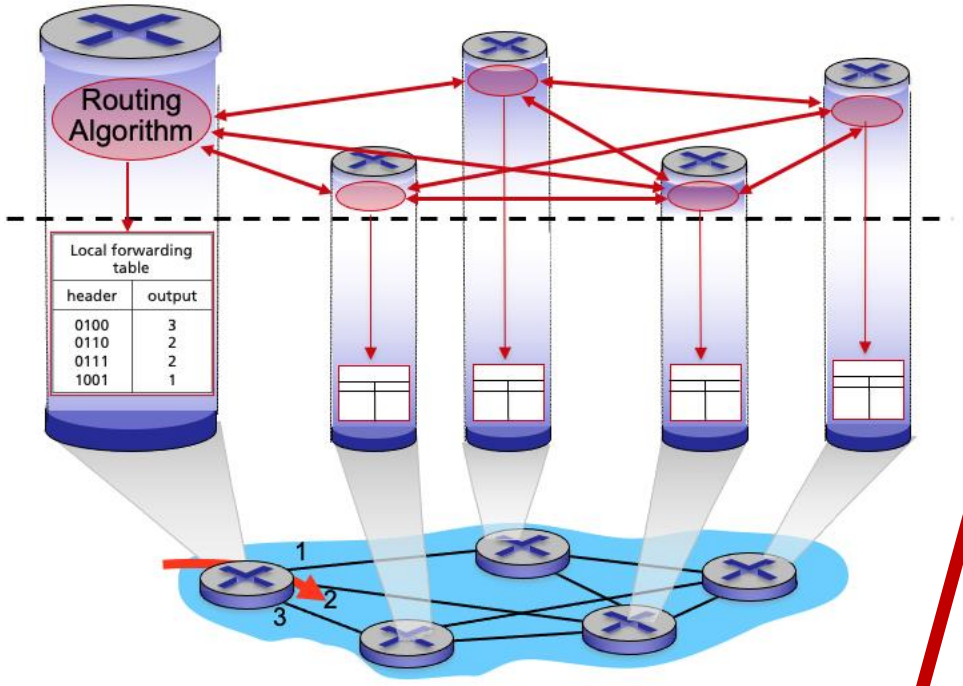


Control Agency (CA)

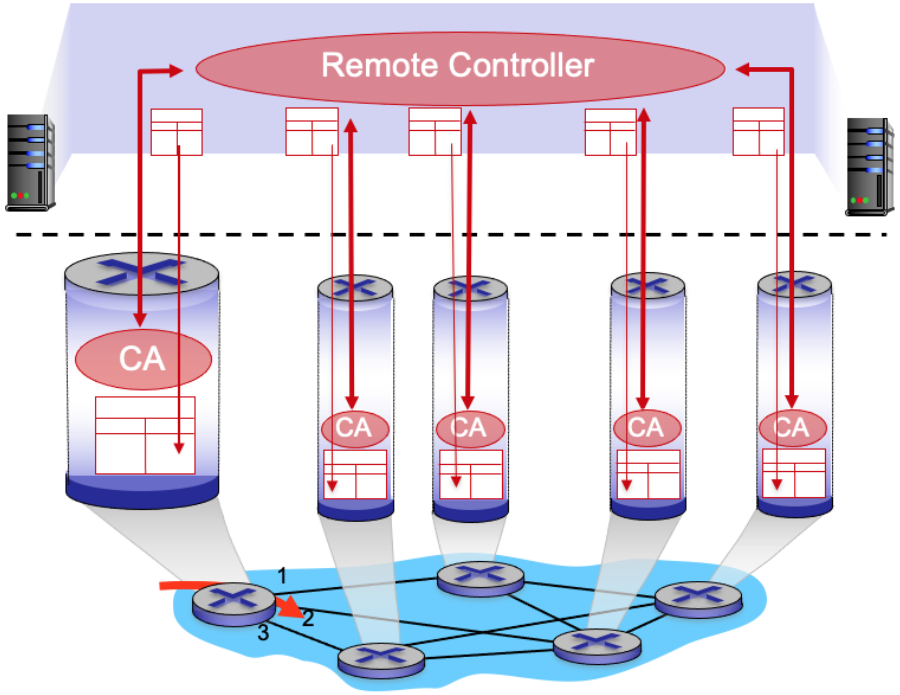
A key distinction between per-router control and logically centralized control:

- The CAs do not directly interact with each other nor do they actively take part in computing the forwarding table.
- Typically, the CA has minimum functionality (communicate with the controller)

Per-router control plane



SDN control plane



Network layer: "control plane" roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing: RIP, OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



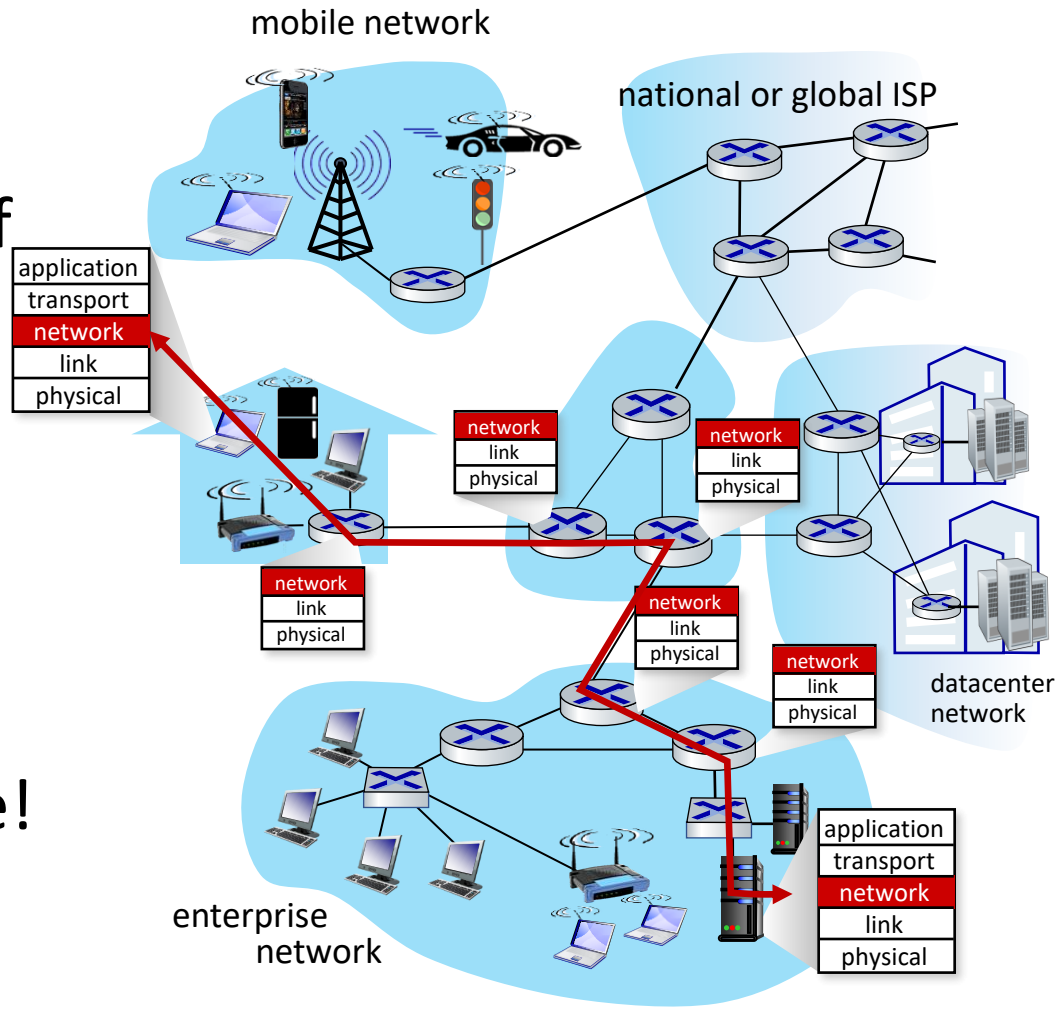
- network management, configuration
  - SNMP
  - NETCONF/YANG

Routing protocols

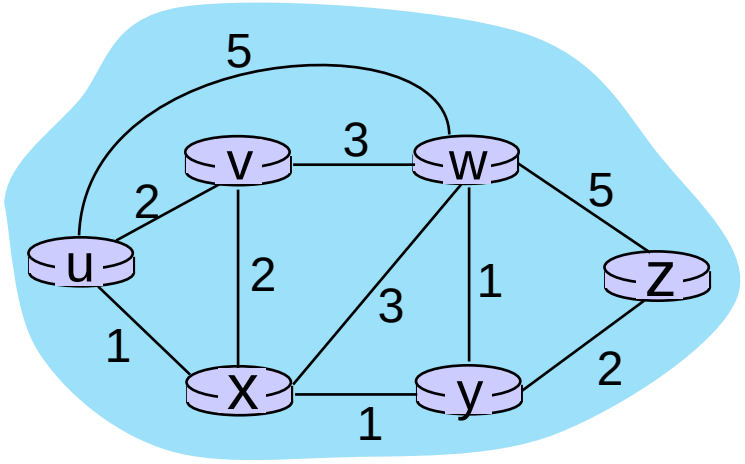
Routing protocol goal: determine "good" paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- path: sequence of routers packets traverse from given initial source host to final destination host
- "good": least "cost", "fastest", "least congested"
- **routing**: a "top-10" networking challenge!





Graph abstraction: link costs



$c_{a,b}$: cost of *direct* link connecting *a* and *b*

e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

cost defined by network operator:

- could always be 1,
- or inversely related to bandwidth
- or related to congestion,
- may reflect the physical length, the link speed, or the monetary cost associated with a link.

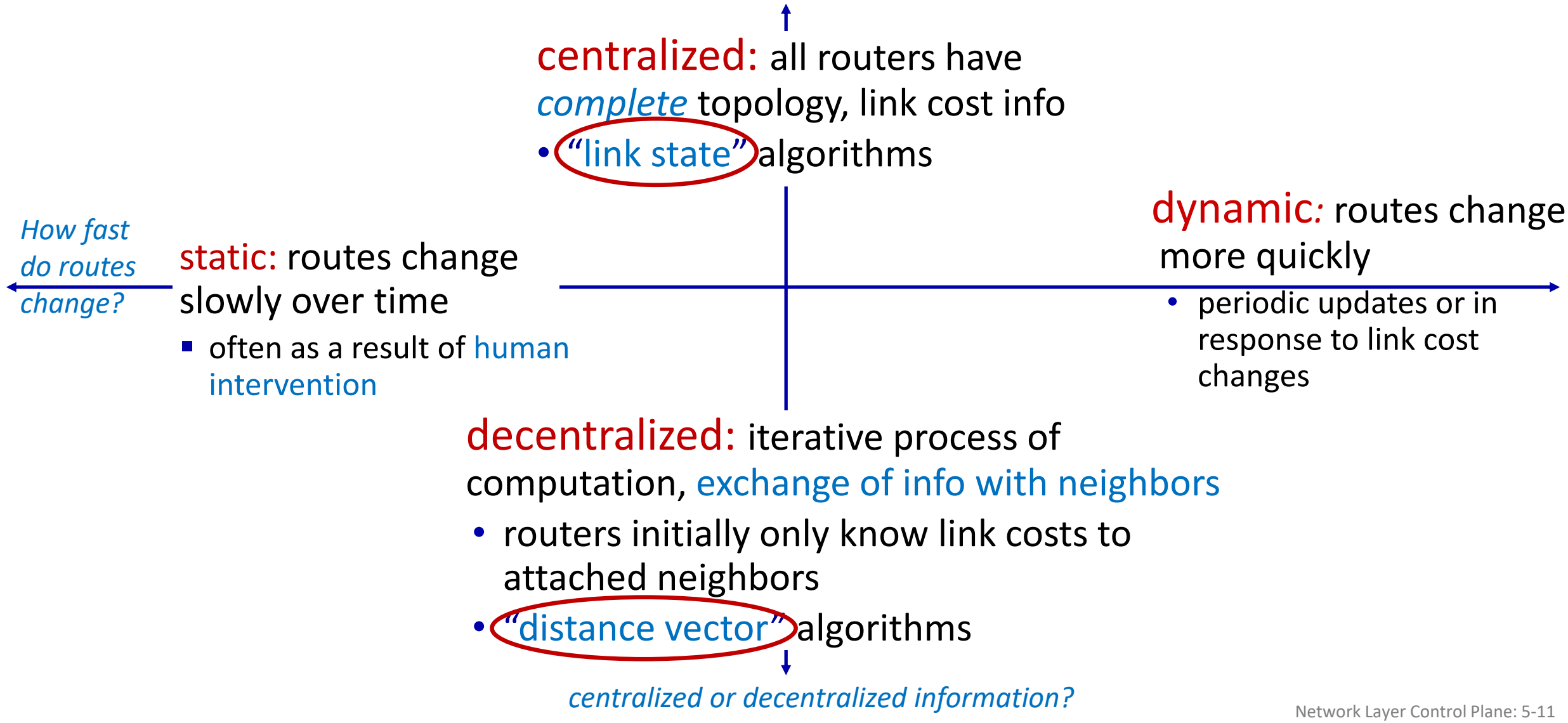
For our purposes, we'll simply take the edge costs as a given and won't worry about how they are determined.

graph: *G = (N,E)*

N: set of routers = { *u, v, w, x, y, z* }

E: set of links ={ *(u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z)* }

Routing algorithm classification



Routing Algorithm classification (1)

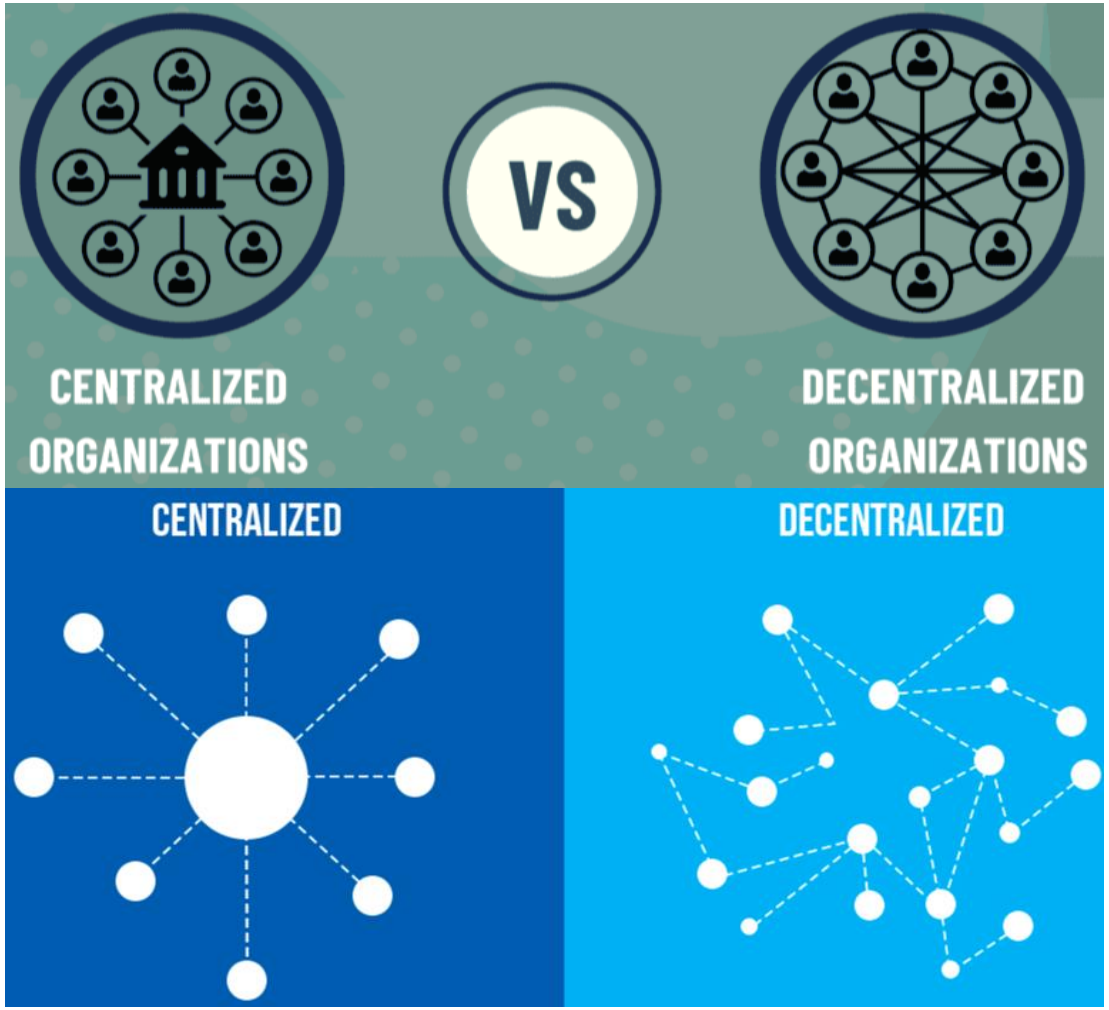
Centralized or decentralized information?

Centralized:

- all routers have complete topology, link cost info
- "link state" algorithms

Decentralized:

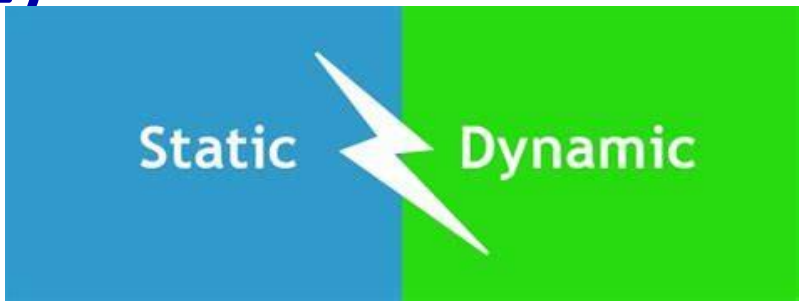
- router knows physically-connected neighbors, link costs to neighbors
- iterative process of computation, exchange of info with neighbors
- a node gradually calculates the least-cost path to a destination or set of destinations.
- "distance vector" algorithms



	CENTRALIZATION	DECENTRALIZATION
Ownership	 One to many	 Many (no controlling entity)
Decision-making	 Top-down	 Consensus
Decision time	 Fast	 Slow (requires majority)
Requirements	 Trust in decision-maker	 Distributed operation and control
Risks	 Corruption, exploitation	 Capture, concentrated ownership
Works well with…	 Competition	 Public goods, open protocols
Works badly with…	 Monopolies	 Individual products and services

Routing Algorithm classification (2)

Static or dynamic?



Static:

- routes change slowly over time
  - often as a result of human intervention

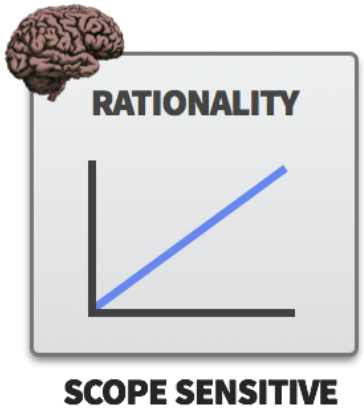
Dynamic:

- routes change more quickly
  - periodic update
  - in direct response to topology or link cost changes
- more responsive to network changes
  - more susceptible to problems
  - such as routing loops and route oscillation.



Routing Algorithm classification (3)

Load-sensitive or load-insensitive?







- Load-sensitive:
  - Link costs vary dynamically to reflect congestion
  - A routing algorithm will tend to choose routes around the link with a high cost
    - e.g., early ARPAnet routing encountered many difficulties
- Load-insensitive:
  - today's Internet routing algorithms
  - a link's cost does not explicitly reflect its current (or recent past) level of congestion
    - e.g., RIP, OSPF, BGP

The resulting problem of inefficient traffic distribution is addressed through **overlay technologies** like Traffic Engineering (TE), SDN, and CDNs, which add a layer of **load-aware control** on top of the stable, load-insensitive underlay.

Network layer: "control plane" roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing: RIP, OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
  - SNMP
  - NETCONF/YANG

Dijkstra's link-state routing algorithm

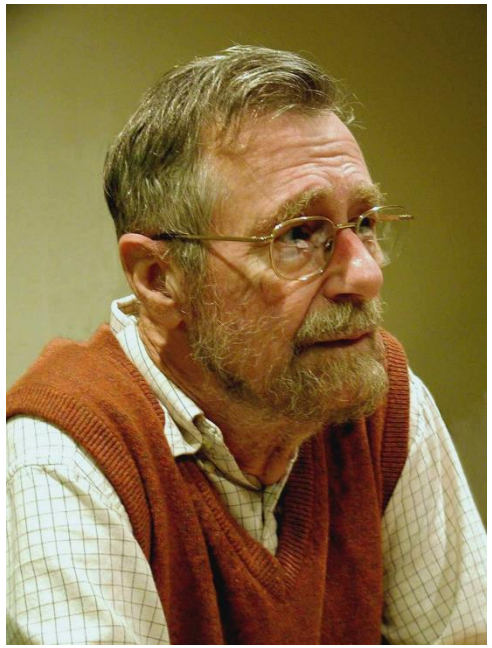
- centralized: network topology, link costs known to *all* nodes
  - accomplished via "link state broadcast"
  - all nodes have same info
- computes least cost paths from one node ("source") to all other nodes
  - gives *forwarding table* for that node
- iterative: after *k* iterations, know least cost path to *k* destinations

Understanding Routing

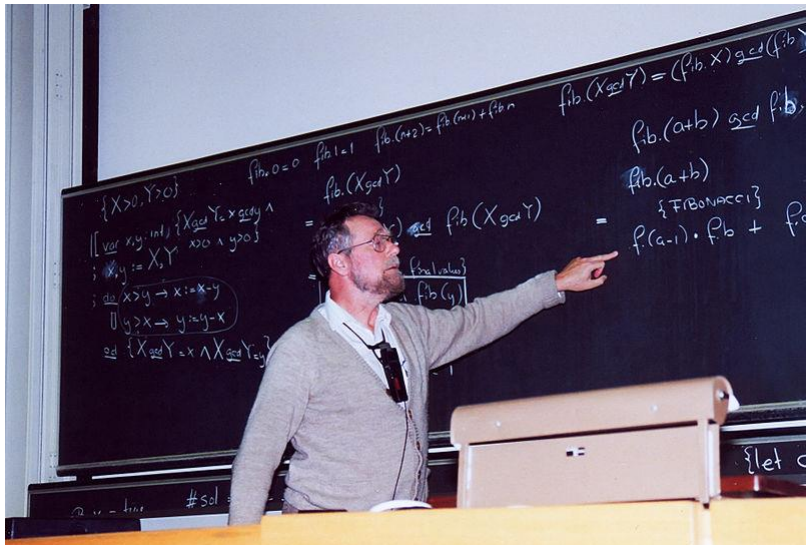
notation

- $c_{x,y}$: direct link cost from node *x* to *y*; = ∞ if not direct neighbors
- *D(v): current* estimate of cost of least-cost-path from source to destination *v*
- *p(v):* predecessor node along path from source to *v*
- *N':* set of nodes whose least-cost-path *definitively* known

Dijkstra biography



- Edsger Wybe Dijkstra (May 11, 1930 – August 6, 2002) was a Dutch computer scientist.
- received the 1972 Turing Award for fundamental contributions to developing programming languages
- was the Schlumberger Centennial Chair of Computer Sciences at The University of Texas at Austin from 1984 until 2000.



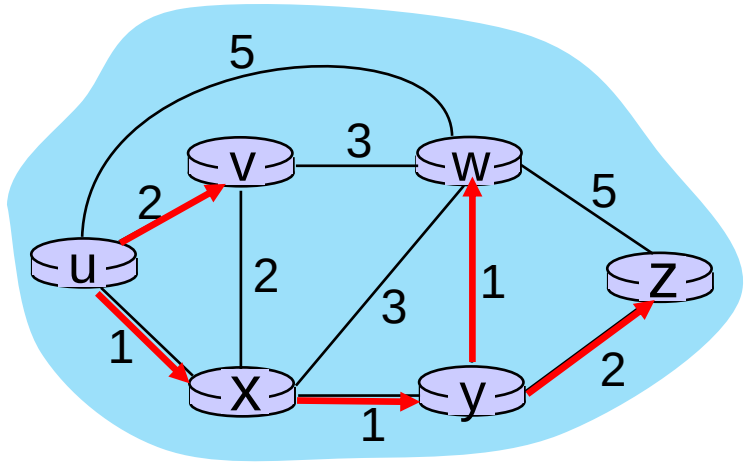
Dijkstra at the blackboard during a conference at ETH Zurich in 1994

Dijkstra's link-state routing algorithm

```
1  Initialization:
2    N' = {u}                    /* compute least cost path from u to all other nodes */
3    for all nodes v
4      if v adjacent to u        /* u initially knows direct-path-cost only to direct neighbors */
5        then D(v) = c_u,v       /* but may not be minimum cost!                                 */
6      else D(v) = ∞
7
8  Loop
9      find w not in N' such that D(w) is a minimum
10     add w to N'
11     update D(v) for all v adjacent to w and not in N' :
12         D(v) = min ( D(v),  D(w) + c_w,v )
13     /* new least-path-cost to v is either old least-cost-path to v or known
14     least-cost-path to w plus direct-cost from w to v */
15 until all nodes in N'
```

Dijkstra's algorithm: an example

Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

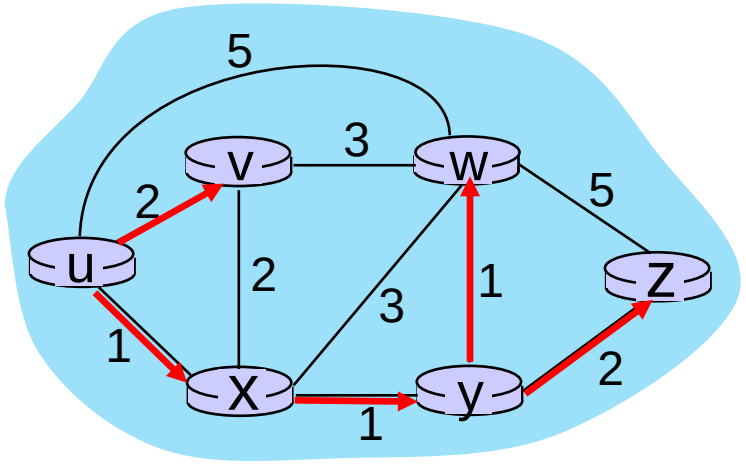


Initialization (step 0): For all a: if *a* adjacent to then $D(a) = c_{u,a}$

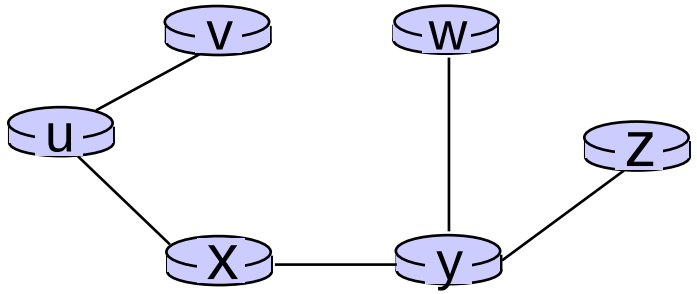
find *a* not in *N'* such that *D(a)* is a minimum
add *a* to *N'*
update *D(b)* for all *b* adjacent to *a* and not in *N'* :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



resulting forwarding table in u:

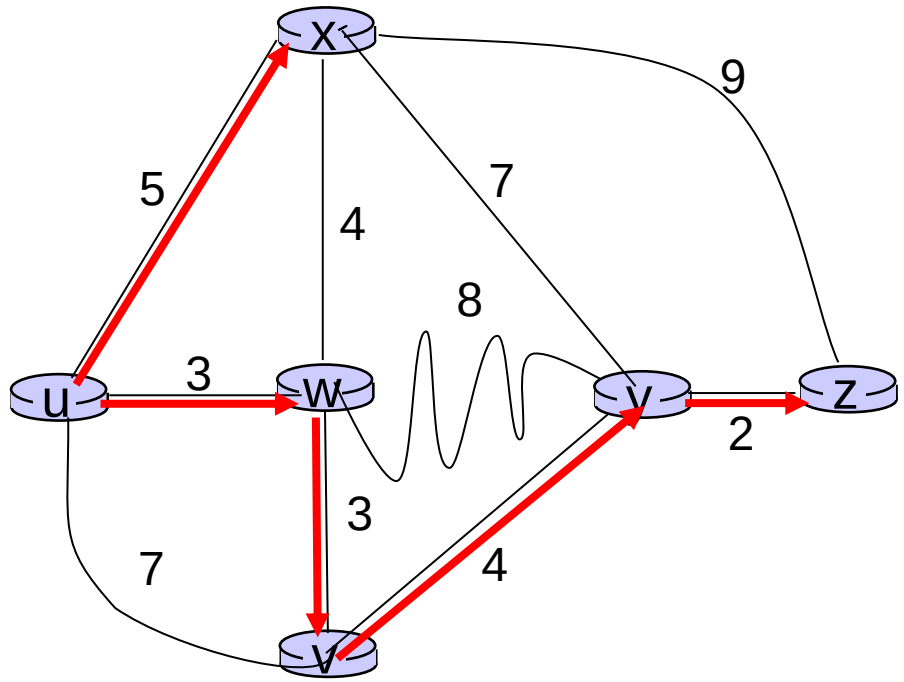
destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

route from *u* to *v* directly

route from u to all other destinations via *x*

Dijkstra's algorithm: another example

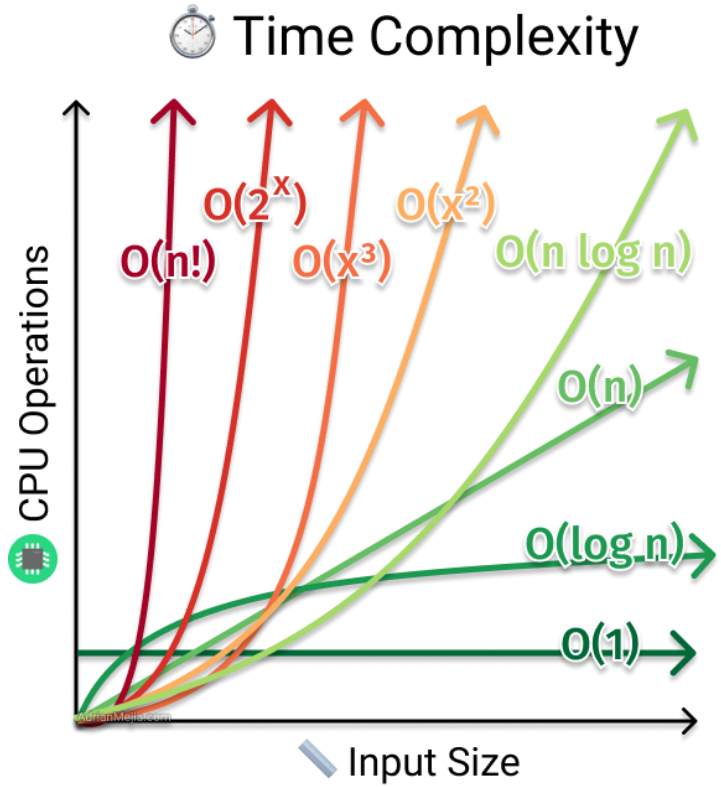
Step	N'	v D(v), p(v)	w D(w), p(w)	x D(x), p(x)	y D(y), p(y)	z D(z), p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					



notes:

- construct least-cost-path tree by tracing predecessor nodes
- ties can exist (can be broken arbitrarily)

Dijkstra's algorithm: discussion



algorithm complexity: *n* nodes

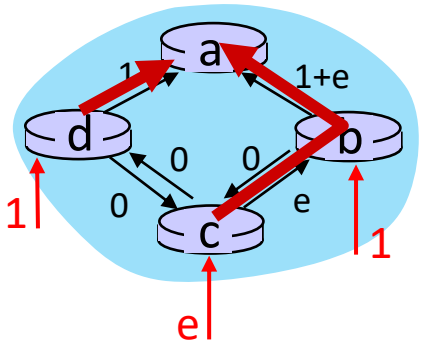
- each of *n* iteration: need to check all nodes, *w*, not in *N'*
- $n(n+1)/2$ comparisons: $O(n^2)$ complexity
- more efficient implementations possible: $O(n\log n)$

message complexity:

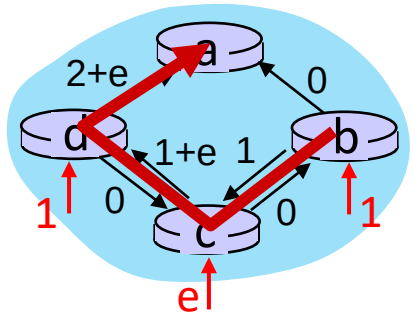
- each router must *broadcast* its link state information to other *n* routers
- efficient (and interesting!) broadcast algorithms: $O(n)$ link crossings to disseminate a broadcast message from one source
- each router's message crosses $O(n)$ links: overall message complexity: $O(n^2)$

Dijkstra's algorithm: oscillations possible

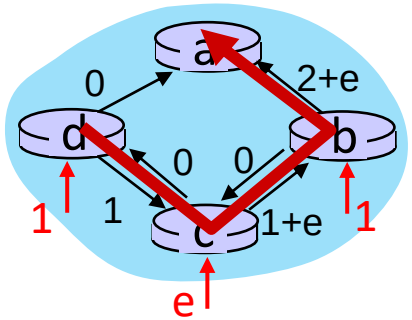
- when link costs depend on traffic volume, route oscillations possible
- sample scenario:
  - routing to destination a, traffic entering at d, c, b with rates 1, e (<1), 1
  - link costs are directional, and volume-dependent



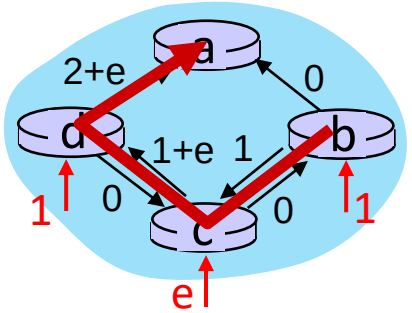
initially



given these costs,
find new routing….
resulting in new costs



given these costs,
find new routing….
resulting in new costs



given these costs,
find new routing….
resulting in new costs

Network layer: "control plane" roadmap

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing: RIP, OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
  - SNMP
  - NETCONF/YANG

Distance vector algorithm

Based on *Bellman-Ford* (BF) equation (dynamic programming):

Bellman-Ford equation

Let $D_x(y)$: cost of least-cost path from x to y.
Then:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

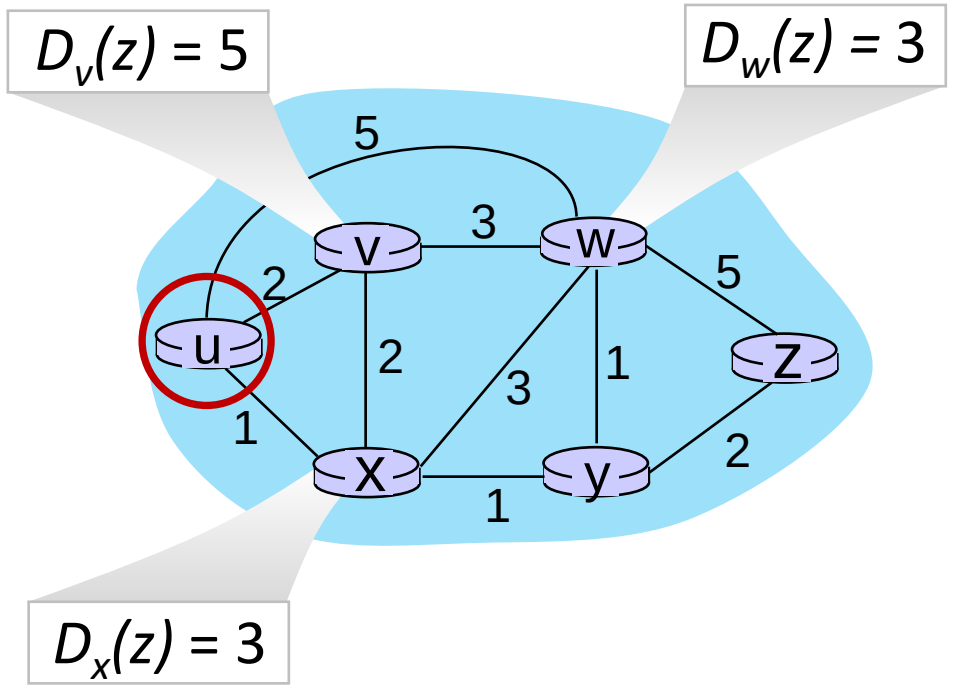
min taken over all neighbors v of x

direct cost of link from x to v

v's estimated least-cost-path cost to y

Bellman-Ford Example

Suppose that *u*'s neighboring nodes, *x,v,w,* know that for destination *z*:



Bellman-Ford equation says:

$$D_u(z) = \min \{ c_{u,v} + D_v(z), \; c_{u,x} + D_x(z), \; c_{u,w} + D_w(z) \}$$
$$= \min \{2 + 5, \; 1 + 3, \; 5 + 3\} = 4$$

node achieving minimum (x) is next hop on estimated least-cost path to destination (z)

Distance vector algorithm

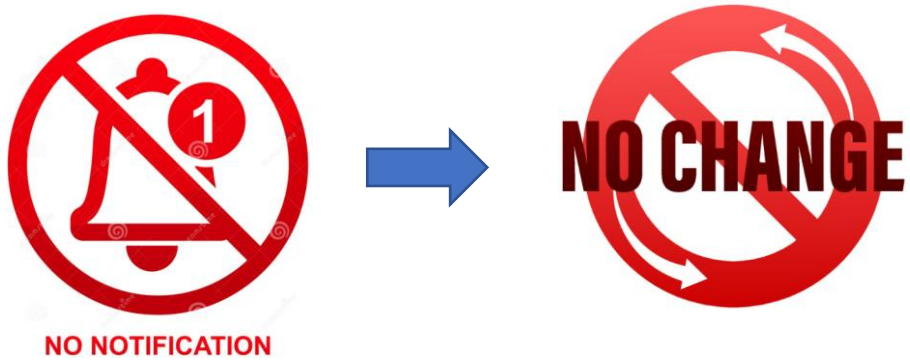
key idea:

- from time-to-time, each node sends its own distance vector estimate to neighbors
- when *x* receives new DV estimate from any neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow min_v\{c_{x,v} + D_v(y)\} \text{ for each node } y \in N$$

- under minor, natural conditions, the estimate $D_x(y)$ *converge to the actual least cost* $d_x(y)$

Distance vector algorithm:



each node:

wait for (change in local link cost or msg from neighbor)

recompute DV estimates using DV received from neighbor

if DV to any destination has changed, *notify* neighbors

iterative, asynchronous: each local iteration caused by:

- local link cost change
- DV update message from neighbor

distributed, self-stopping: each node notifies neighbors *only* when its DV changes

- neighbors then notify their neighbors – *only if necessary*
- no notification received, no actions taken!

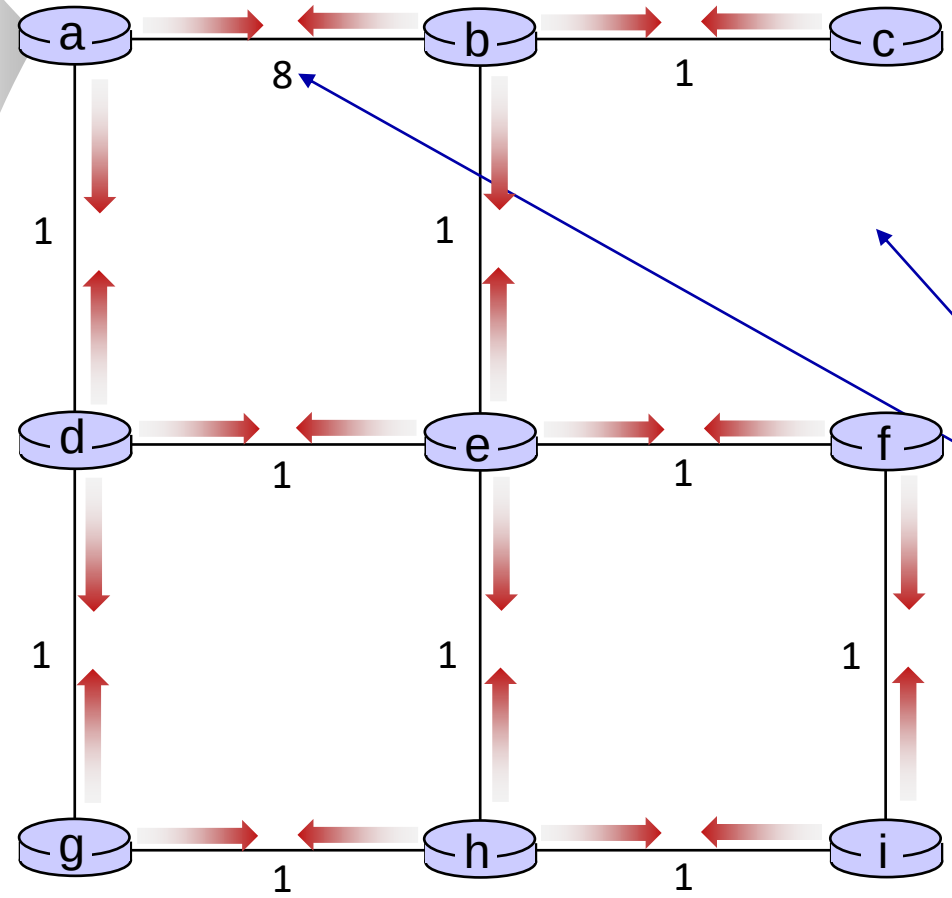
Distance vector: example



t=0

- All nodes have distance estimates to nearest neighbors (only)
- All nodes send their local distance vector to their neighbors

DV in a:
$D_a(a) = 0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$



A few asymmetries:
- missing link
- larger cost

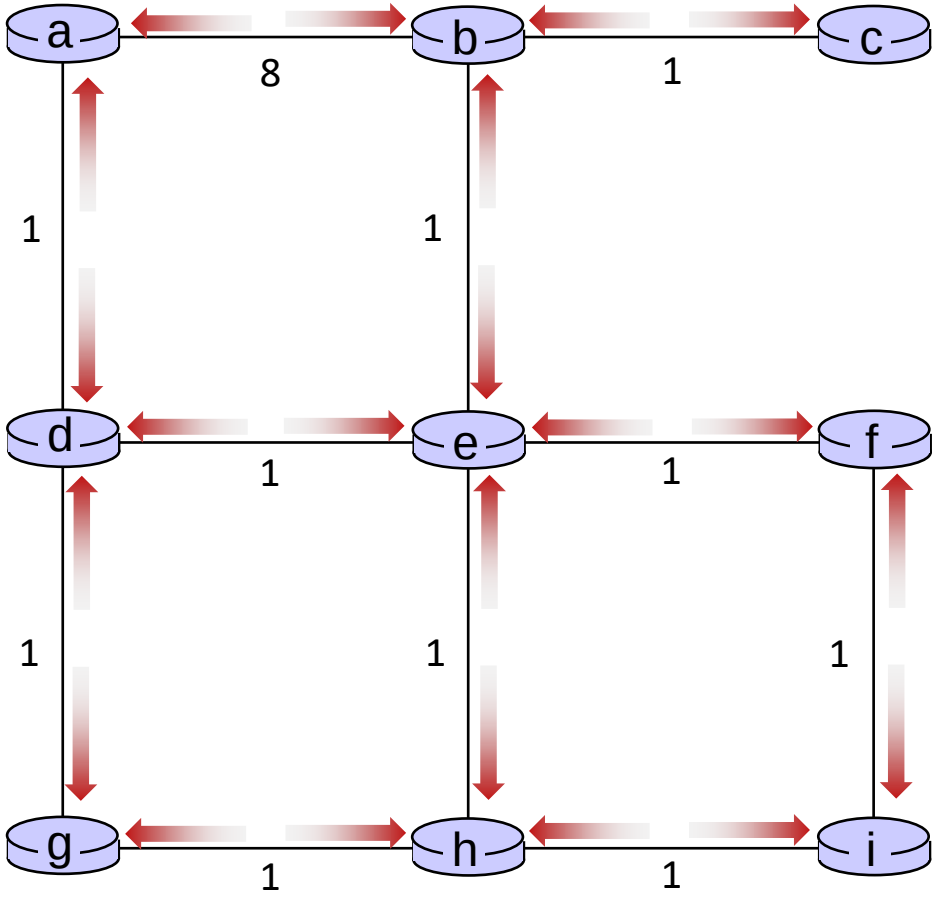
Distance vector example: iteration



t=1

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



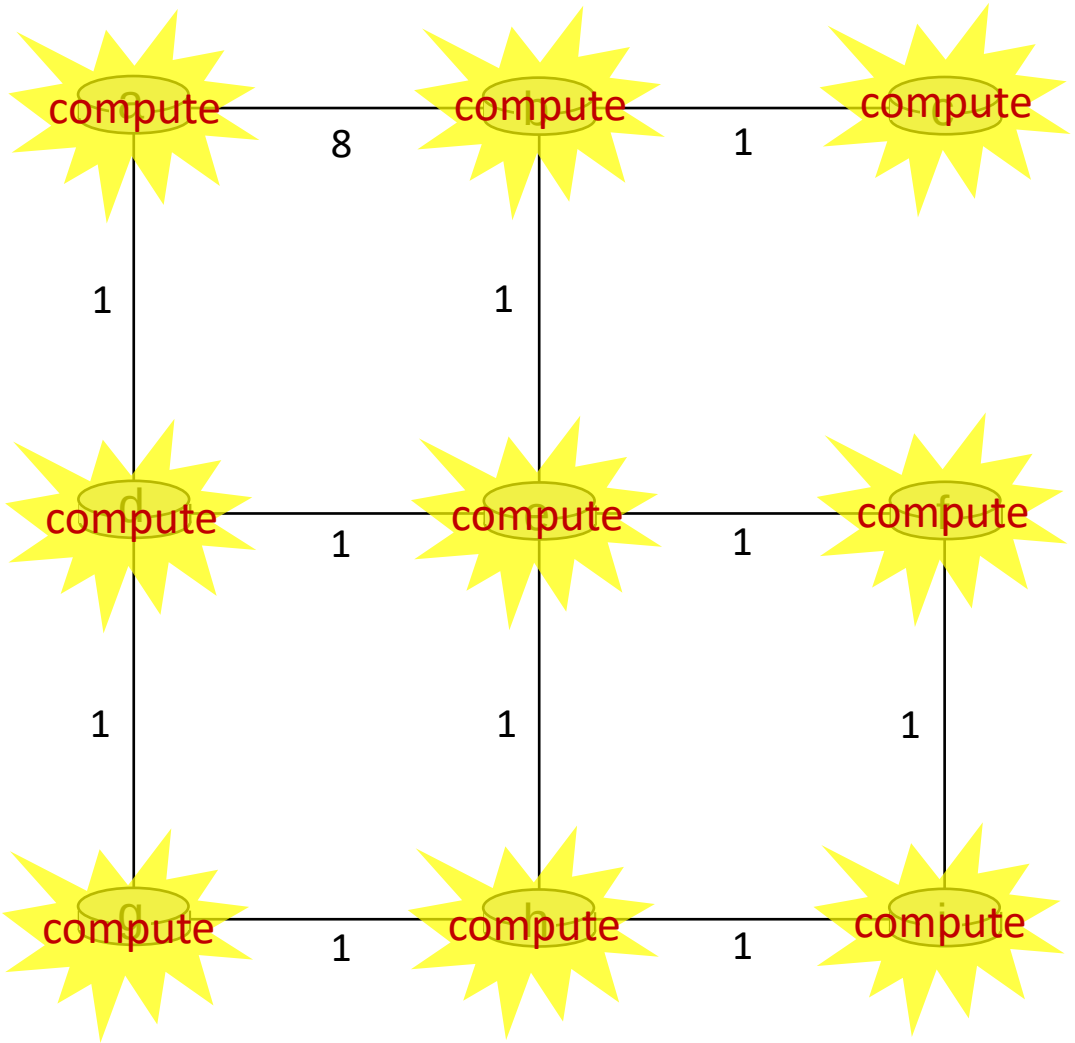
Distance vector example: iteration



t=1

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



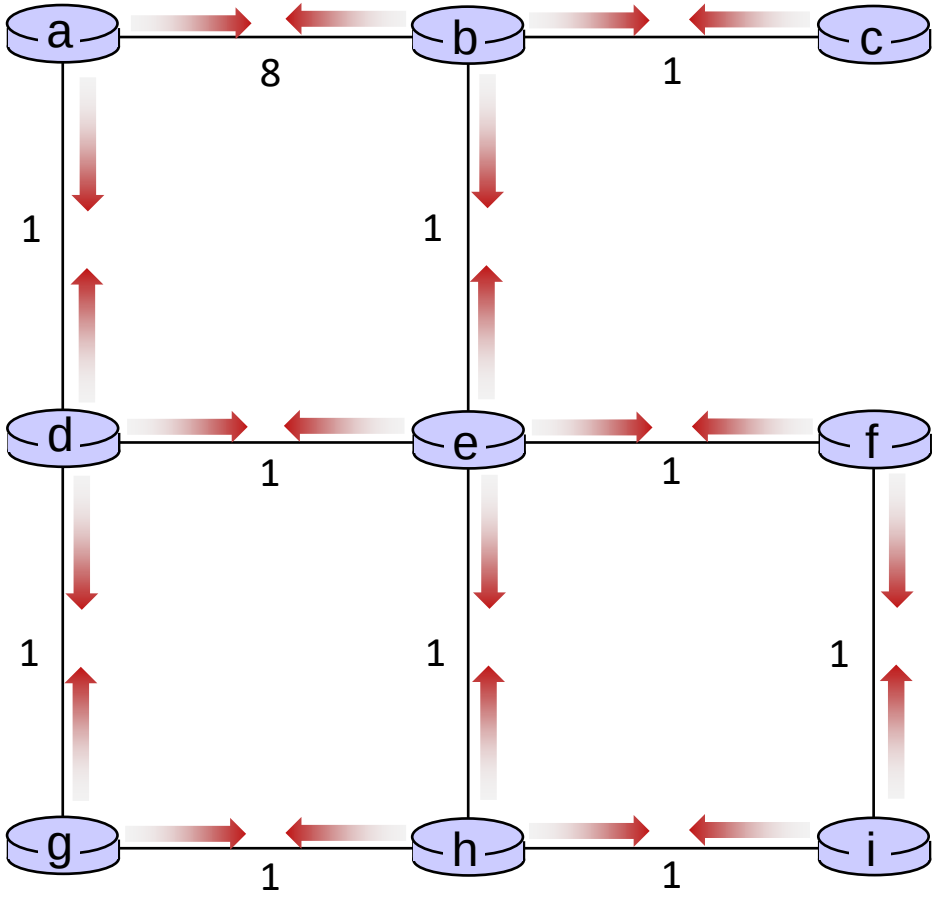
Distance vector example: iteration



t=1

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



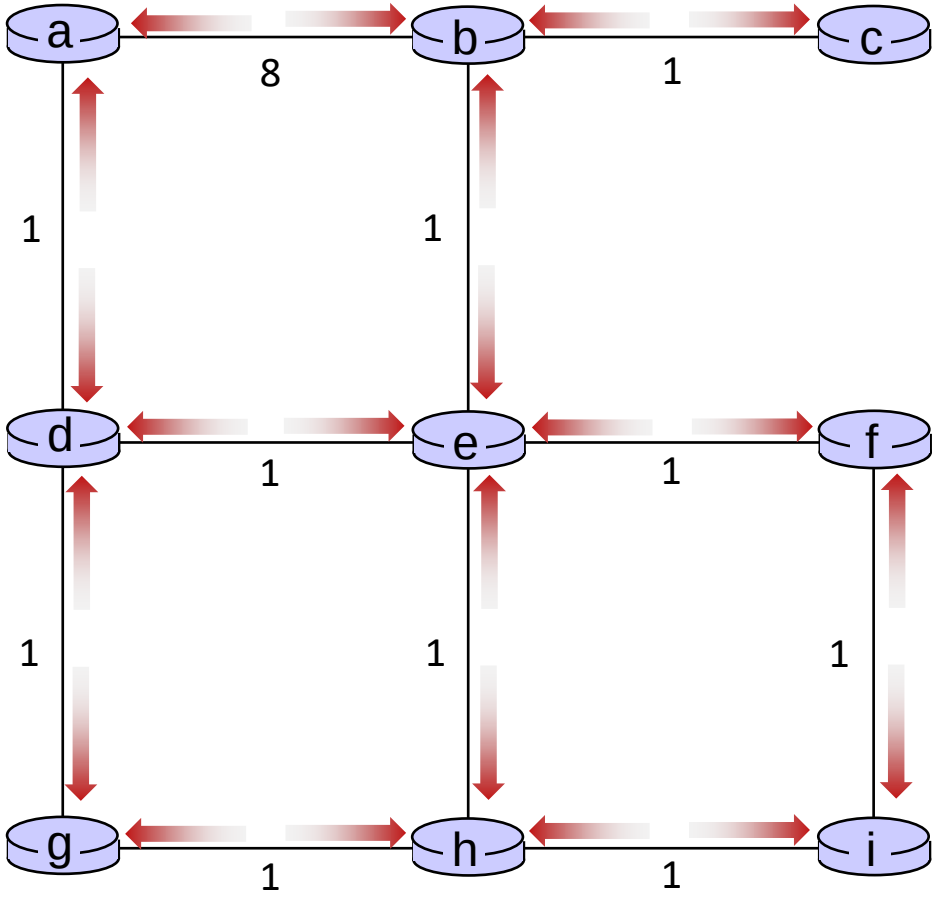
Distance vector example: iteration



t=2

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



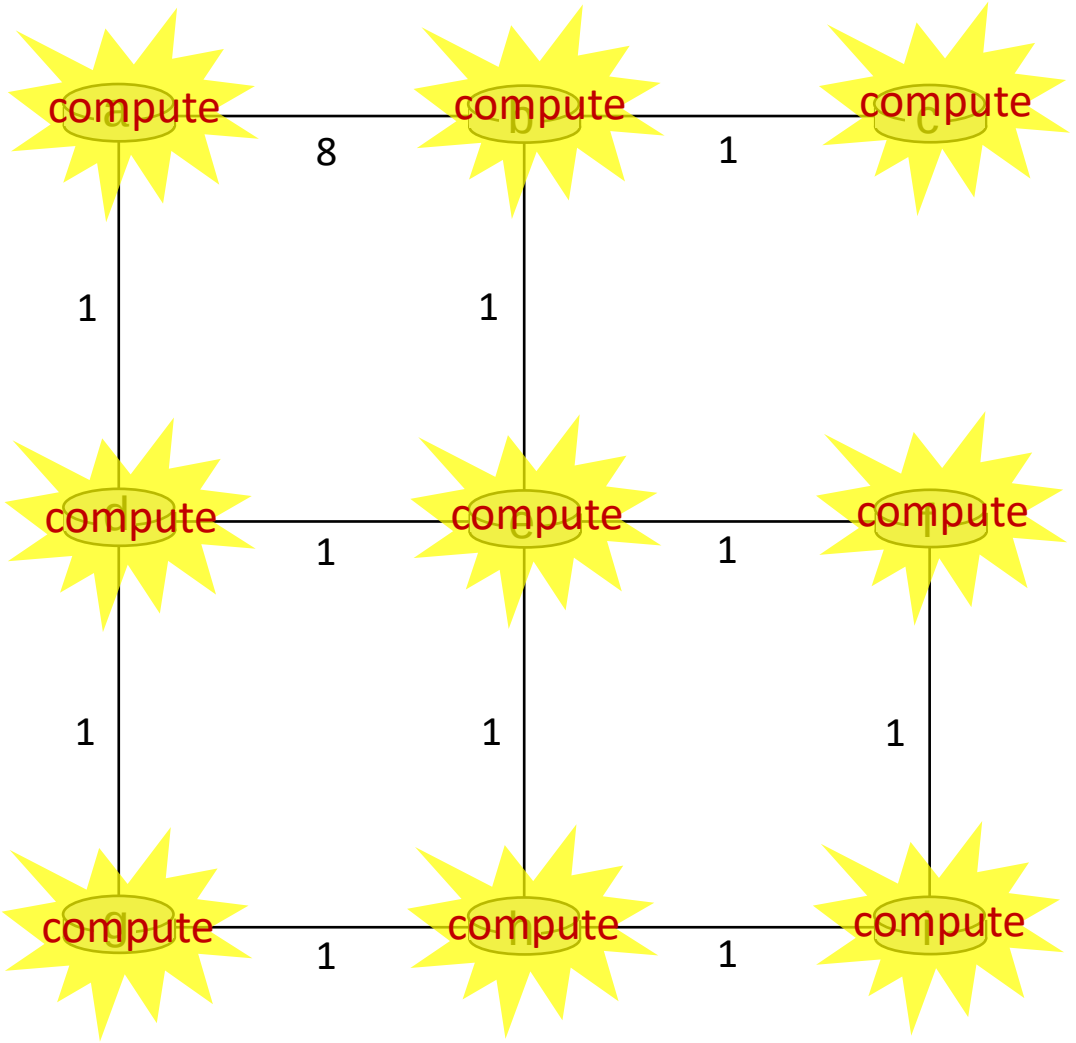
Distance vector example: iteration



t=2

All nodes:

- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



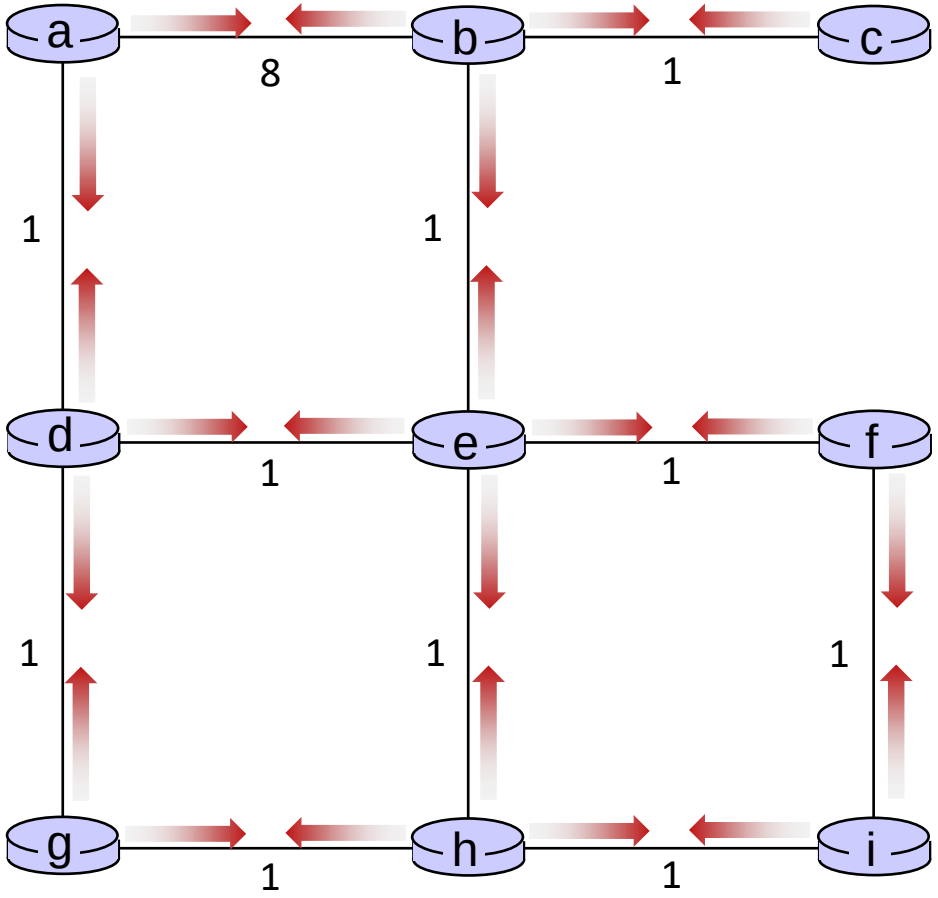
Distance vector example: iteration



t=2

All nodes:

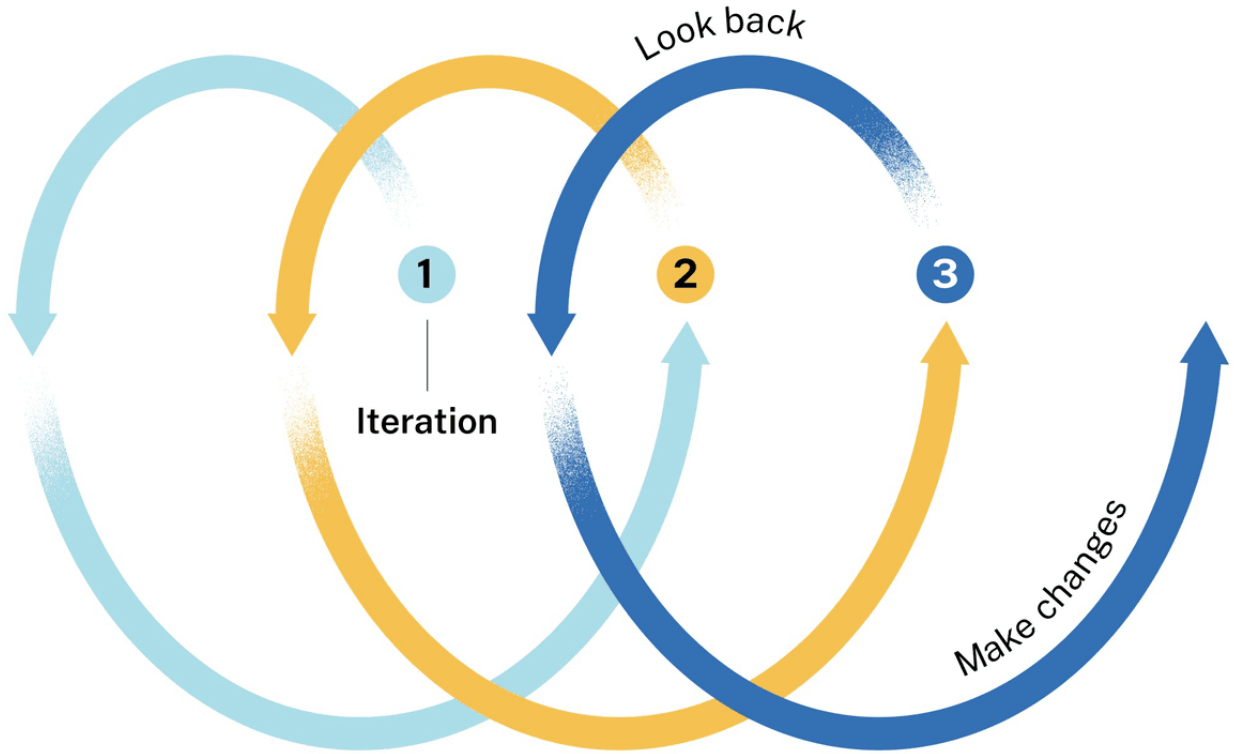
- receive distance vectors from neighbors
- compute their new local distance vector
- send their new local distance vector to neighbors



Distance vector example: iteration

…. and so on

Let’s next take a look at the iterative *computations* at nodes

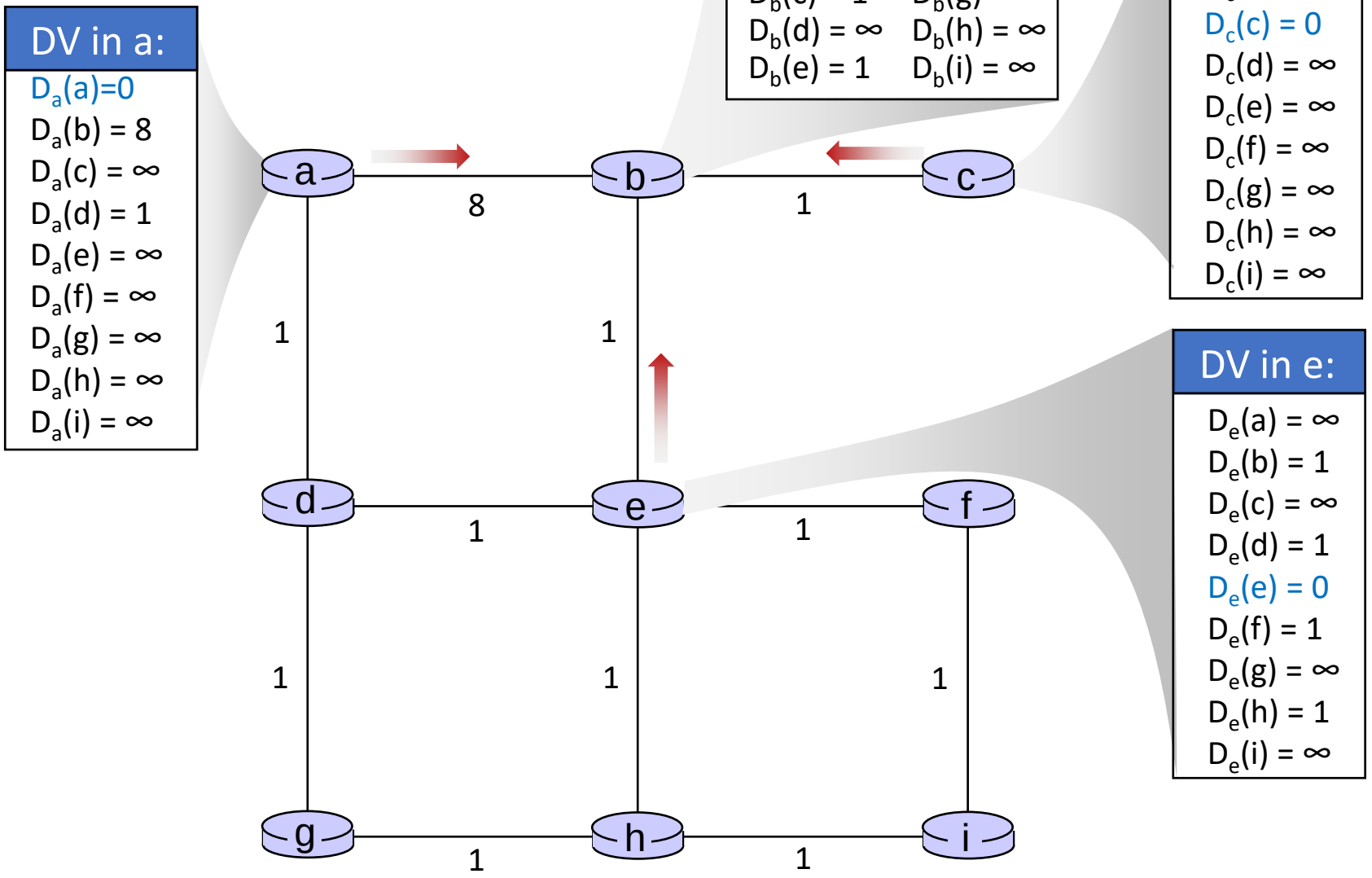


Distance vector example: computation



t=1

- b receives DVs from a, c, e

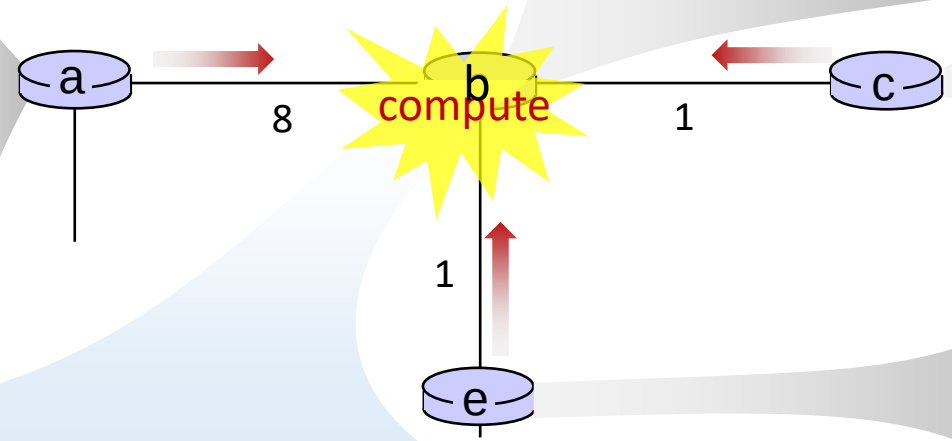


Distance vector example: c



t=1

- b receives DVs from a, c, e, computes:



DV in a:

$D_a(a)=0$
$D_a(b) = 8$
$D_a(c) = \infty$
$D_a(d) = 1$
$D_a(e) = \infty$
$D_a(f) = \infty$
$D_a(g) = \infty$
$D_a(h) = \infty$
$D_a(i) = \infty$

DV in b:

$D_b(a) = 8$ $D_b(f) = \infty$
$D_b(c) = 1$ $D_b(g) = \infty$
$D_b(d) = \infty$ $D_b(h) = \infty$
$D_b(e) = 1$ $D_b(i) = \infty$

DV in c:

$D_c(a) = \infty$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = \infty$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$

DV in e:

$D_e(a) = \infty$
$D_e(b) = 1$
$D_e(c) = \infty$
$D_e(d) = 1$
$D_e(e) = 0$
$D_e(f) = 1$
$D_e(g) = \infty$
$D_e(h) = 1$
$D_e(i) = \infty$

$D_b(a) = \min\{c_{b,a}+D_a(a), c_{b,c}+D_c(a), c_{b,e}+D_e(a)\} = \min\{8,\infty,\infty\} = 8$

$D_b(c) = \min\{c_{b,a}+D_a(c), c_{b,c}+D_c(c), c_{b,e}+D_e(c)\} = \min\{\infty,1,\infty\} = 1$

$D_b(d) = \min\{c_{b,a}+D_a(d), c_{b,c}+D_c(d), c_{b,e}+D_e(d)\} = \min\{9,\infty,2\} = 2$

$D_b(e) = \min\{c_{b,a}+D_a(e), c_{b,c}+D_c(e), c_{b,e}+D_e(e)\} = \min\{\infty,\infty,1\} = 1$

$D_b(f) = \min\{c_{b,a}+D_a(f), c_{b,c}+D_c(f), c_{b,e}+D_e(f)\} = \min\{\infty,\infty,2\} = 2$

$D_b(g) = \min\{c_{b,a}+D_a(g), c_{b,c}+D_c(g), c_{b,e}+D_e(g)\} = \min\{\infty, \infty, \infty\} = \infty$

$D_b(h) = \min\{c_{b,a}+D_a(h), c_{b,c}+D_c(h), c_{b,e}+D_e(h)\} = \min\{\infty, \infty, 2\} = 2$

$D_b(i) = \min\{c_{b,a}+D_a(i), c_{b,c}+D_c(i), c_{b,e}+D_e(i)\} = \min\{\infty, \infty, \infty\} = \infty$

DV in b:

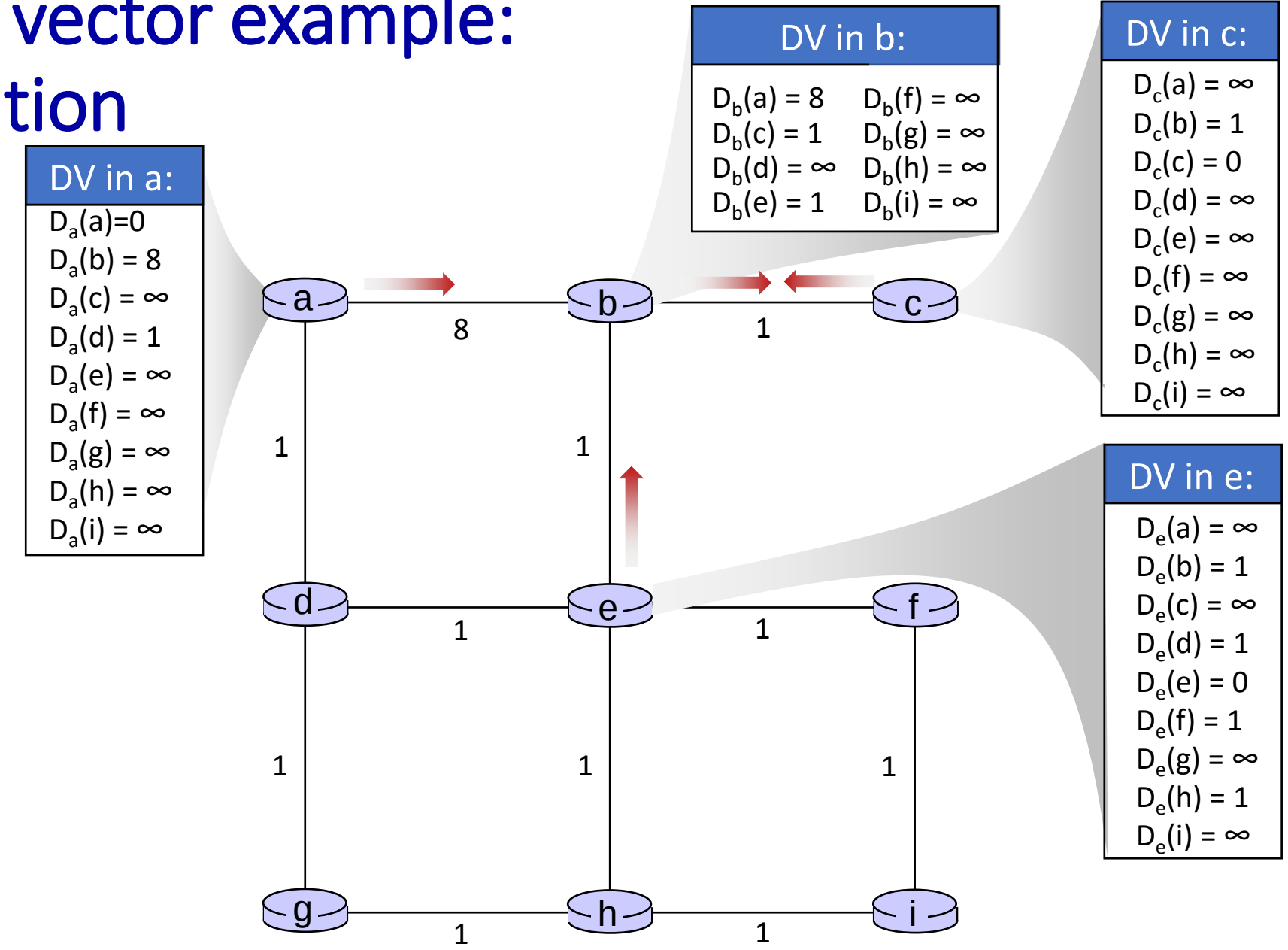
$D_b(a) = 8$ $D_b(f) = 2$
$D_b(c) = 1$ $D_b(g) = \infty$
$D_b(d) = 2$ $D_b(h) = 2$
$D_b(e) = 1$ $D_b(i) = \infty$

Distance vector example: computation



t=1

- c receives DVs from b

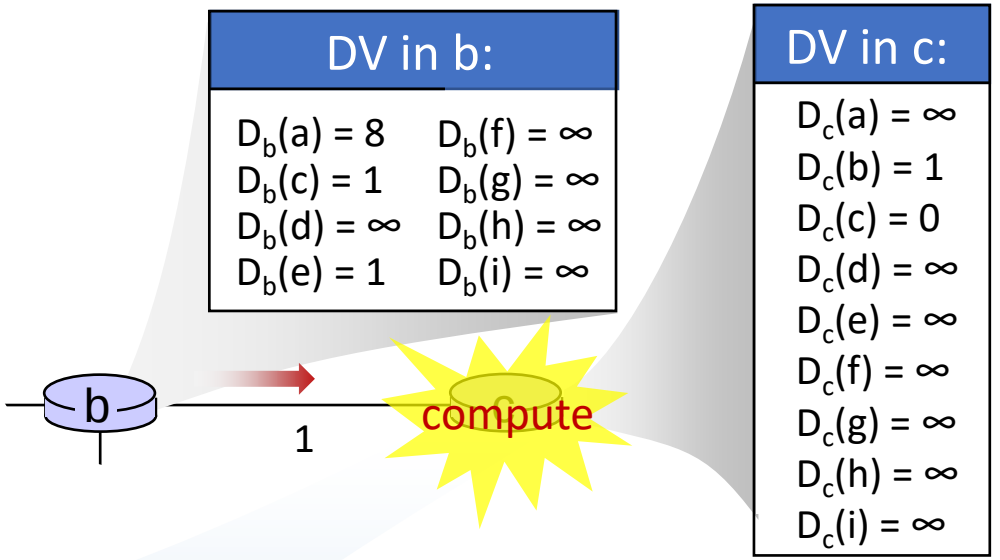


Distance vector example: computation



t=1

- c receives DVs from b computes:



$D_c(a) = \min\{c_{c,b}+D_b(a)\} = 1 + 8 = 9$

$D_c(b) = \min\{c_{c,b}+D_b(b)\} = 1 + 0 = 1$

$D_c(d) = \min\{c_{c,b}+D_b(d)\} = 1+ \infty = \infty$

$D_c(e) = \min\{c_{c,b}+D_b(e)\} = 1 + 1 = 2$

$D_c(f) = \min\{c_{c,b}+D_b(f)\} = 1+ \infty = \infty$

$D_c(g) = \min\{c_{c,b}+D_b(g)\} = 1+ \infty = \infty$

$D_c(h) = \min\{c_{bc,b}+D_b(h)\} = 1+ \infty = \infty$

$D_c(i) = \min\{c_{c,b}+D_b(i)\} = 1+ \infty = \infty$

DV in c:

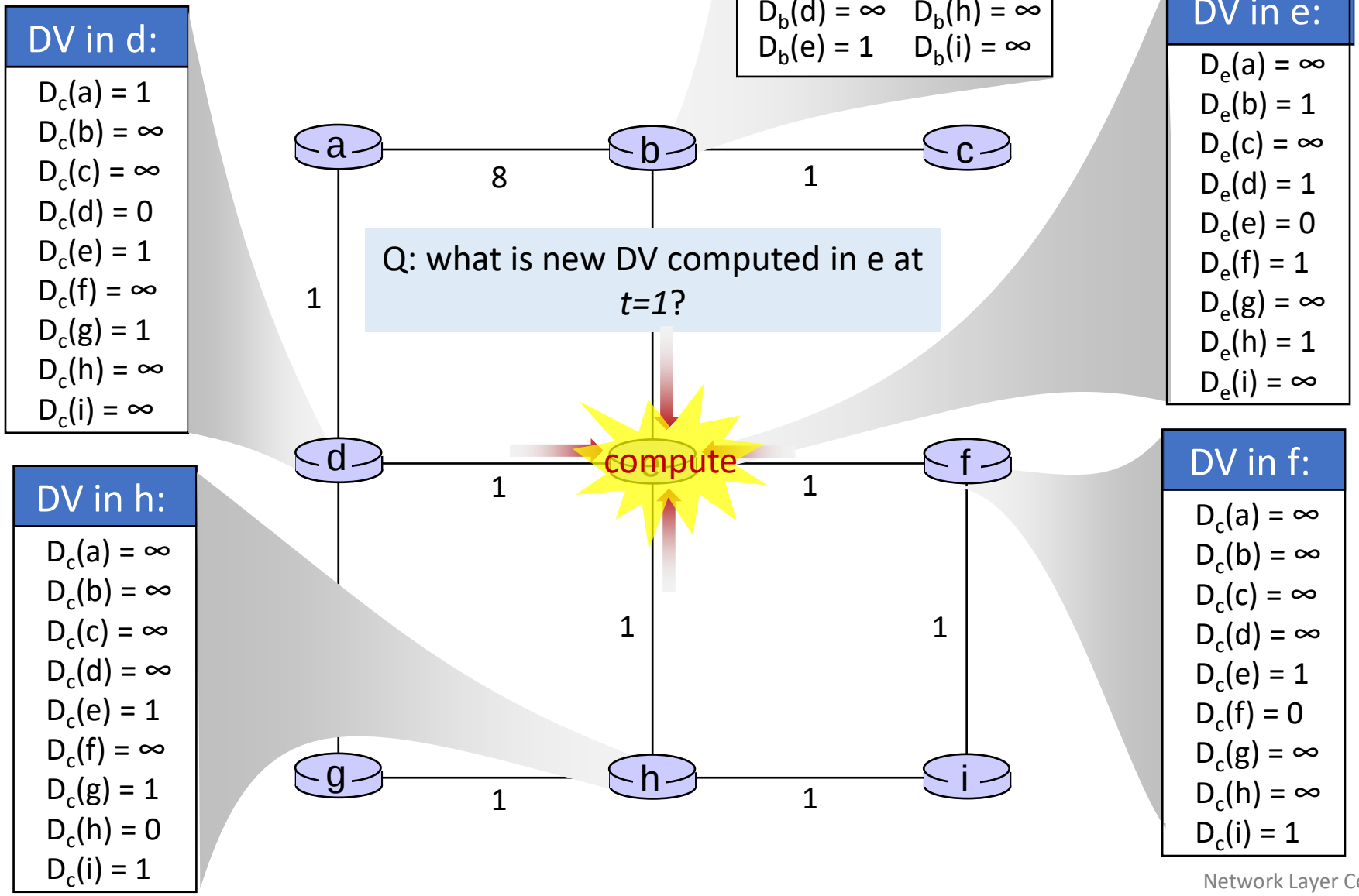
$D_c(a) = 9$
$D_c(b) = 1$
$D_c(c) = 0$
$D_c(d) = 2$
$D_c(e) = \infty$
$D_c(f) = \infty$
$D_c(g) = \infty$
$D_c(h) = \infty$
$D_c(i) = \infty$

Distance vector example: computation



t=1

- e receives DVs from b, d, f, h



Distance vector: state information diffusion

Iterative communication, computation steps diffuses information through network:

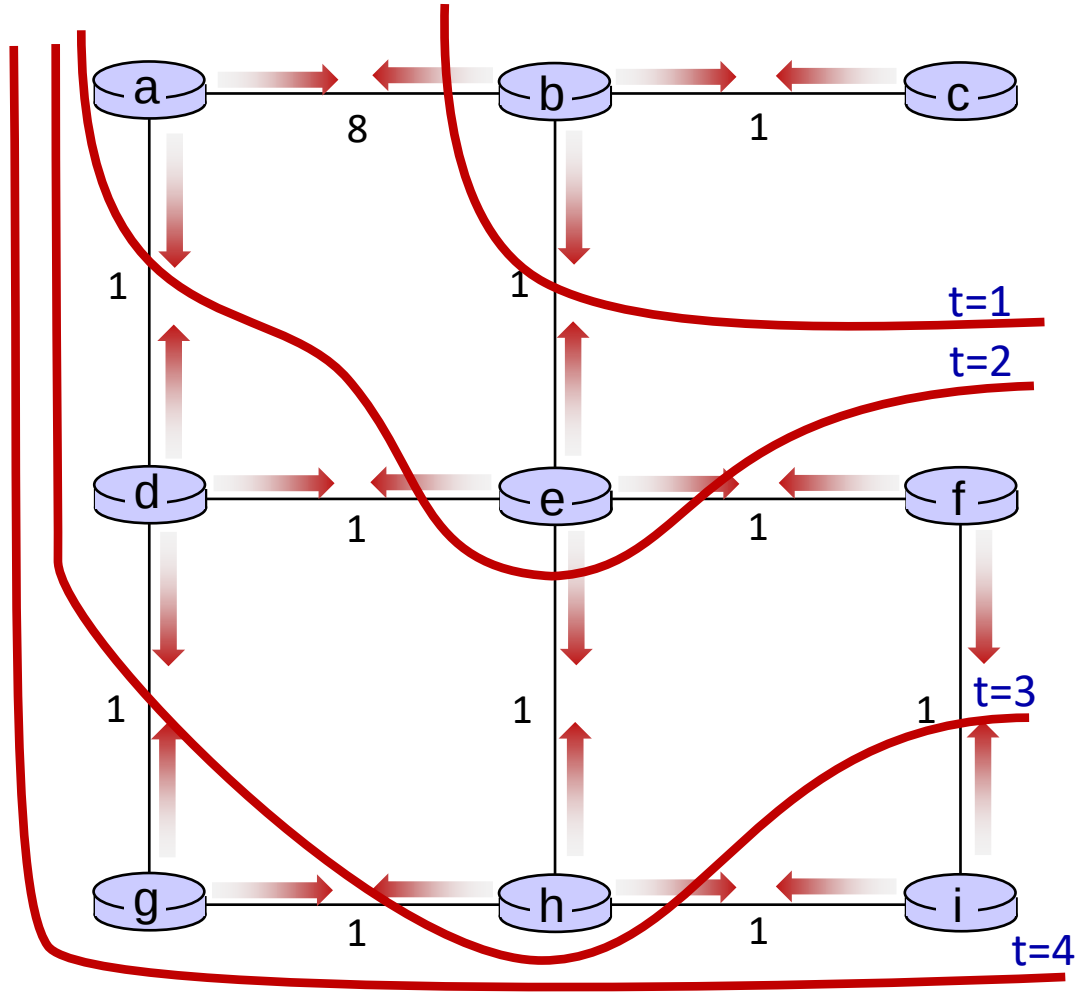
t=0 c's state at t=0 is at c only

t=1 c's state at t=0 has propagated to b, and may influence distance vector computations up to **1** hop away, i.e., at b

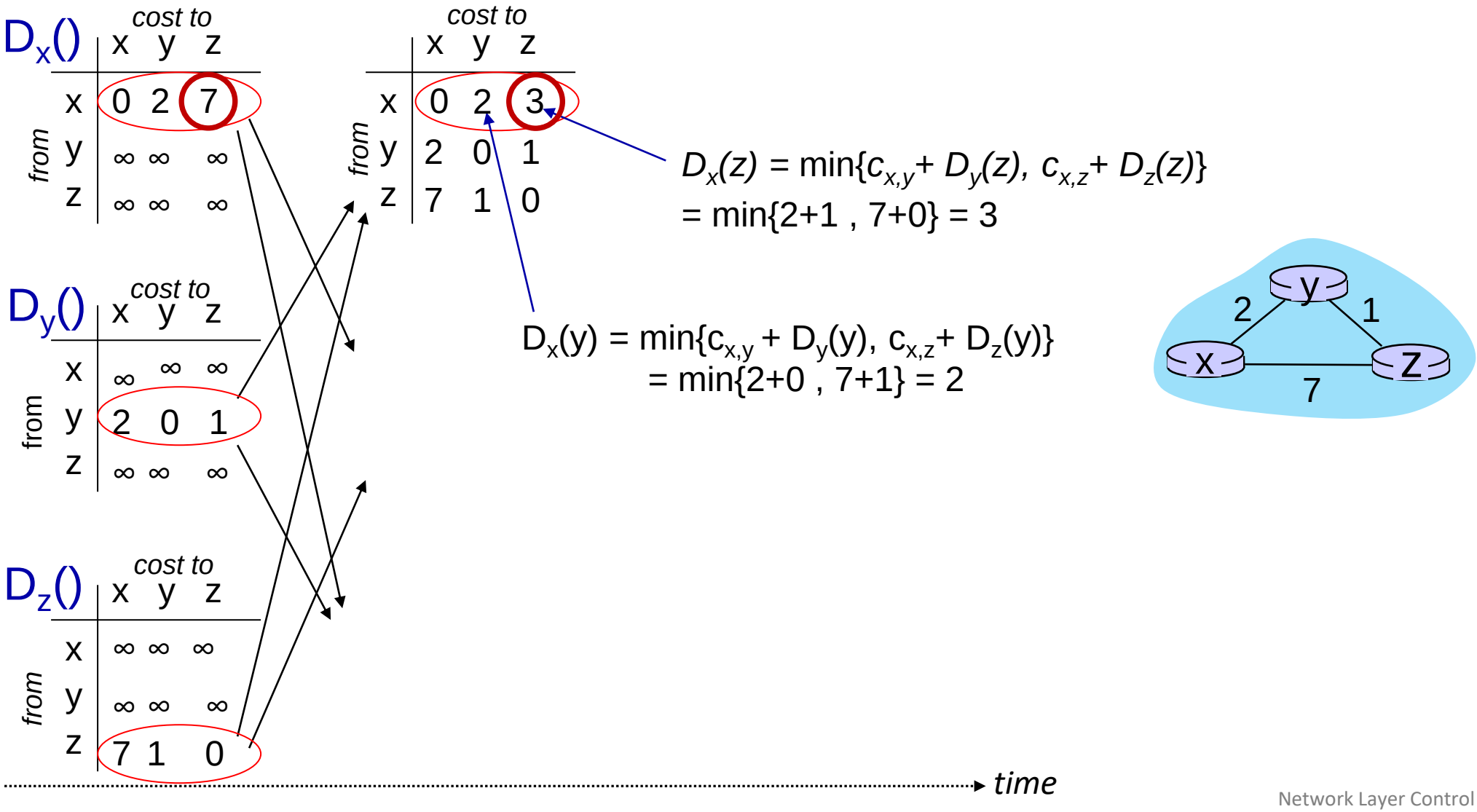
t=2 c's state at t=0 may now influence distance vector computations up to **2** hops away, i.e., at b and now at a, e as well

t=3 c's state at t=0 may influence distance vector computations up to **3** hops away, i.e., at b,a,e and now at c,f,h as well

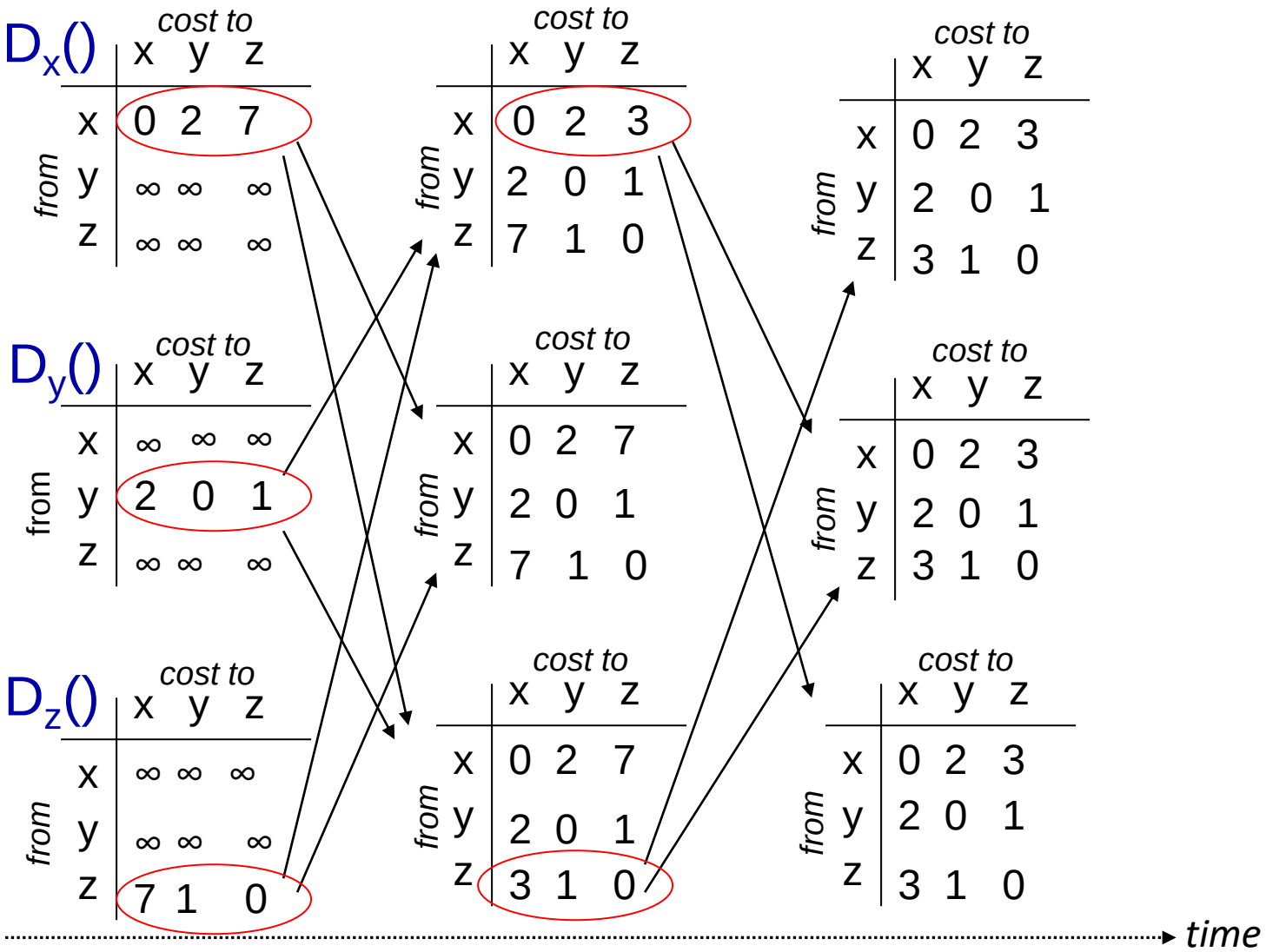
t=4 c's state at t=0 may influence distance vector computations up to **4** hops away, i.e., at b,a,e, c, f, h and now at g,i as well

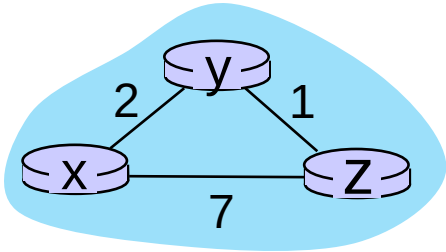


Distance vector: another example



Distance vector: another example

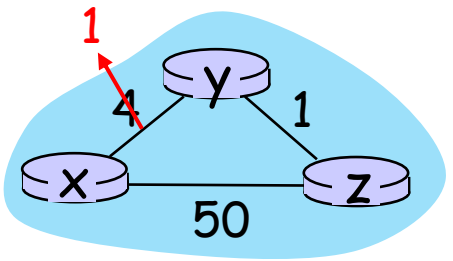




Distance vector: link cost changes

link cost changes:

- node detects local link cost change
- updates routing info, recalculates local DV
- if DV changes, notify neighbors



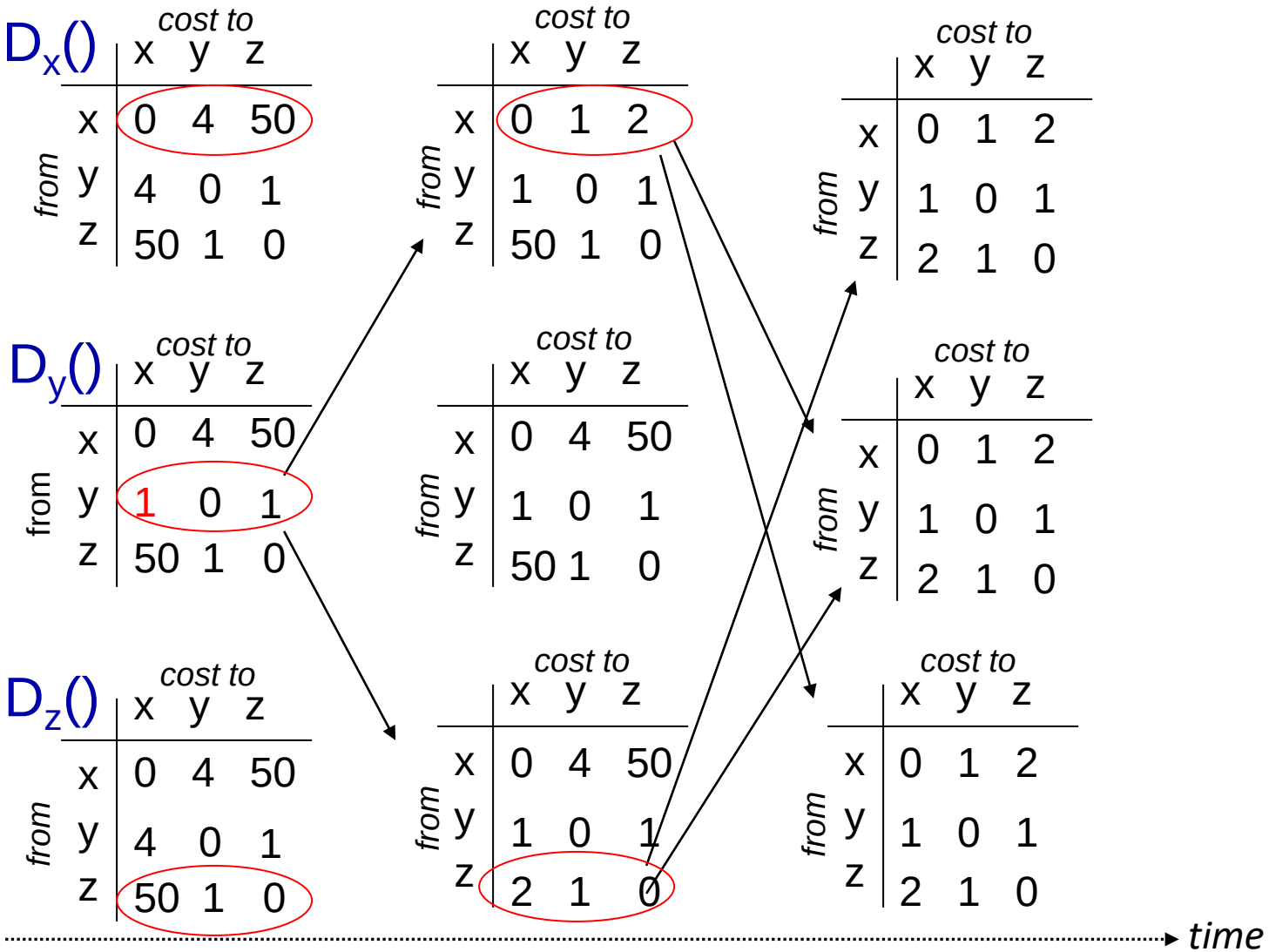
"good news travels fast"

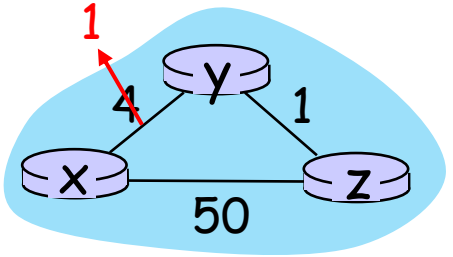
t_0 : *y* detects link-cost change, updates its DV, informs its neighbors.

t_1 : *z* receives update from *y*, updates its table, computes new least cost to *x* , sends its neighbors its DV.

t_2 : *y* receives *z*'s update, updates its distance table. *y*'s least costs do *not* change, so *y* does *not* send a message to *z*.

Good news travels fast

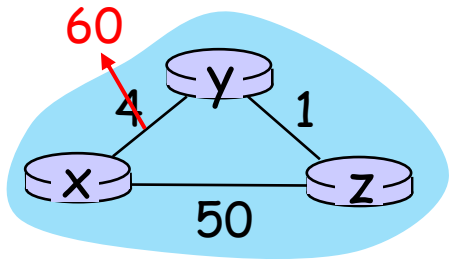




Distance vector: link cost changes

link cost changes:

- node detects local link cost change
- "bad news travels slow" – count-to-infinity problem:
  - Before the link cost changes, $D_y(x) = 4$, $D_y(z) = 1$, $D_z(y) = 1$, and $D_z(x) = 5$.
  - At t_0, the link-cost change (4 → 60). $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\}$
    $= \min\{60 + 0, 1 + 5\} = 6$



  - At t_1, z learns that path to x via y has new cost 6, so z computes "my new cost to x will be 7 via y), notifies y of new cost of 7 to x.
  - At t_2, y learns that path to x via z has new cost 7, so y computes "my new cost to x will be 8 via y), notifies z of new cost of 8 to x.

  …
- A routing loop is like a black hole—a packet destined for x arriving at y or z as of t_1 will bounce back and forth between these two nodes forever (or until the forwarding tables are changed). (The specific looping scenario can be avoided using a technique known as *poisoned reverse*.)

Comparison of LS and DV algorithms

message complexity

LS: with n nodes, E links, O(nE) msgs sent

DV: exchange between neighbors; convergence time varies

speed of convergence

LS: $O(n^2)$ algorithm requires O(nE) msgs

- may have oscillations

DV: convergence time varies

- may have routing loops
- count-to-infinity problem

Conclusion: Neither algorithm is an obvious winner over the other; indeed, both algorithms are used in the Internet.

robustness: what happens if router malfunctions, or is compromised?

LS:

- router can advertise incorrect *link* cost
- each router computes only its *own* table

DV:

- DV router can advertise incorrect *path* cost ("I have a *really* low cost path to everywhere"): black-holing
- each router's table used by others: error propagate thru network

Network layer: "control plane" roadmap

- introduction
- routing protocols
- intra-ISP routing: RIP, OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



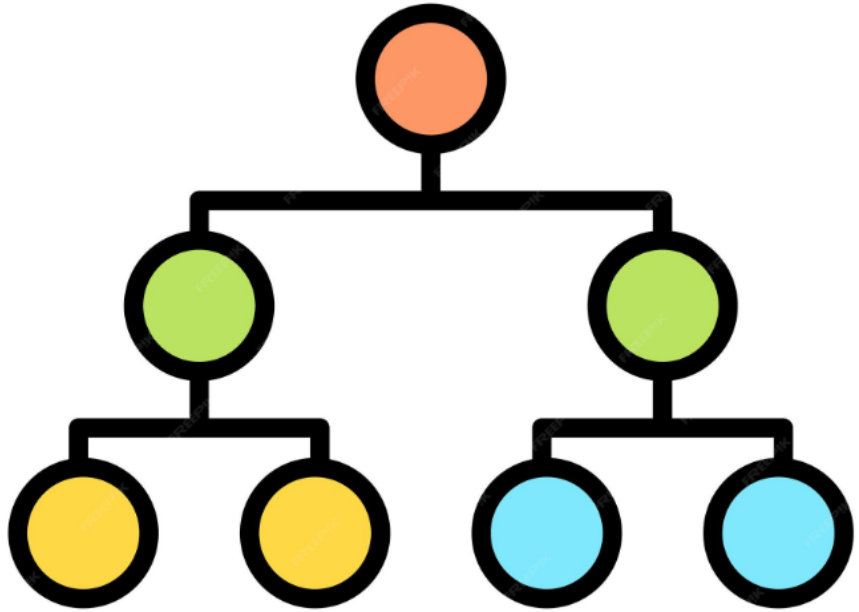
- network management, configuration
  - SNMP
  - NETCONF/YANG

Making routing scalable

our routing study thus far - idealized

- all routers identical
- network "flat"

… not true in practice



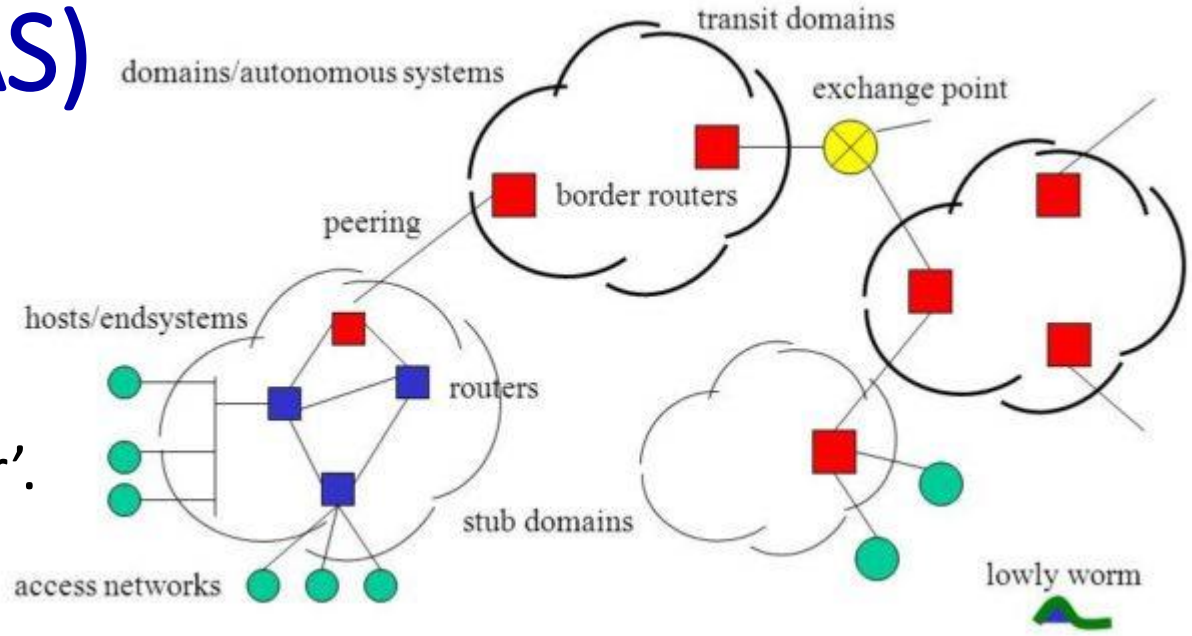
scale: billions of destinations:

- can't store all destinations in routing tables!
- routing table exchange would swamp links!

administrative autonomy:

- Internet: a network of networks
- each network admin may want to control routing in its own network

Autonomous System (AS)



- An AS is a network under a single administrative control with a single routing protocol, identified by 'AS number'.

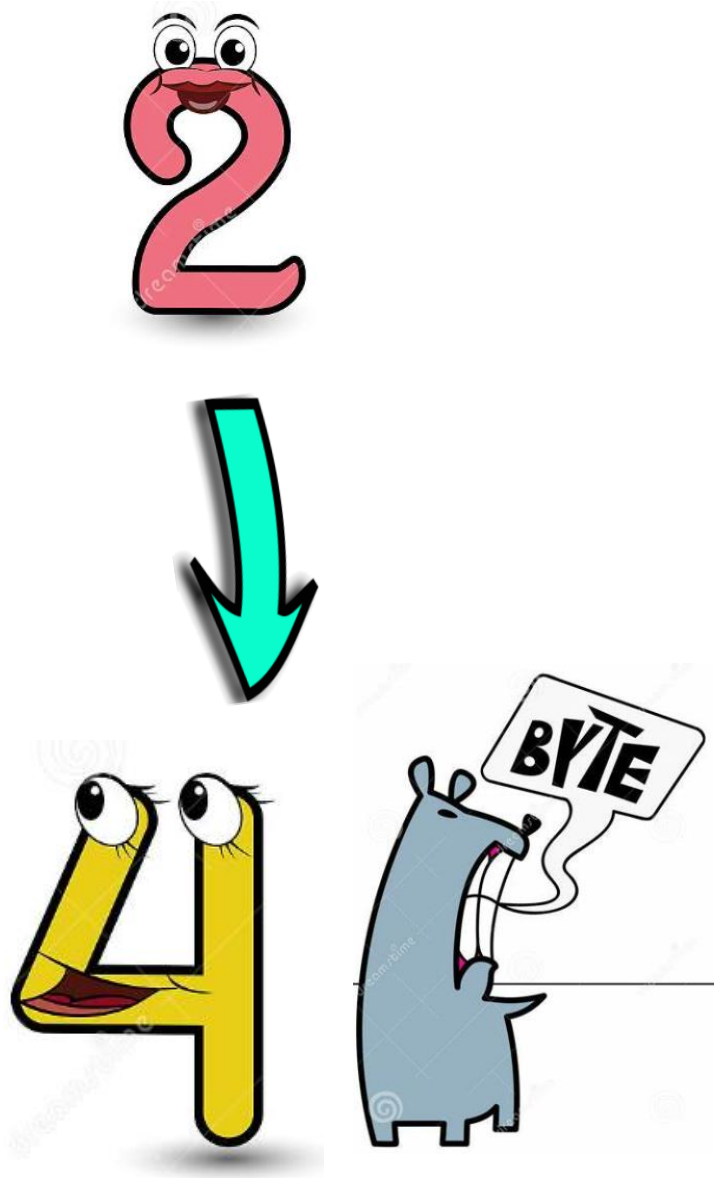
- An AS
  - can independently decide who to exchange traffic with on the Net,
  - isn't dependent upon a third party for access.

- **Examples:** Networks of ISPs, hosting providers, telecommunications monopolists, multinationals, schools, hospitals, individuals or anyone needing policy discrimination.

Autonomous System Number (ASN)

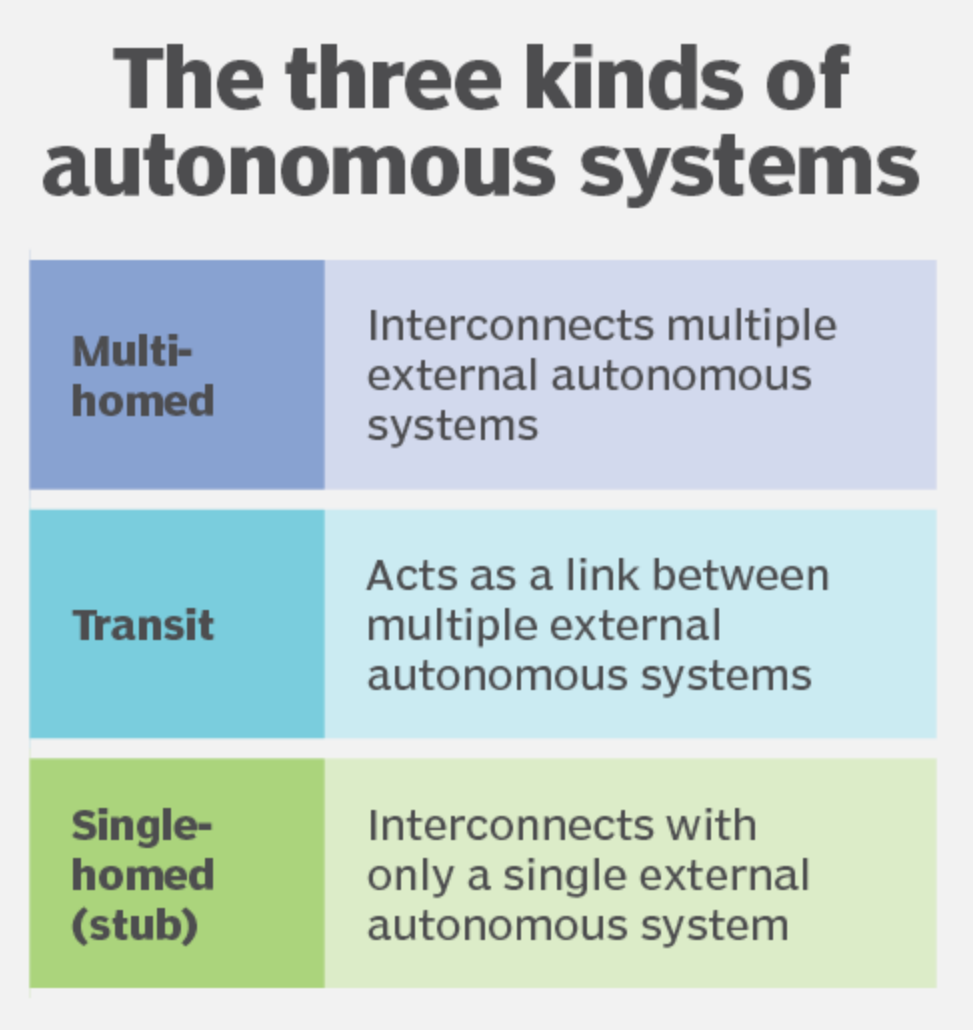
a globally unique identifier

- Two different formats to represent ASNs:
  - A **2-byte ASN** is a 16-bit number (0 to 65535).
    - IANA reserved 1,023 of them (64512 to 65534) for private use.
  - A **4-byte ASN** is a 32-bit number (0 to 4294967295).
    - IANA reserved a block of 94,967,295 ASNs (4200000000 to 4294967294) for private use.
- Up until IETF proposed a gradual transition to 4-byte ASNs in 2007, all ASNs were 2-byte.
  - There is no longer a distinction between a 2-byte and 4-byte ASN, and all ASNs should be considered 4-byte.



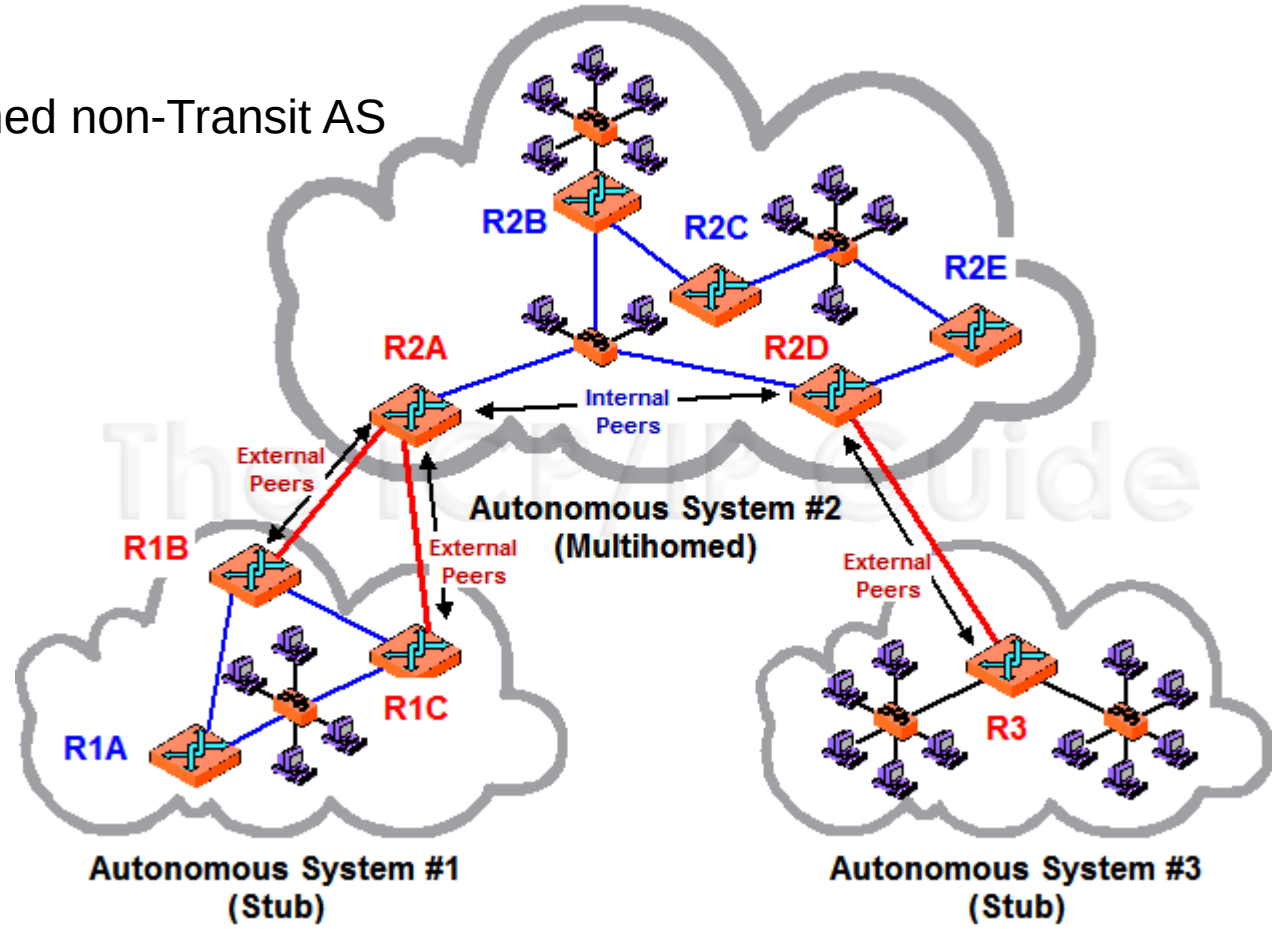
Three different kinds of Autonomous Systems

1. A multi-homed AS connects to two or more ASes, so it can maintain its Internet connection should one AS connection fail.
2. A stub AS connects to only one other AS, though it may have its own private connections not visible to the rest of the Internet.
3. A transit AS acts as a link between two or more other ASes, allowing for data to pass through it, even data from unassociated networks ISPs,
   - for example, offer their customers and their customers' networks access to other networks and the Internet via transit AS.

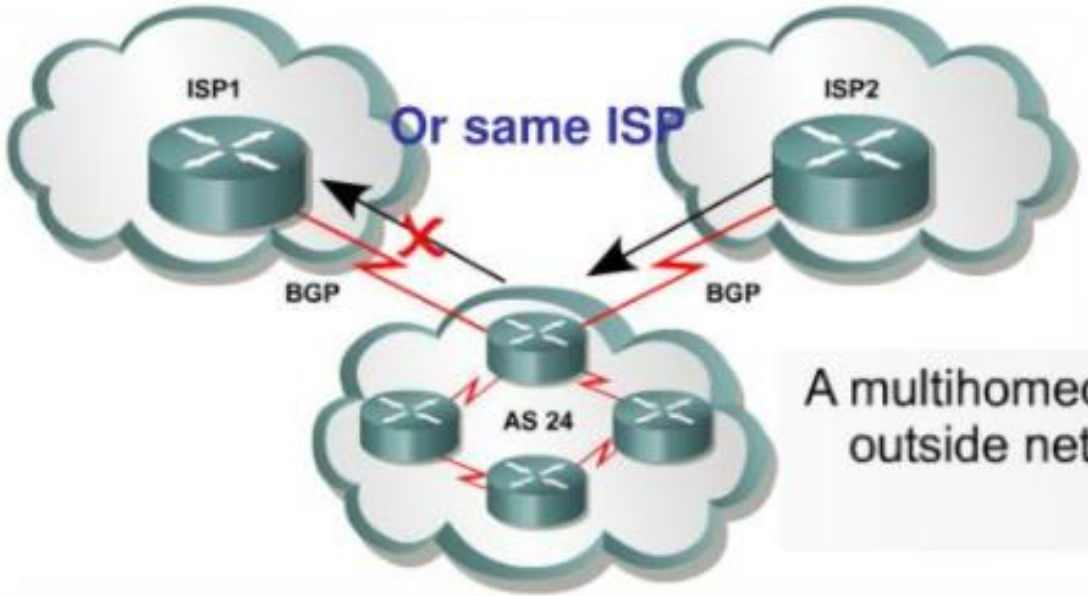


Three different kinds of Autonomous Systems

1. A multi-homed AS
   - Multi-homed Transit AS
   - Multi-homed non-Transit AS
2. A stub AS
3. A transit AS



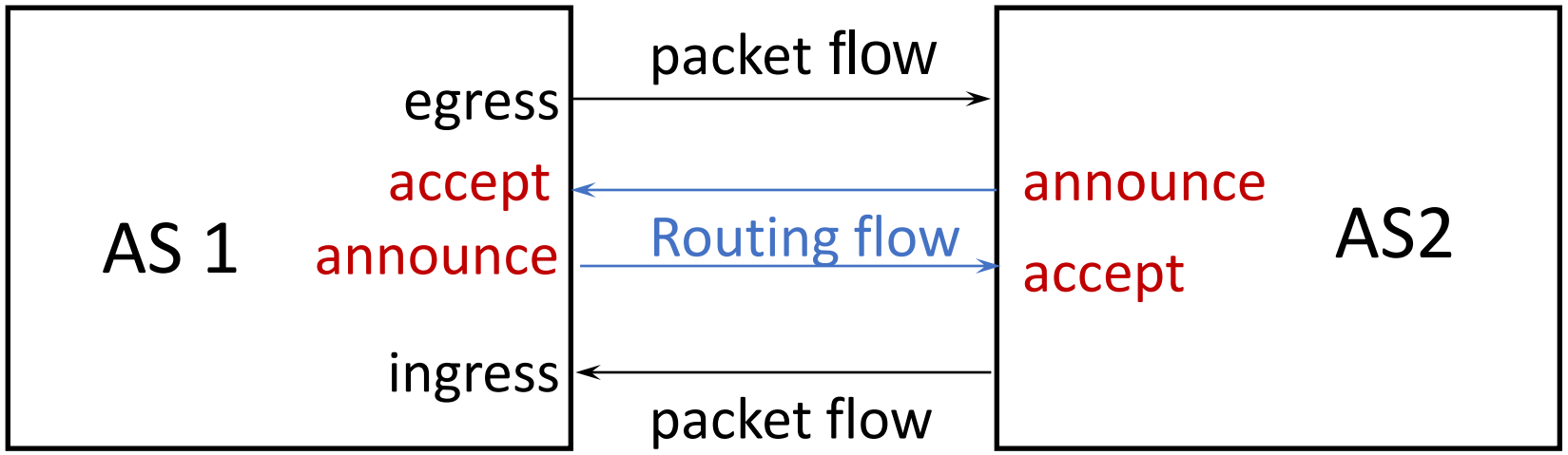
Multihomed nontransit autonomous systems



A multihomed nontransit AS features more than one exit point to outside networks, but does not allow traffic to pass from one outside connection to another.

- Multihomed nontransit autonomous systems do not really need to run BGP4 with their providers.
- It is usually **recommended** and **often required** by ISPs.

Routing flow and packet flow



For networks in AS1 and AS2 to communicate:

AS1 must announce routes to AS2

AS2 must accept routes from AS1

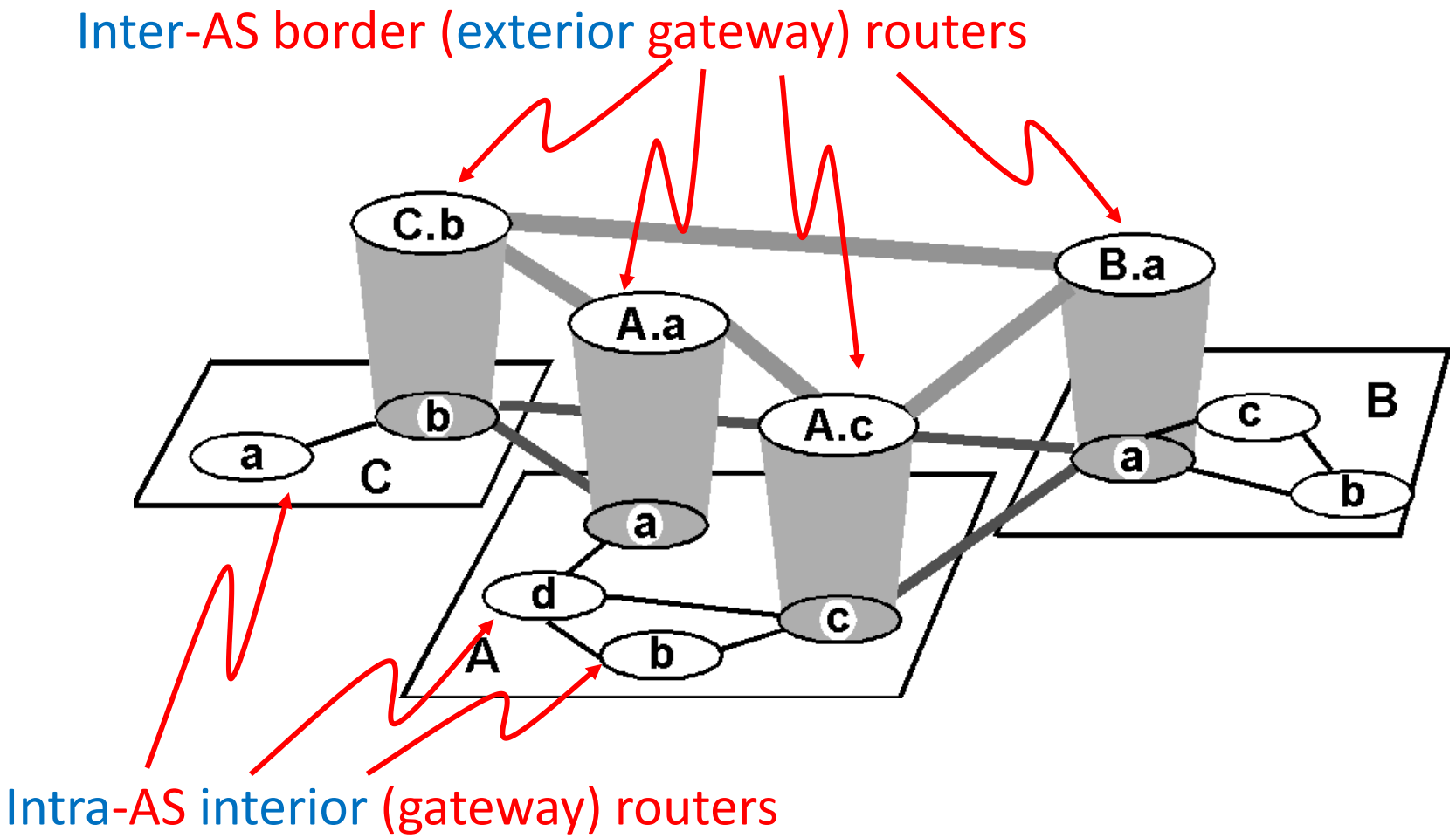
AS2 must announce routes to AS1

AS1 must accept routes from AS2

Types of Routes

- Static Routes
  - configured manually
- Connected Routes
  - created automatically when an interface is 'up'
- Interior Routes
  - Routes within an AS
- Exterior Routes
  - Routes exterior to AS

Internet AS Hierarchy



Internet approach to scalable routing

aggregate routers into regions known as "autonomous systems" (AS) (a.k.a. "domains")

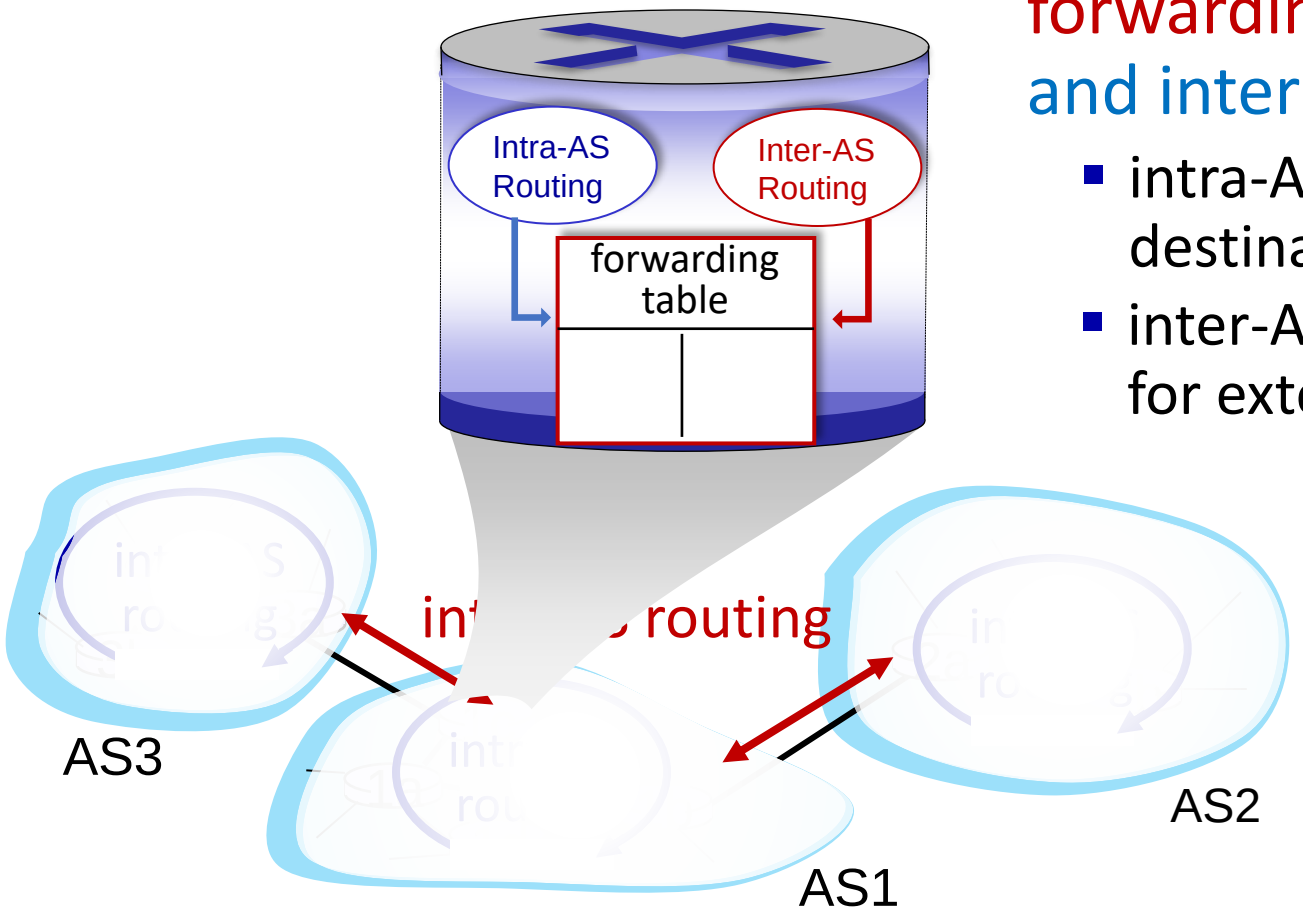
intra-AS (aka "intra-domain"): routing *within same AS ("network")*

- all routers in AS must run same intra-domain protocol
- routers in different AS can run different intra-domain routing protocols
- gateway router: at "edge" of its own AS, has link(s) to router(s) in other AS'es

inter-AS (aka "inter-domain"): routing *among* AS'es

- gateways perform inter-domain routing (as well as intra-domain routing)

Interconnected ASes



forwarding table configured by intra- and inter-AS routing algorithms

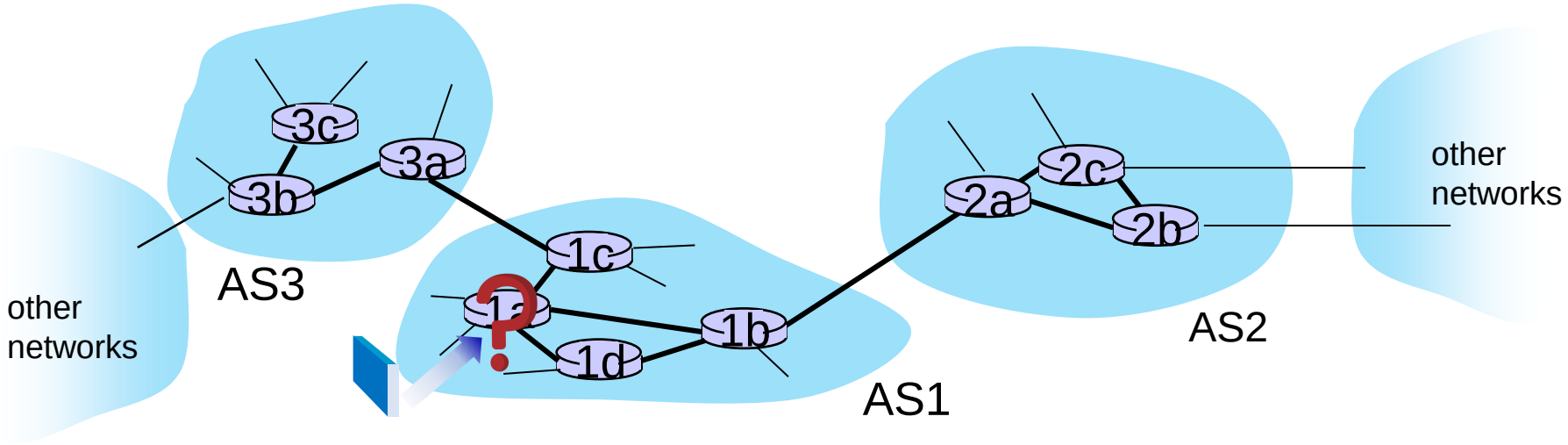
- intra-AS routing determine entries for destinations within AS
- inter-AS & intra-AS determine entries for external destinations

Inter-AS routing: a role in intradomain forwarding

- suppose router in AS1 receives datagram destined outside of AS1:
  - router should forward packet to gateway router in AS1, but which one?

AS1 inter-domain routing must:

1. learn which destinations reachable through AS2, which through AS3
2. propagate this reachability info to all routers in AS1

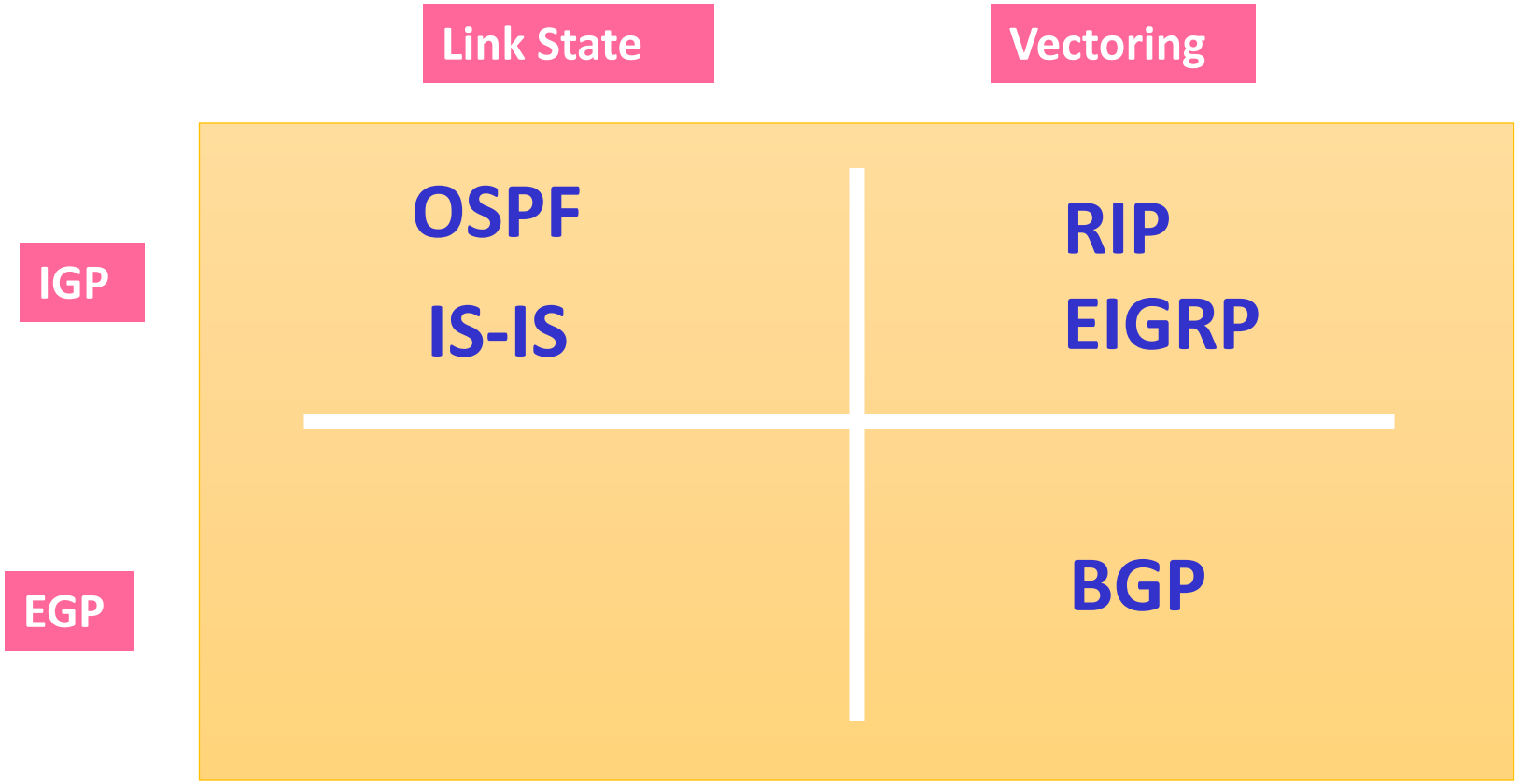


Intra-AS routing: routing within an AS

most common intra-AS routing protocols:

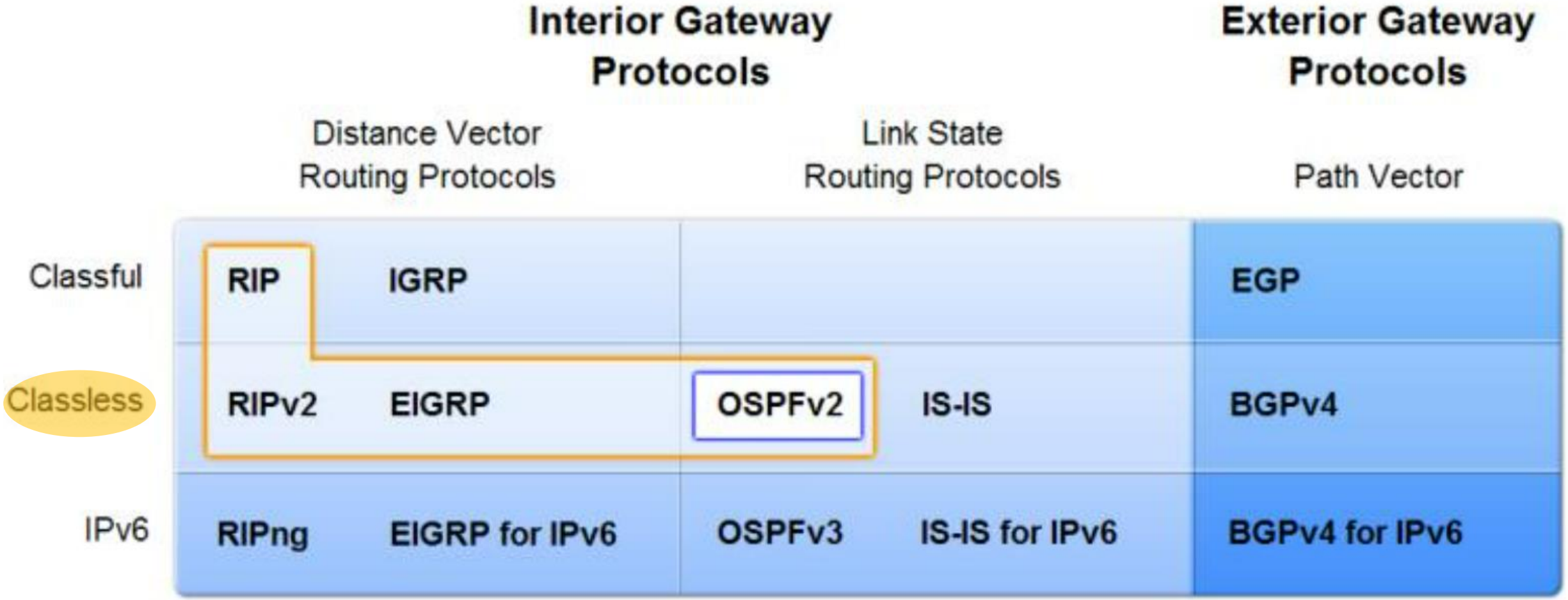
- RIP: Routing Information Protocol [RFC 1723]
  - classic DV: DVs exchanged every 30 secs
  - no longer widely used
- EIGRP: Enhanced Interior Gateway Routing Protocol
  - DV based
  - formerly Cisco-proprietary for decades (became open in 2013 [RFC 7868])
- OSPF: Open Shortest Path First [RFC 2328]
  - link-state routing
  - IS-IS protocol (ISO standard, not RFC standard) essentially same as OSPF

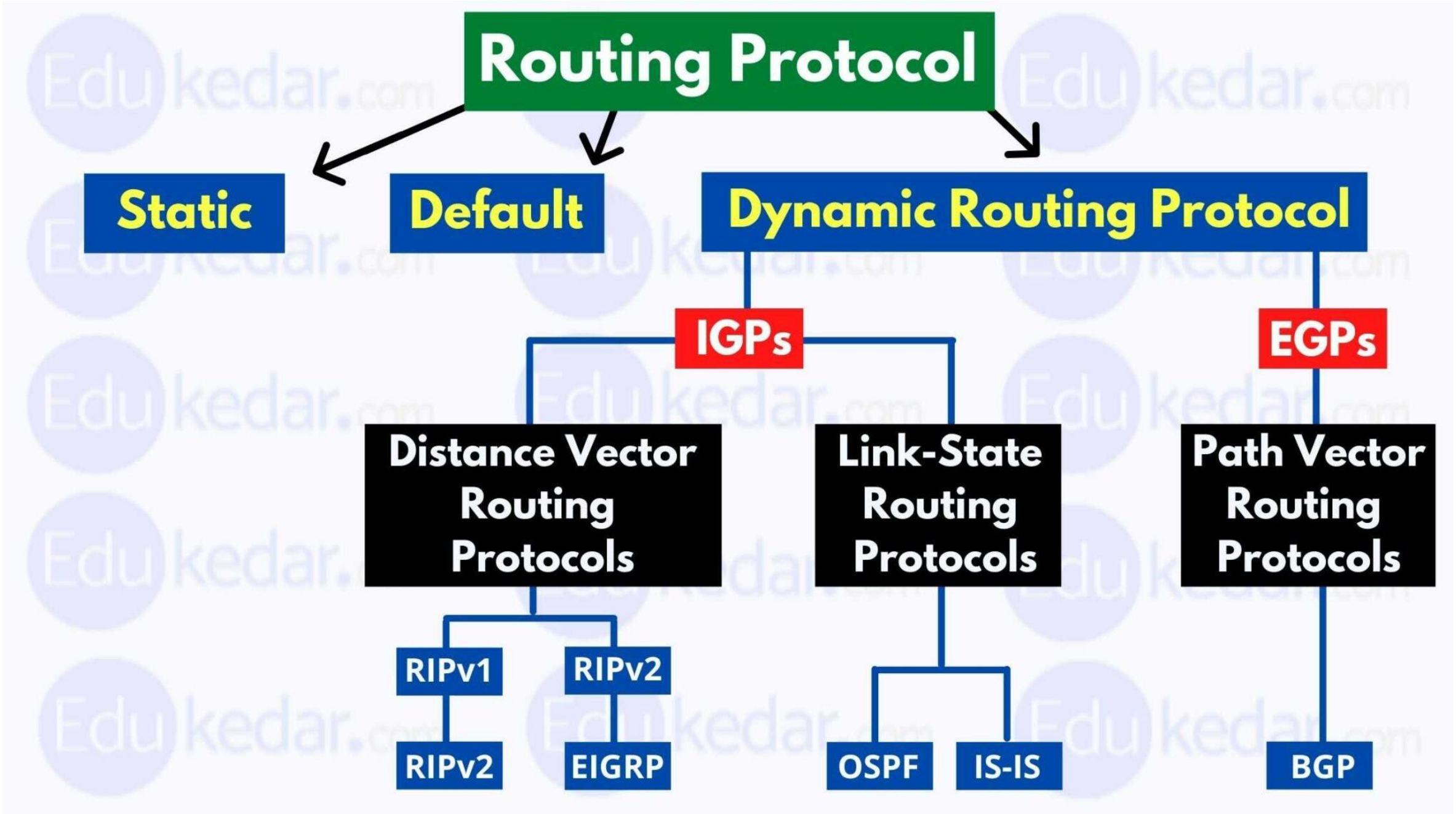
IGP and EGP (1)



IGP – Interior Gateway Protocol
EGP – Exterior Gateway Protocol

IGP and EGP (2)


Interior Gateway Protocols
Exterior Gateway Protocols
Distance Vector Routing Protocols
Link State Routing Protocols
Path Vector
Classful
Classless
IPv6
RIP
IGRP
EGP
RIPv2
EIGRP
OSPFv2
IS-IS
BGPv4
RIPng
EIGRP for IPv6
OSPFv3
IS-IS for IPv6
BGPv4 for IPv6



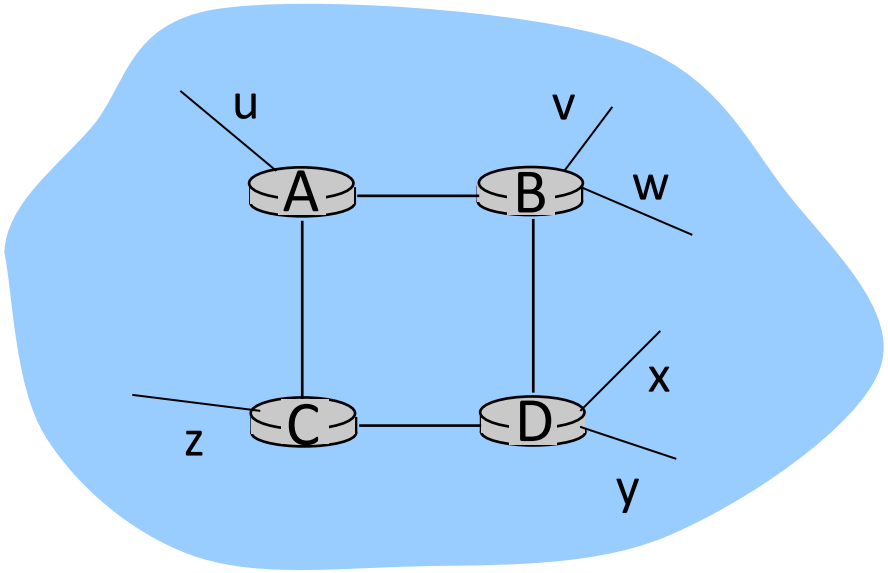
Edukedar.com

Intra-AS Routing

- also known as Interior Gateway Protocols (IGP)
- most common Intra-AS routing protocols:
  - RIP: Routing Information Protocol
  - OSPF: Open Shortest Path First
  - IGRP: Interior Gateway Routing Protocol (Cisco proprietary)
  - IS-IS: Intermediate System to Intermediate System

RIP (Routing Information Protocol)

- distance vector algorithm
- included in BSD-UNIX Distribution in 1982
- current standard is RIP2 (IPv4), RIPng (IPv6)
- a router is defined to be one hop from a directly connected network.
- distance metric: # of hops (max = 15 hops)



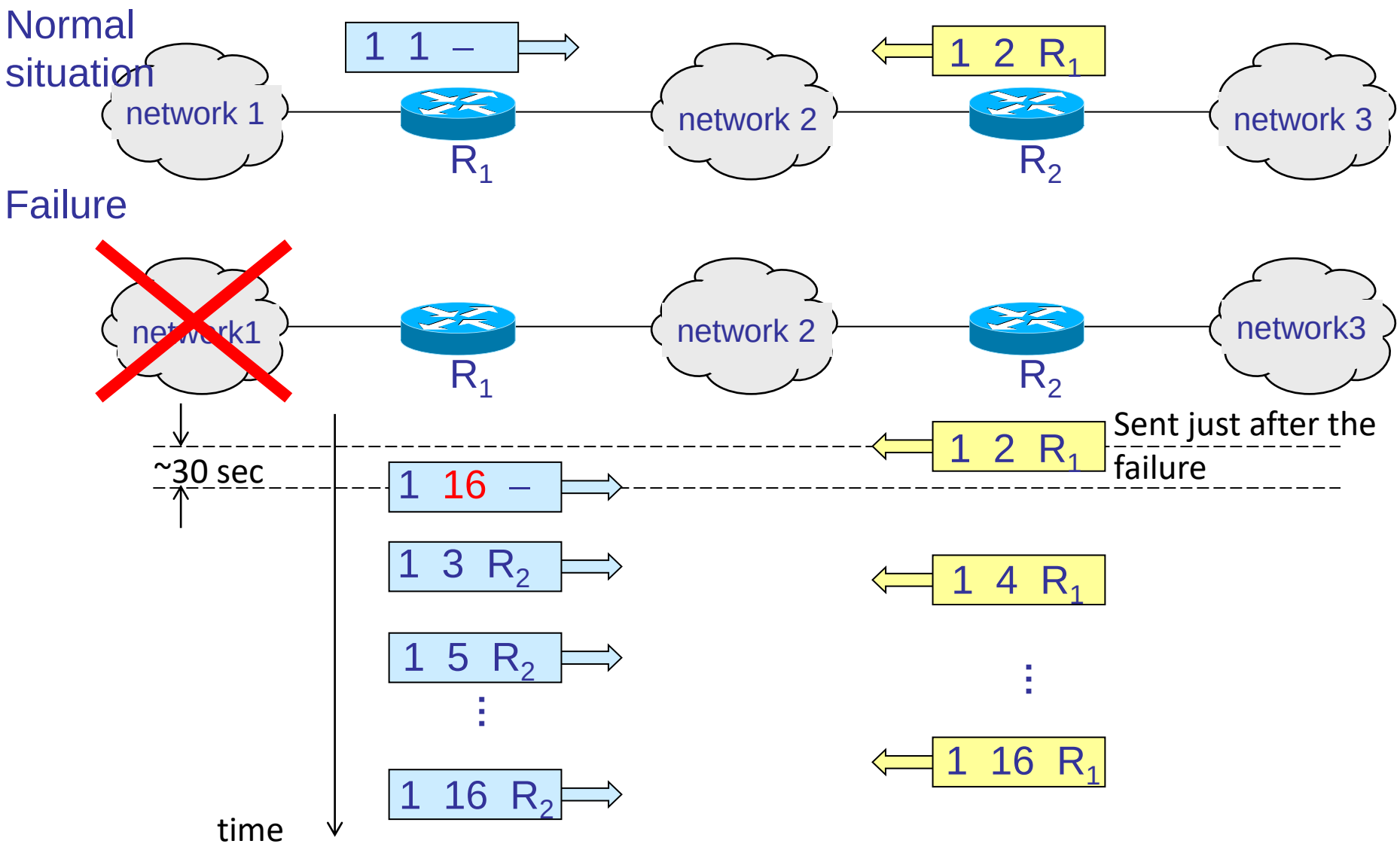
From router A to subnets:

destination	hops
u	1
v	2
w	2
x	3
y	3
z	2

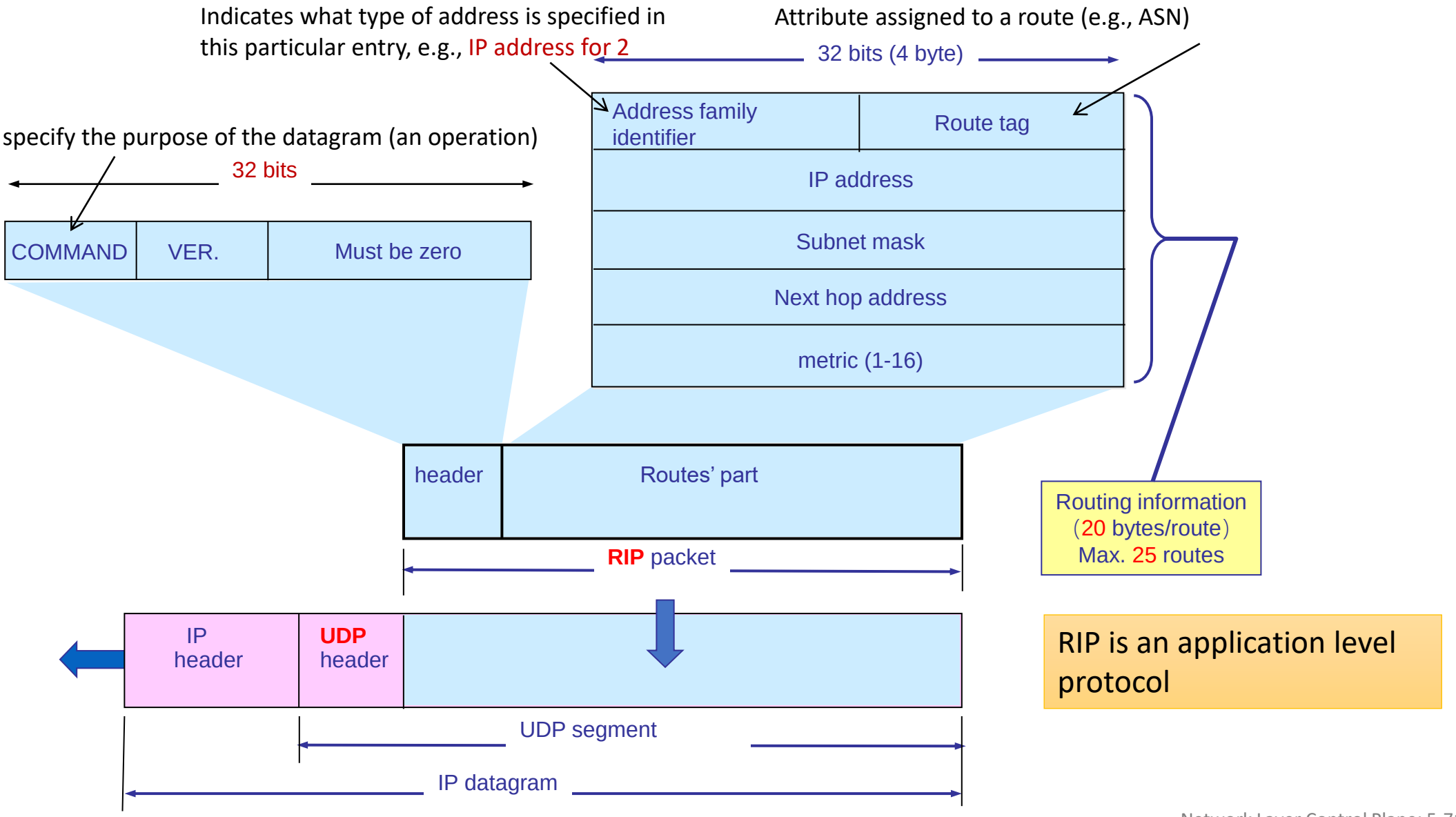
RIP advertisements

- *distance vectors:* exchanged among neighbors every 30 sec via Response Message (also called advertisement)
- each advertisement: list of up to 25 destination subnets within AS

Slow convergence problem (count to infinity)



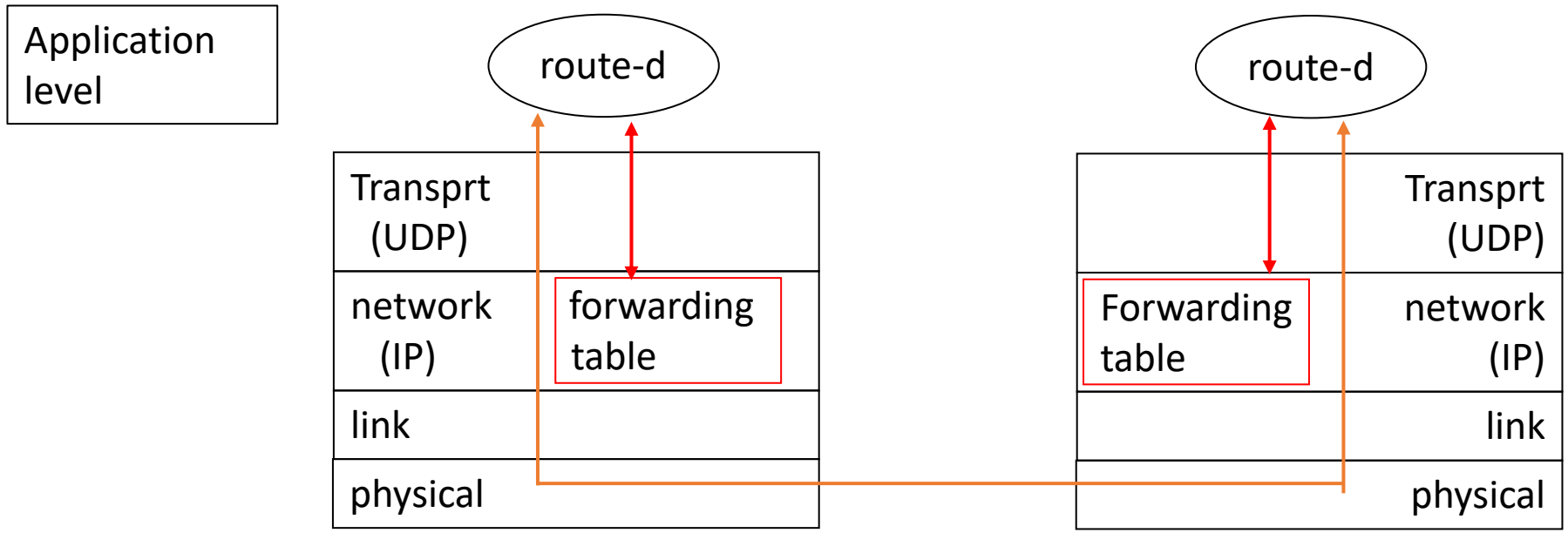
RIP2 message format



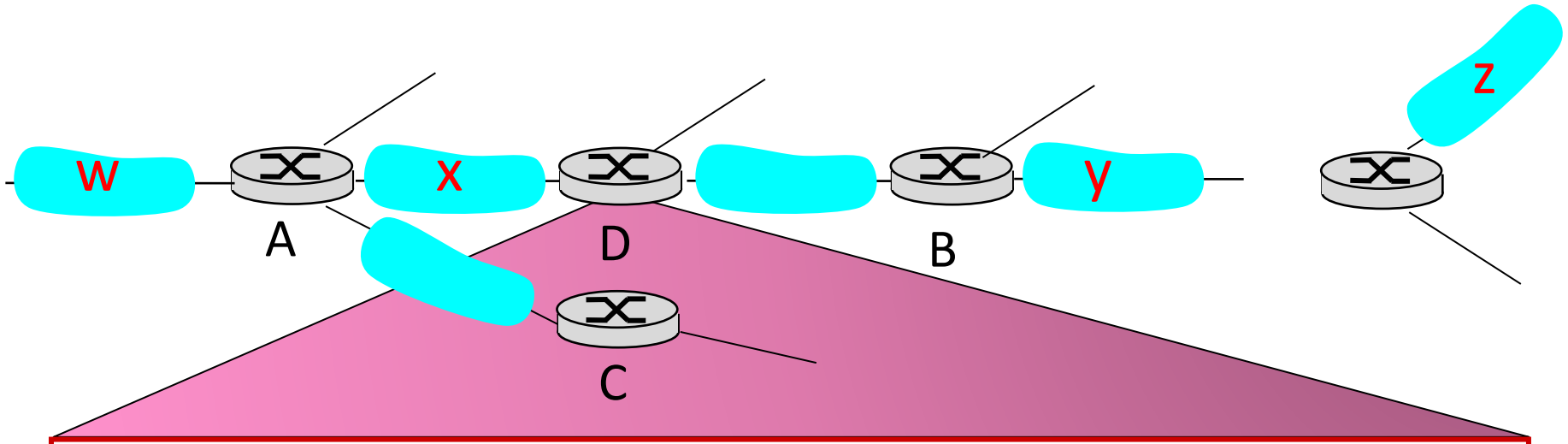
RIP is an application level protocol

RIP Table processing

- RIP routing tables managed by **application-level** process called route-d (daemon)
- advertisements sent in UDP packets, periodically repeated



RIP: Example



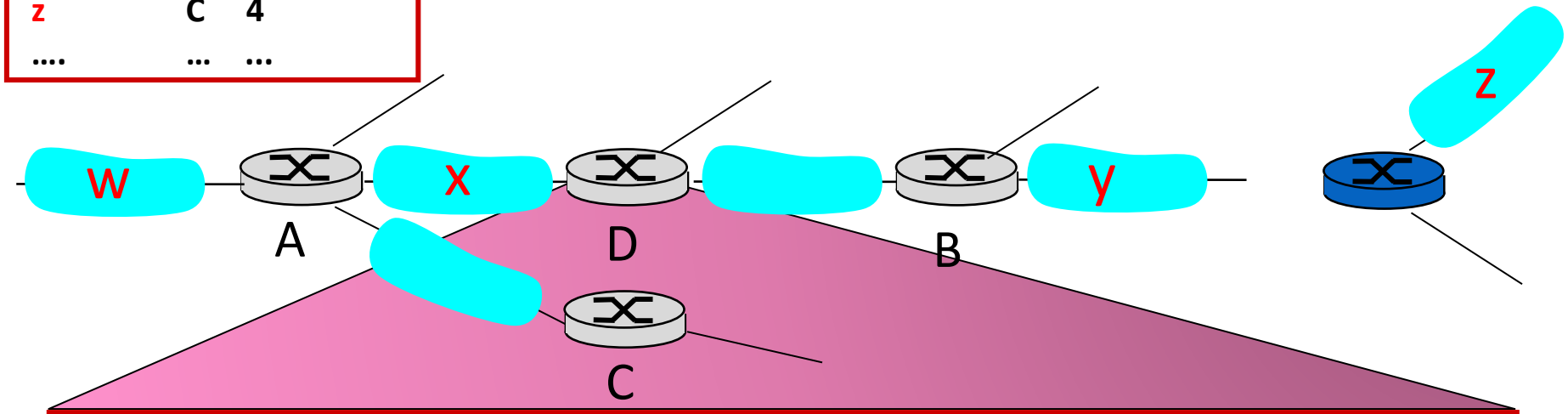
Destination Network	Next Router	Num. of hops to dest.
w	A	2
y	B	2
z	B	3
x	--	1
….	….	….

Routing/Forwarding table in D

RIP: Example

Dest	Next	hops
w	-	1
x	-	1
z	C	4
....	...	...

Advertisement from A to D



Destination Network	Next Router	Num. of hops to dest.
w	A	2
y	B	2
z	~~B~~ A	~~7~~ 5
x	--	1
....	....	....

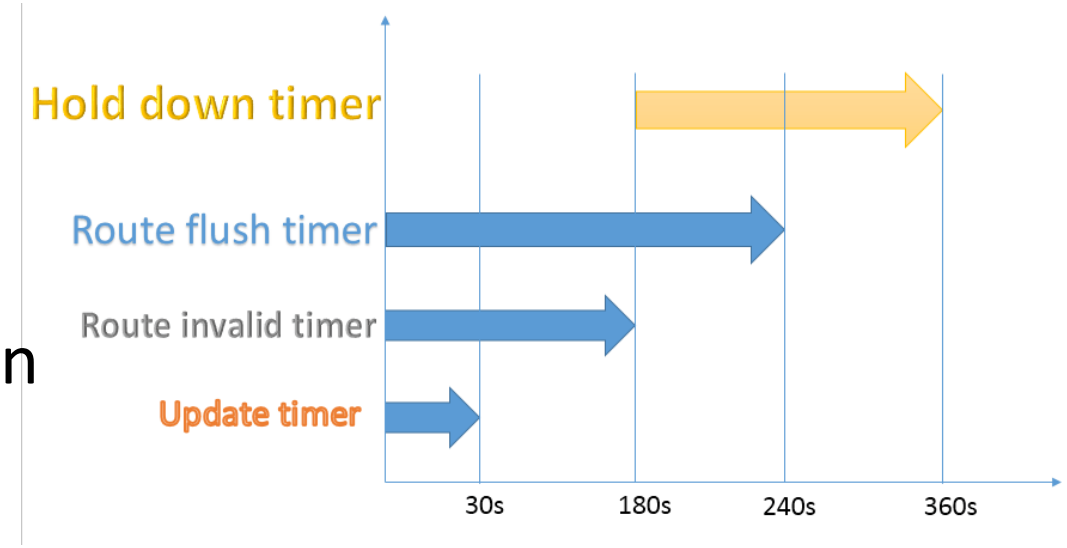
Routing/Forwarding table in D

RIP: Link Failure and Recovery

If no advertisement heard after 180 sec (i.e., 3 min) --> neighbor/link declared dead

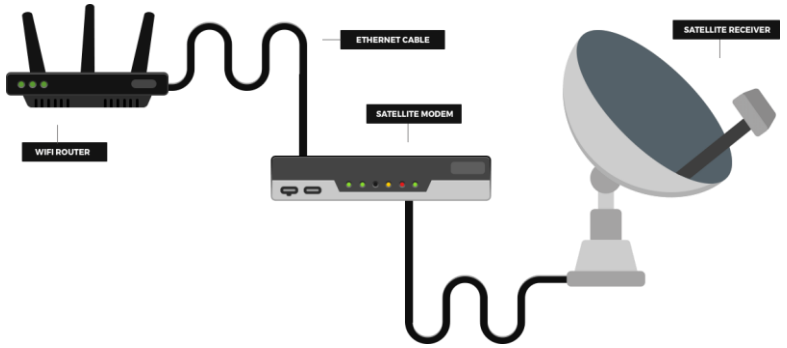
- routes via neighbor invalidated
- new advertisements sent to neighbors
- neighbors in turn send out new advertisements (if tables changed)
- link failure info propagates to entire net
- *poison reverse* used to prevent ping-pong loops (infinite distance = 16 hops)

RIP Timers



- Multiple timers for performance regulation
- Each route contains the following timers.
  - Routing-update timer. ---- 30 Seconds
    - *Every 30 seconds, the output process is instructed to generate a complete response to every neighboring router/gateway.*
    - *30 seconds updates are triggered by a clock not affected system load or service.*
    - *To prevent collisions – 30 second timer offset by addition of a small random time.*
  - Time out timers. ---- 180 Seconds
    - *Set route metric to 16 (Infinity); notify neighbors*
  - Garbage collection timer. ---- 120 Seconds
    - *Route is deleted from routing table; notify neighbors*

Disadvantages of RIP hop counts



- Hop counts provide a crude measure
  - does not always yield optimal routes with least delay or highest capacity
    - E.g., a path with hop count 3 crossing three Ethernets may be much faster than a path with hop count 2 crossing two satellite connections
  - makes relatively static routing (not respond to load changes)
- Using RIP as an IGP limits routing in two ways
  - Hop-count metric
  - Limit on network span restricts the size of the Internet
    - Neither a limit on the total number of routers nor a limit on density
    - Most campus networks have a small span (hierarchy topology)
    - The limit affects only large ASs that do not have a hierarchical organization

RIP2 - Disadvantages

- supports generic notion of authentication, but only "password" is defined so far

  →still not very secure

- packet size increases as the number of networks increases

  →not suitable for large networks

- generates more protocol traffic than OSPF,
  - because it propagates routing information by periodically transmitting the entire routing table to neighbor routers
- may be slow to adjust for link failures

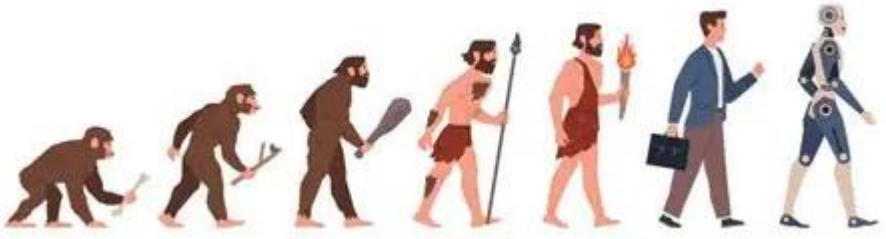
Solution: OSPF within an AS

- Can support fine-grained metrics (vs. RIP)
- Multiple metrics
  - Throughput, Delay, Cost, Reliability
- Can compute a different routing table for each metric.
- OSPFv2 supports an extension that allows the metric to be used specified in the packet

OSPF (Open Shortest Path First) routing

- "open": publicly available
- classic link-state
  - each router floods OSPF link-state advertisements (directly over IP rather than using TCP/UDP) to all other routers in entire AS
  - multiple link costs metrics possible: bandwidth, delay
  - each router has full topology, uses Dijkstra's algorithm to compute forwarding table
- *security:* all OSPF messages authenticated (to prevent malicious intrusion)

OSPF Development History



1987 OSPF Group formed

1989 OSPFv1 published RFC 1131

1991 **OSPFv2** published RFC 1247

1993 Becomes recommended

CIDR

OSPFv2 update RFC 1583

MOSPF

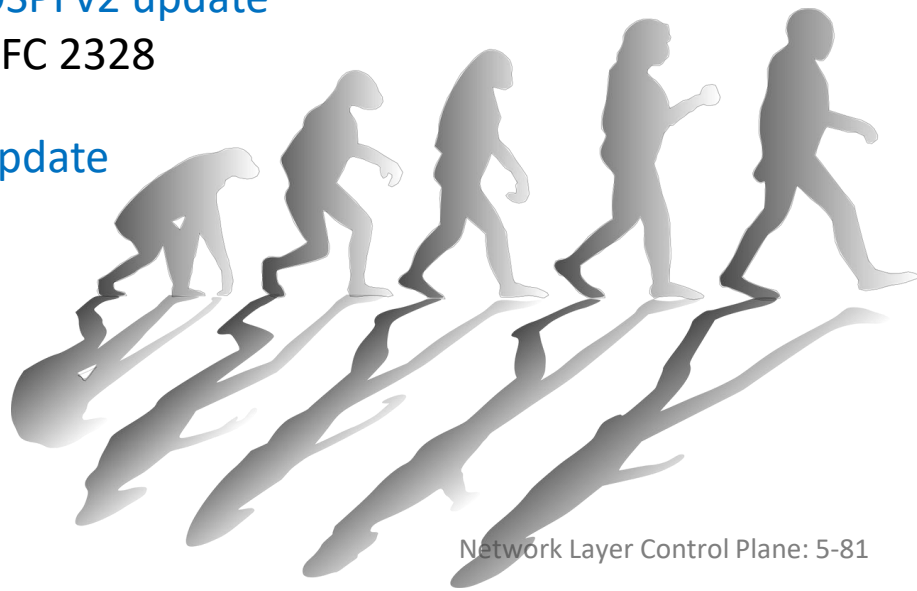
Point-to-multipoint interfaces

1995 Cryptographic authentication

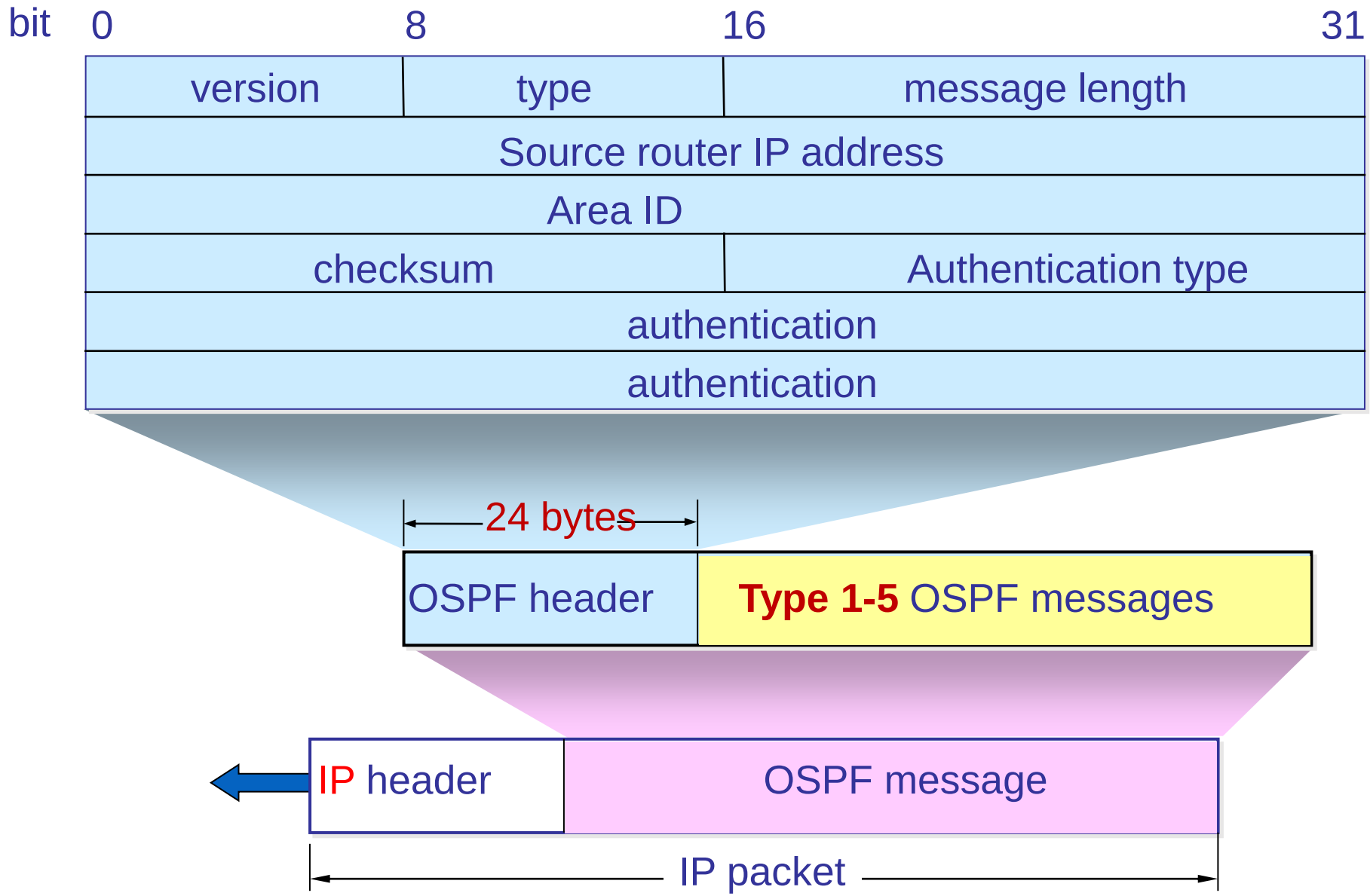
1997 OSPFv2 update RFC 2178

OSPFv2 update RFC 2328

1999 **OSPFv3** RFC 2740



Fixed 24-byte OSPF message header



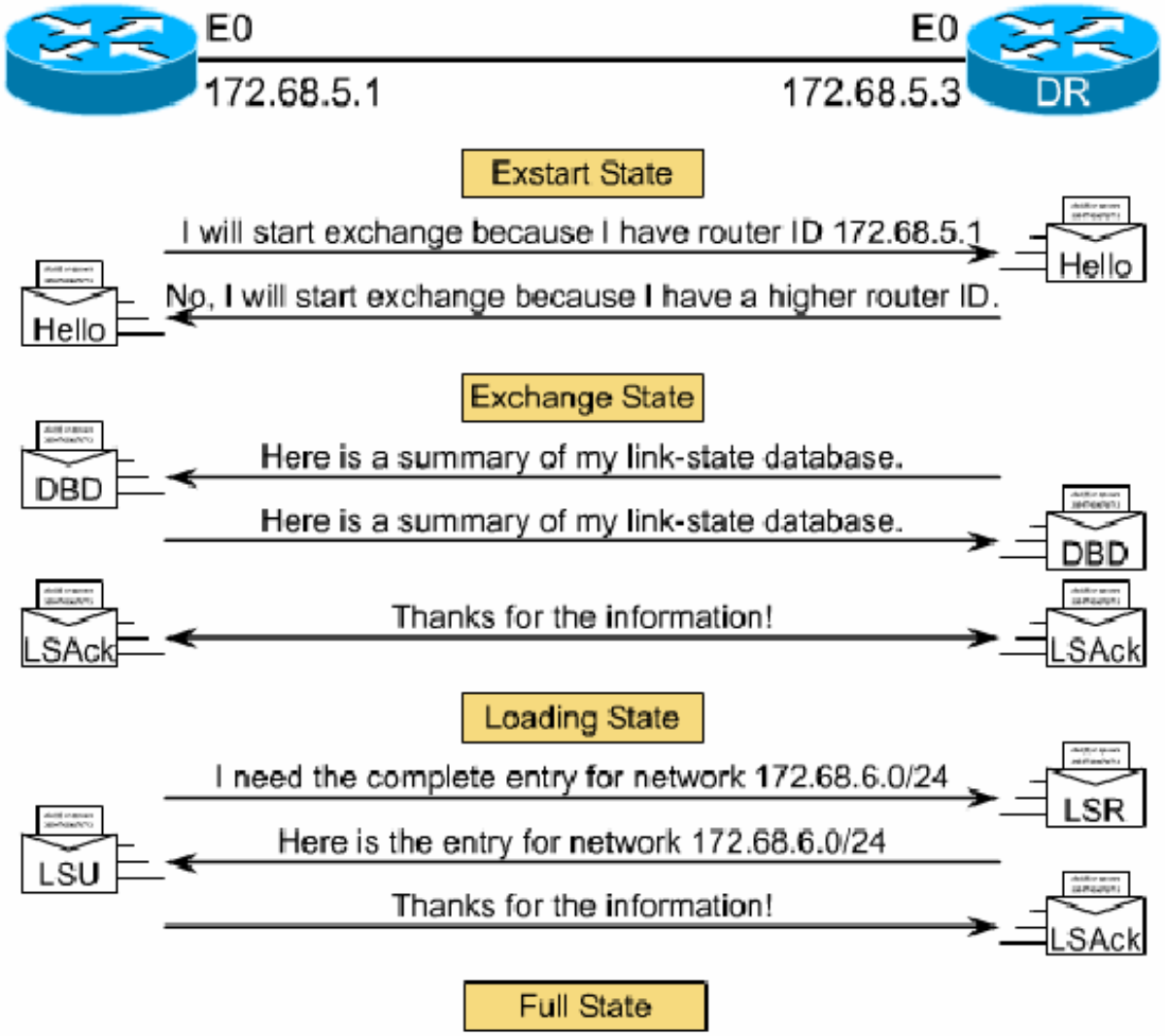
OSPF Basics

- Distributed, replicated database model
  - Each router builds a topology database
  - Describes complete routing topology
- Link state database
  - Identical for all the routers
- LSA: Link State Advertisements
  - Information about adjacencies sent to all routers

OSPF message types

Type	Message Name	Description
1	Hello	Used to test reachability Discovers neighbors and builds adjacencies between them
2	Database Description (DBD)	Checks for database synchronization between routers
3	Link Status Request (LSR)	Requests specific link-state records from router to router
4	Link Status Update (LSU)	Sends specifically requested link-state records
5	Link Status Acknowledgement (LSAck)	Acknowledges the other packet types

Discovering Routes and Reaching Full State



"adjacent"

Reachability

OSPF Type-1 (Hello)

OSPF Type-1 (Hello)

Database synchronization

OSPF Type-2 (DBD)

OSPF Type-2 (DBD)

OSPF Type-5 (LSAck)

New link state synch.

OSPF Type-3 (LSR)

OSPF Type-4 (LSU)

OSPF Type-5 (LSAck)

Purpose of OSPF hello packet

- Discover OSPF neighbors & establish adjacencies
- Advertise guidelines on which routers must agree to become neighbors
- Used by multi-access networks to elect
  - a Designated Router (DR) :
    - DR is responsible for updating all other OSPF routers
  - a Backup Designated Router (BDR)
    - BDR router takes over DR's responsibilities if DR fails

LSU and LSA

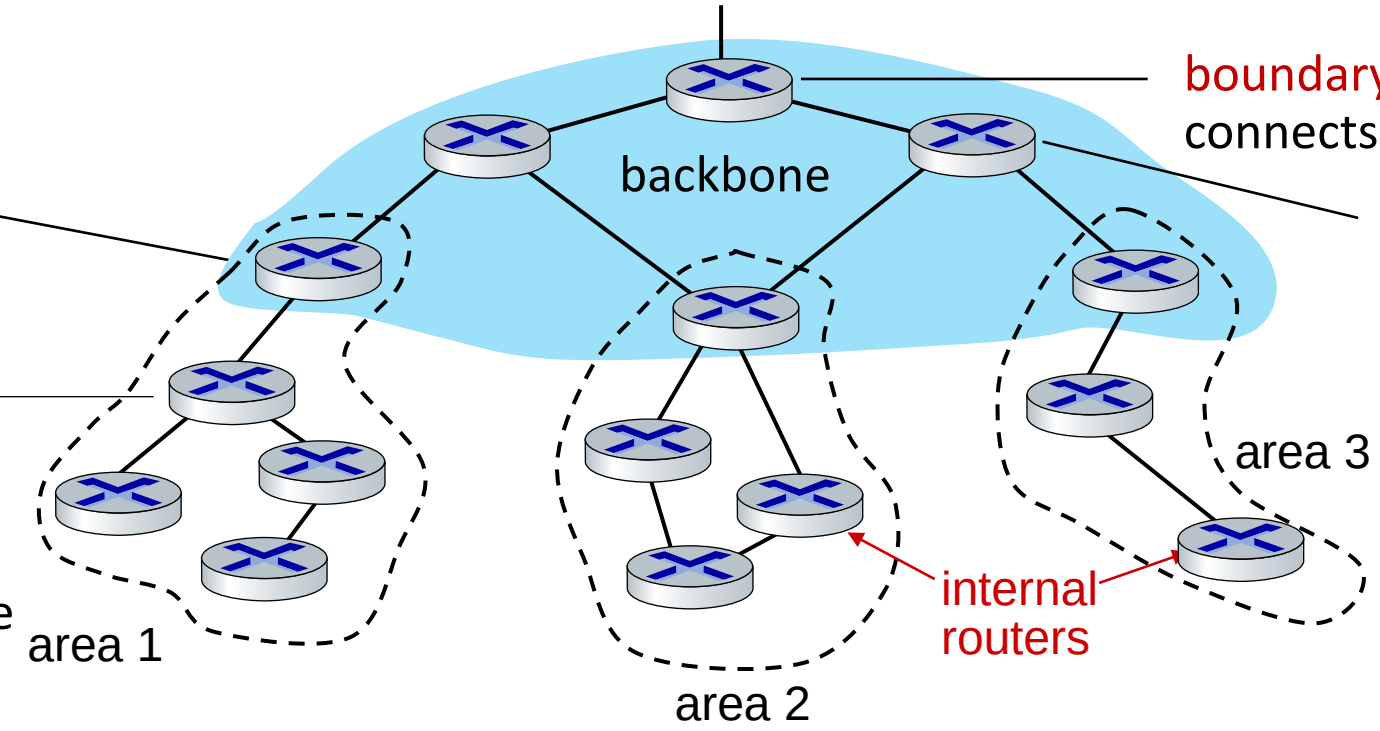
- Purpose of a Link State Update (LSU)
  - Used to deliver link state advertisements (LSA)
- Purpose of a Link State Advertisement (LSA)
  - Contains information about neighbors path costs
- The acronyms LSA and LSU are often used interchangeably.
- An LSU contains one or more LSAs
- LSAs contain route information for destination networks

Hierarchical OSPF

- **two-level hierarchy**: local area, backbone.
  - link-state advertisements (LSA) flooded only in area, or backbone
  - each node has detailed area topology; only knows direction to reach other destinations

area border routers (ABR):
"summarize" distances to destinations in own area, advertise in backbone

local routers:
- flood LS in area only
- compute routing within area
- forward packets to outside via area border router

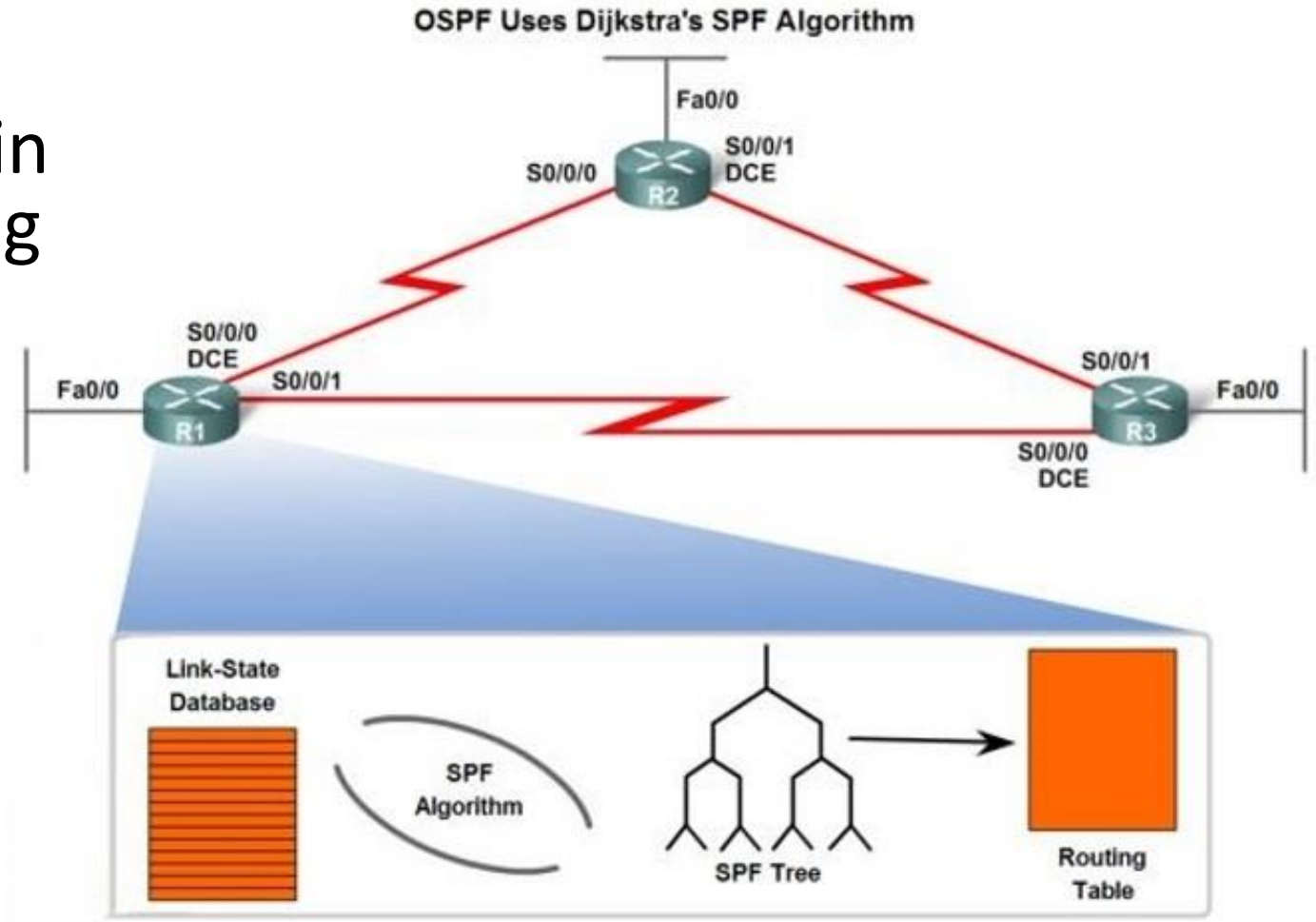


boundary router:
connects to other ASes

backbone router:
runs OSPF limited to backbone

OSPF Algorithm

- OSPF routers build & maintain link-state database containing LSA received from other routers
  - Information found in database is utilized upon execution of Dijkstra SPF algorithm
  - SPF algorithm used to create SPF tree
  - SPF tree used to populate routing table

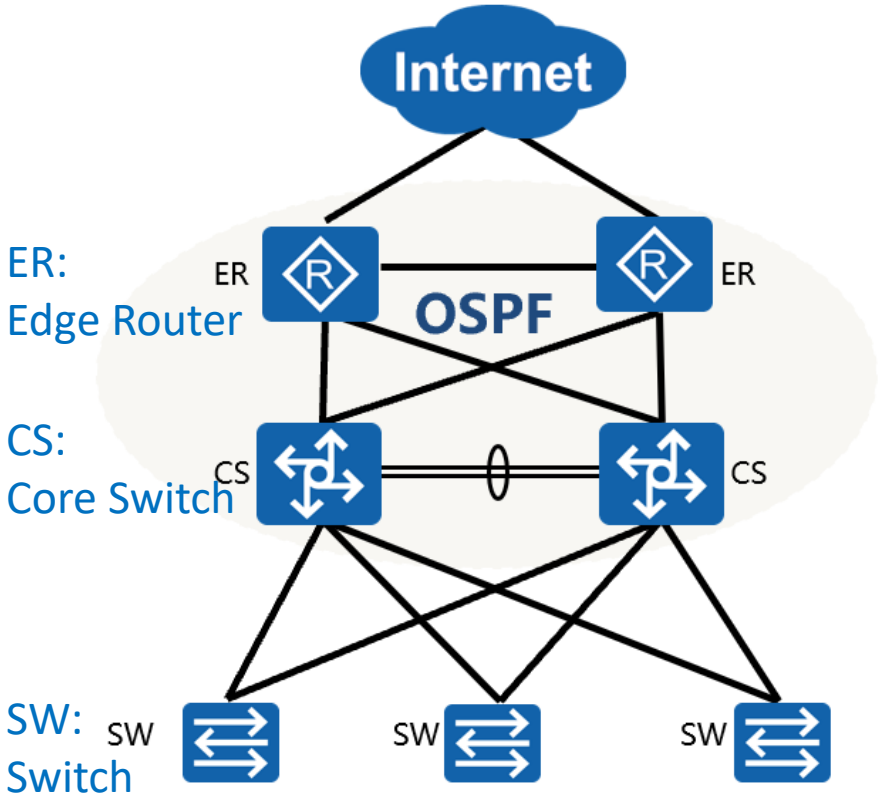


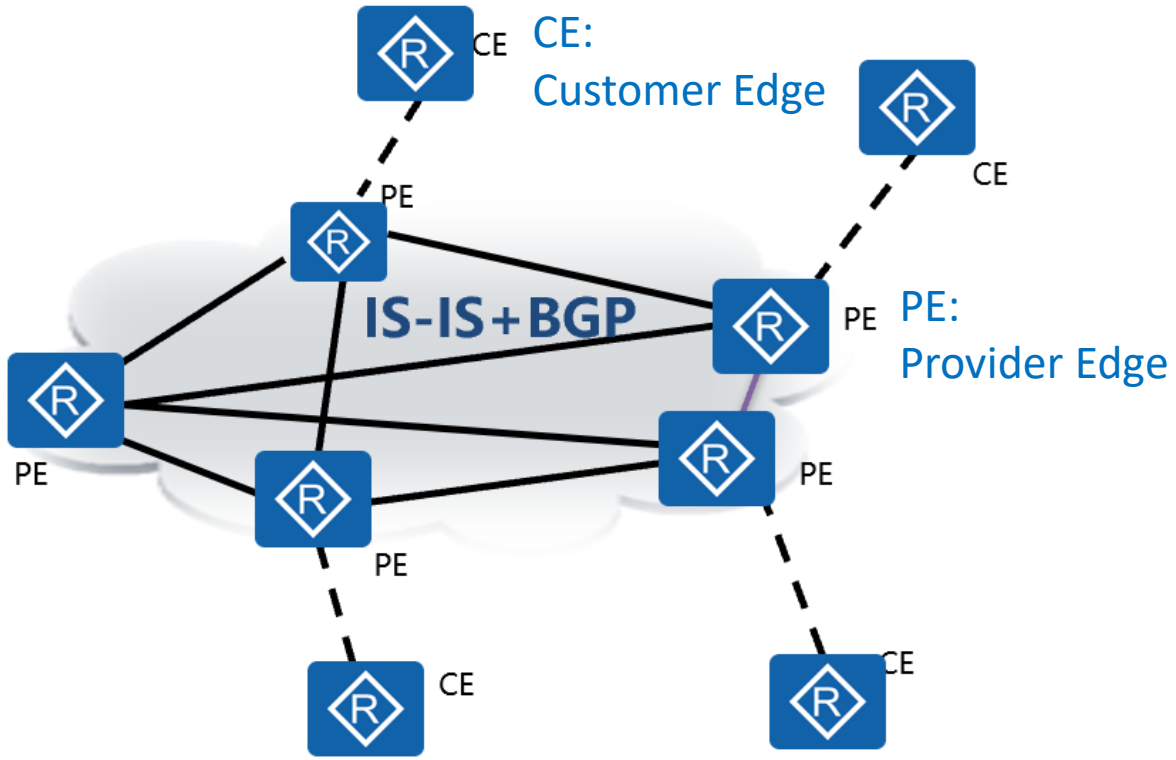
IS-IS protocol

Intermediate System-to-Intermediate System

- Similar to OSPF
- IGP
- Using SPF algorithm
- A dynamic routing protocol, which was initially designed by ISO for CLNP (ConnectionLess Network Protocol)
  - CLNP network has become obsolete, however, IS-IS protocol has been still kept due to its excellent features.
- Through expansion and modification, IS-IS can be applied in both TCP/IP and OSI environment, forming Integrated IS-IS or Dual IS-IS.
  - Integrated IS-IS can be mainly applied in MAN and carrier network.

Application Scenarios





Campus Network

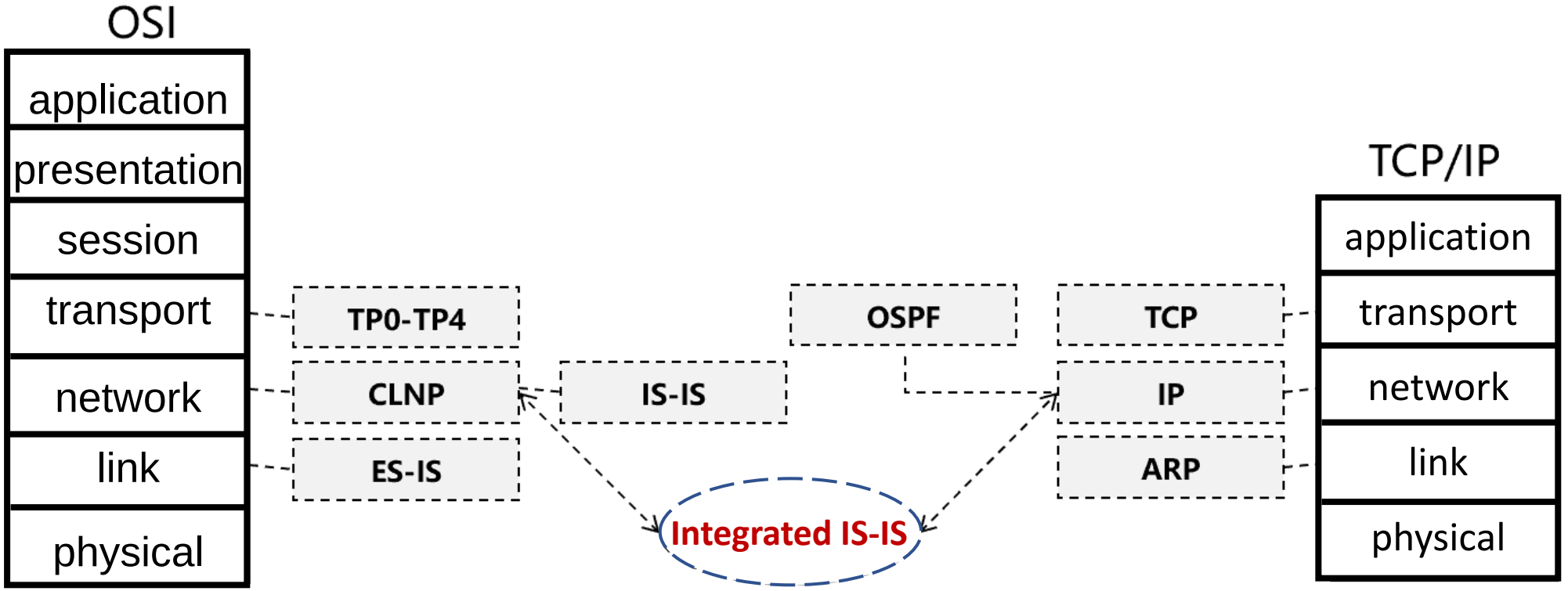
- Various regions, changeable strategies and fine scheduling

Backbone Network

- Flat regions, rapid convergence and huge carrying capacity

https://blog.csdn.net/qq_46254436

IS-IS evolution



Integrated IS-IS features

- Support both CLNP and IP network
- Work on data link layer

OSPF features

- Only support IP network at present
- Work on IP layer

https://blog.csdn.net/qq_46254436

IS-IS basic terminology

- **IS-IS** works on data link layer (layer 2)
- Advantages: Working on layer 2 can avoid more attacks compared with working on layer 3 (network layer)

Abbr.	OSI terminology	IETF terminology
IS	Intermediate System	Router
ES	End System	Host
DIS	Designated Intermediate System	Designated Router (DR) in OSPF
SysID	System ID	Router ID in OSPF
LSP	Link State PDU (Packet Data Units)	OSPF LSA (Link State Advertisement)
IIH	IS-IS Hello PDU	OSPF Hello packet
PSNP	Partial Sequence Number PDU	OSPF LSR (Link State Request) or LSAck
CSNP	Complete Sequence Number PDU	OSPF DD packet

Comparison between IS-IS and OSPF

ITEMS	IS-IS	OSPF
Network topologies	Few (two types) ❶Broadcast and ❷general topology Or ❶point-to-point and ❷broadcast in Cisco Routers	More Types ❶ point-to-point ❷ point-to-multipoint ❸ Broadcast ❹ NBMA ❺ virtual links
LSDB maintenance	Simple LSP (Link State PDU) Itself is a datagram	Various LSA (Link State Advertisements) encapsulated in OSPF header and IP header
Routing calculation (SPF)	Intra-area: Level 1 SPF Inter-area: Level 2 SPF	Intra-area: SPF Inter-area: distance vector algorithm
Two-level hierarchy	Area border is on link layer L1 or/and L2, two levels are not necessary.	Area border is on routers There must be area 0 (backbone).
Overhead	Complex	Simple
Routing convergence rate	Very fast	fast
Scalability	Strong	Normal
Load capacity	Super high	high
Authentication	Simple + MD5, SHA	Simple + MD5, SHA

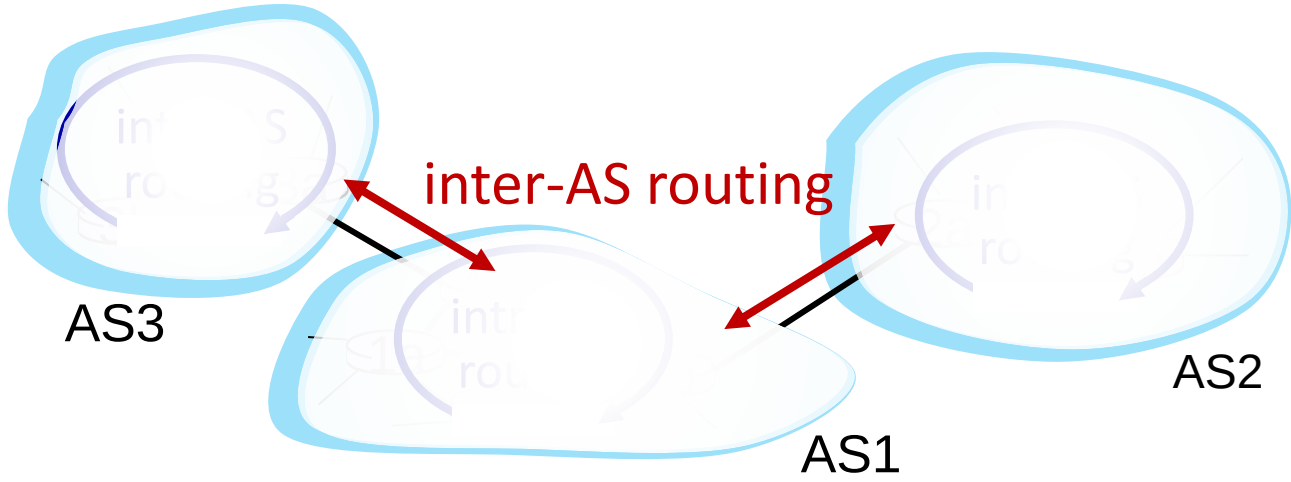
Network layer: "control plane" roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
  - SNMP
  - NETCONF/YANG

Interconnected ASes



intra-AS (aka "intra-domain"): routing among routers *within same AS ("network")*

inter-AS (aka "inter-domain"): routing *among* AS'es

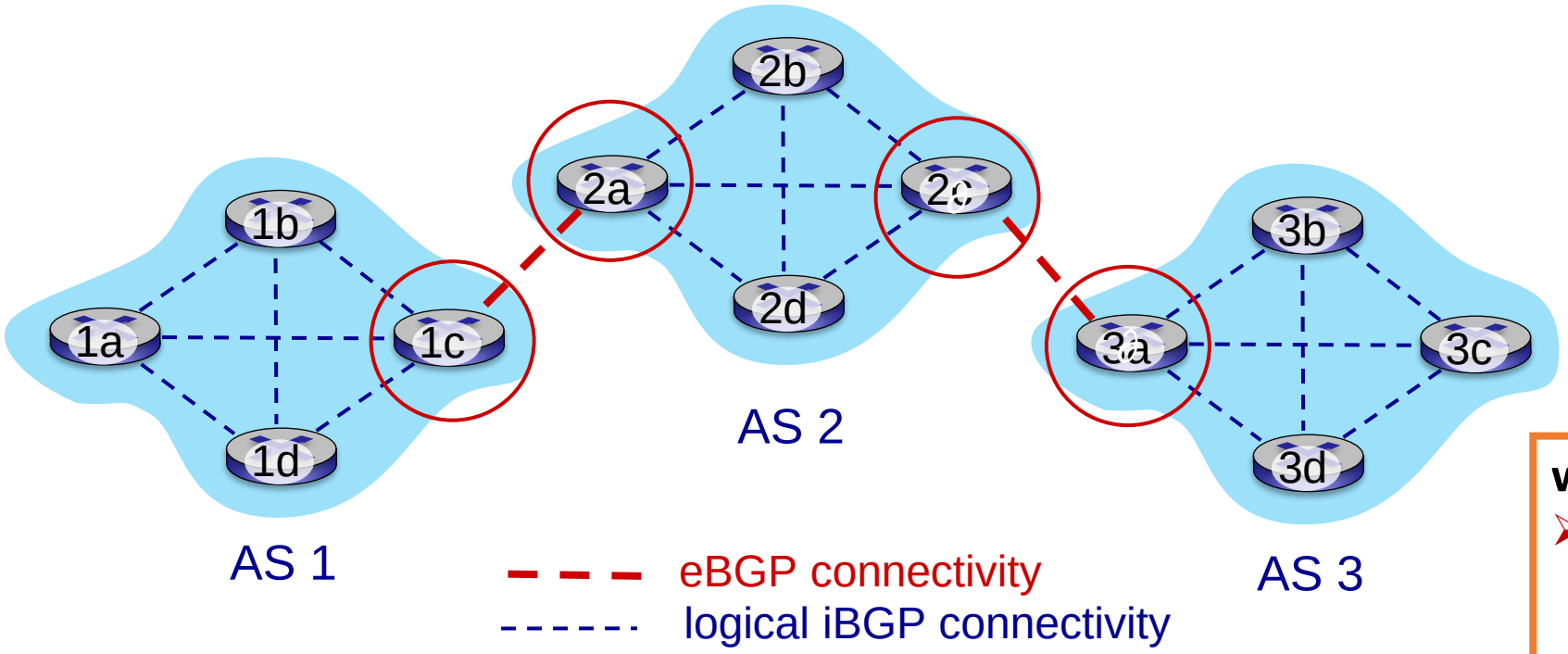
Internet inter-AS routing: BGP



- BGP (Border Gateway Protocol): *the* de facto inter-domain routing protocol
  - In BGP, packets are not routed to a specific destination address, but instead to CIDRized prefixes
  - A router's forwarding table will have entries of the form (*x*, *I*), where *x* is a prefix (such as 138.16.68/22) and *I* is an interface number for one of the router's interfaces.
- allows subnet to advertise its existence, and the destinations it can reach, to rest of Internet: *"I am here, here is who I can reach, and how"*

eBGP, iBGP connections

The BGP protocol plays a vital role in the Internet, but its security is fragile





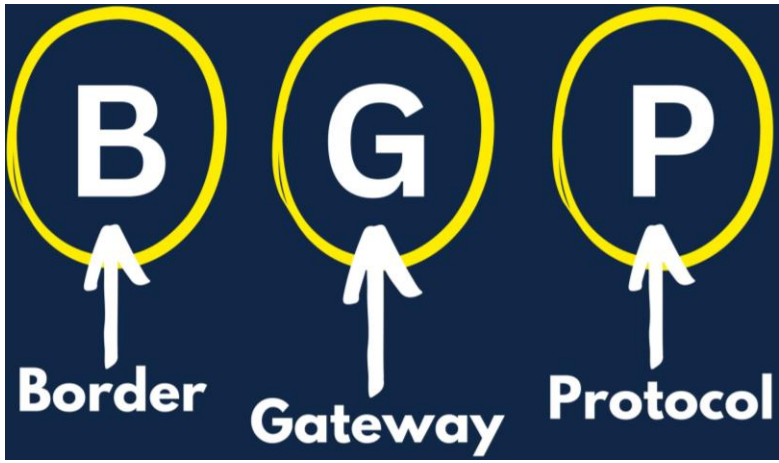
why fragile?

- Unconditionally receive or propagate routes from neighbouring Ases
- Lack of effective authentication mechanism for incoming IP prefix routes



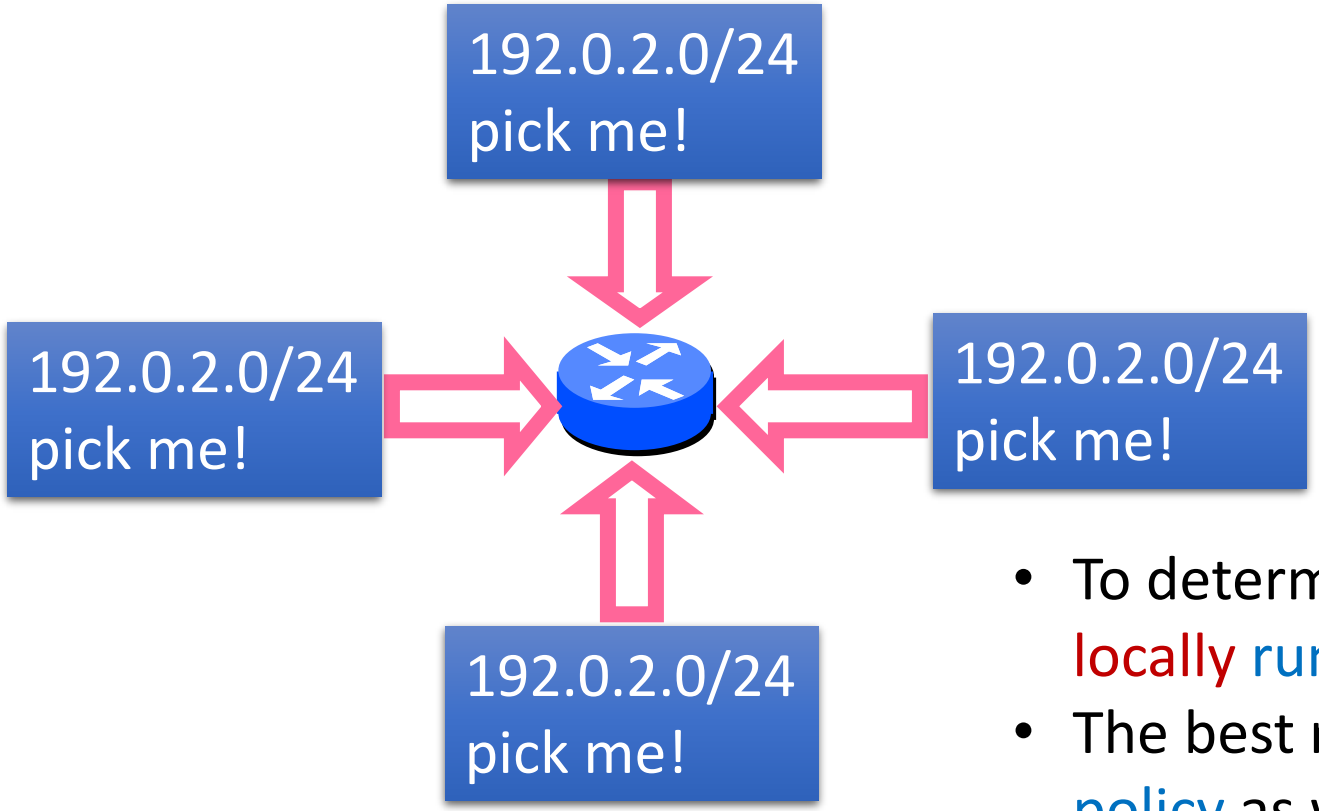
gateway routers run both eBGP and iBGP protocols

Advertising BGP Route Information



- In BGP, pairs of routers exchange routing information over semi-permanent TCP connections using port 179.
- Each such TCP connection, along with all the BGP messages sent over the connection, is called a **BGP connection**.
- BGP provides each AS a means to:
  - eBGP (external BGP): obtain subnet reachability information from neighboring ASes
  - iBGP (internal BGP): propagate reachability information to all AS-internal routers.
  - determine "good" routes to other networks based on reachability information and *policy*

Attributes are Used to Select Best Routes



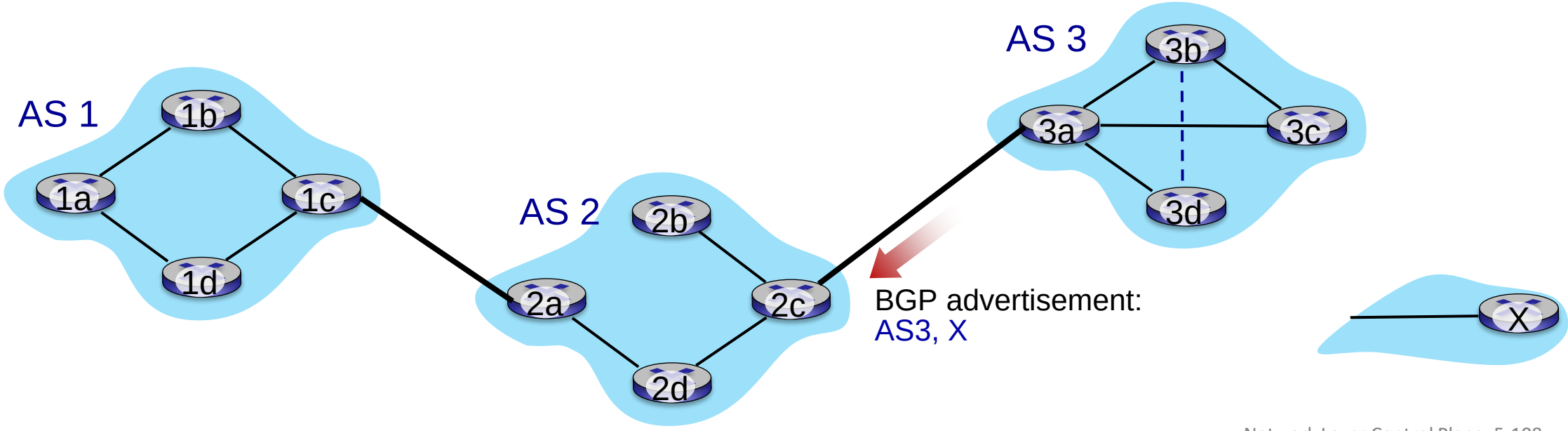
Given multiple routes to the same prefix, a BGP speaker must pick at most ONE best route

(Note: it could reject them all!)

- To determine the best route, the router will locally run a BGP route-selection procedure.
- The best route will be determined based on policy as well as the reachability information.

BGP basics

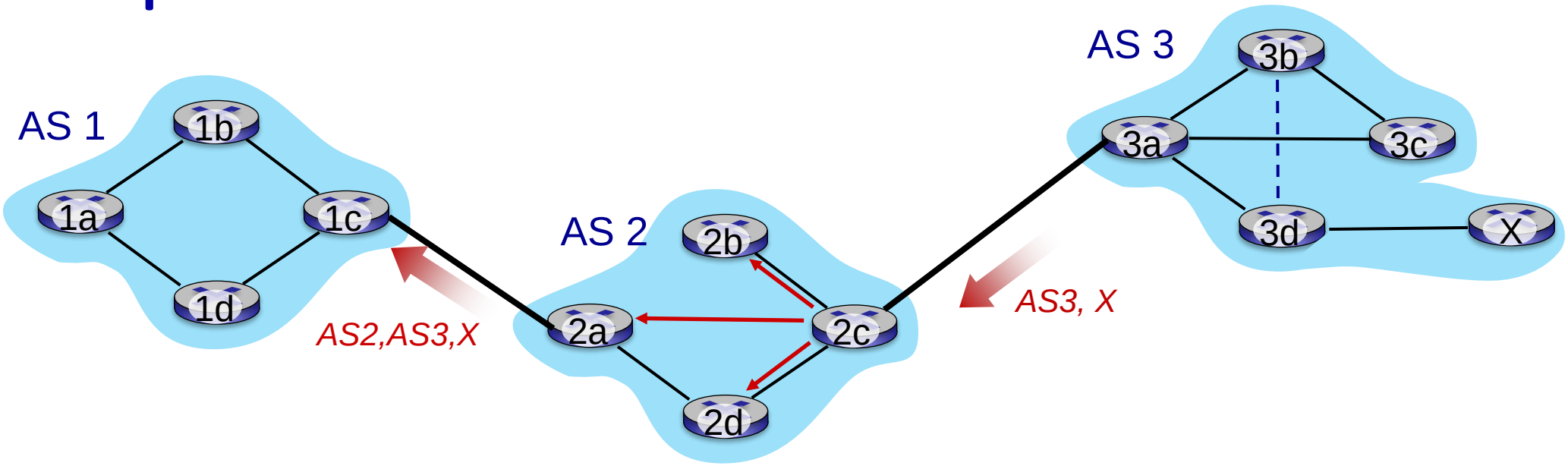
- BGP session: two BGP routers ("peers") exchange BGP messages over semi-permanent TCP connection:
  - advertising *paths* to different destination network prefixes
  - BGP is a "path vector" protocol
- when AS3 gateway 3a advertises path AS3,X to AS2 gateway 2c:
  - AS3 *promises* to AS2 it will forward datagrams towards X



Path attributes and BGP routes

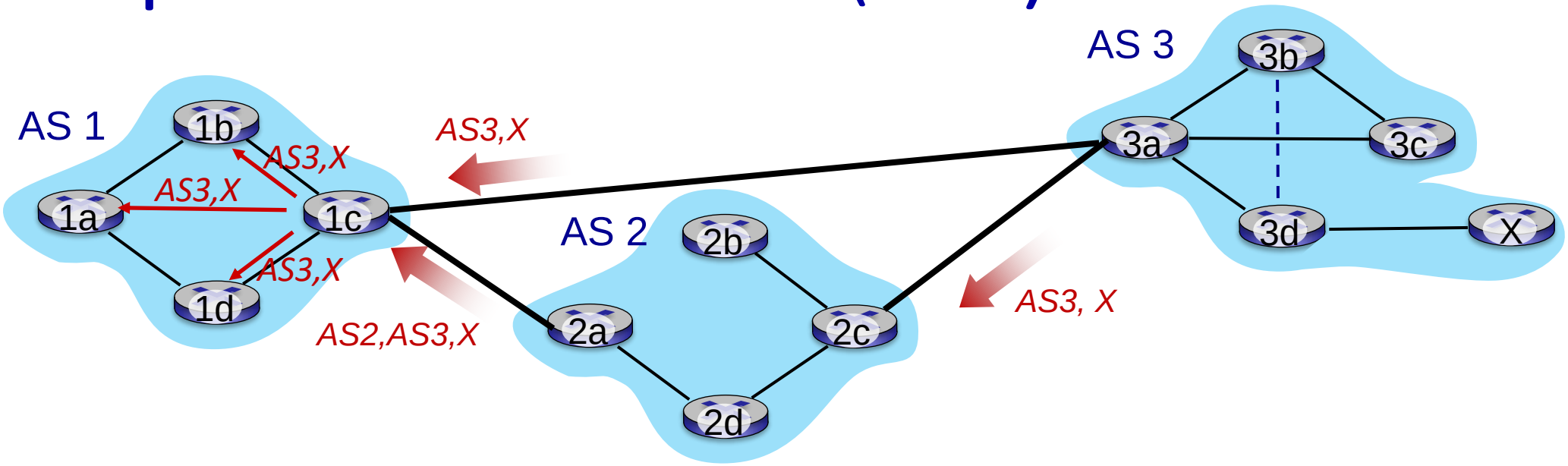
- BGP advertised route: prefix + attributes
  - prefix: destination being advertised
  - two important attributes:
    - AS-PATH: list of ASes through which prefix advertisement has passed
    - NEXT-HOP: indicates specific internal-AS router to next-hop AS
- policy-based routing:
  - gateway receiving route advertisement uses *import policy* to accept/decline path (e.g., never route through AS Y).
  - AS policy also determines whether to *advertise* path to other neighboring ASes

BGP path advertisement



- AS2 router 2c receives path advertisement AS3,X (via eBGP) from AS3 router 3a
- based on AS2 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all AS2 routers
- based on AS2 policy, AS2 router 2a advertises (via eBGP) path AS2, AS3, X to AS1 router 1c

BGP path advertisement (more)



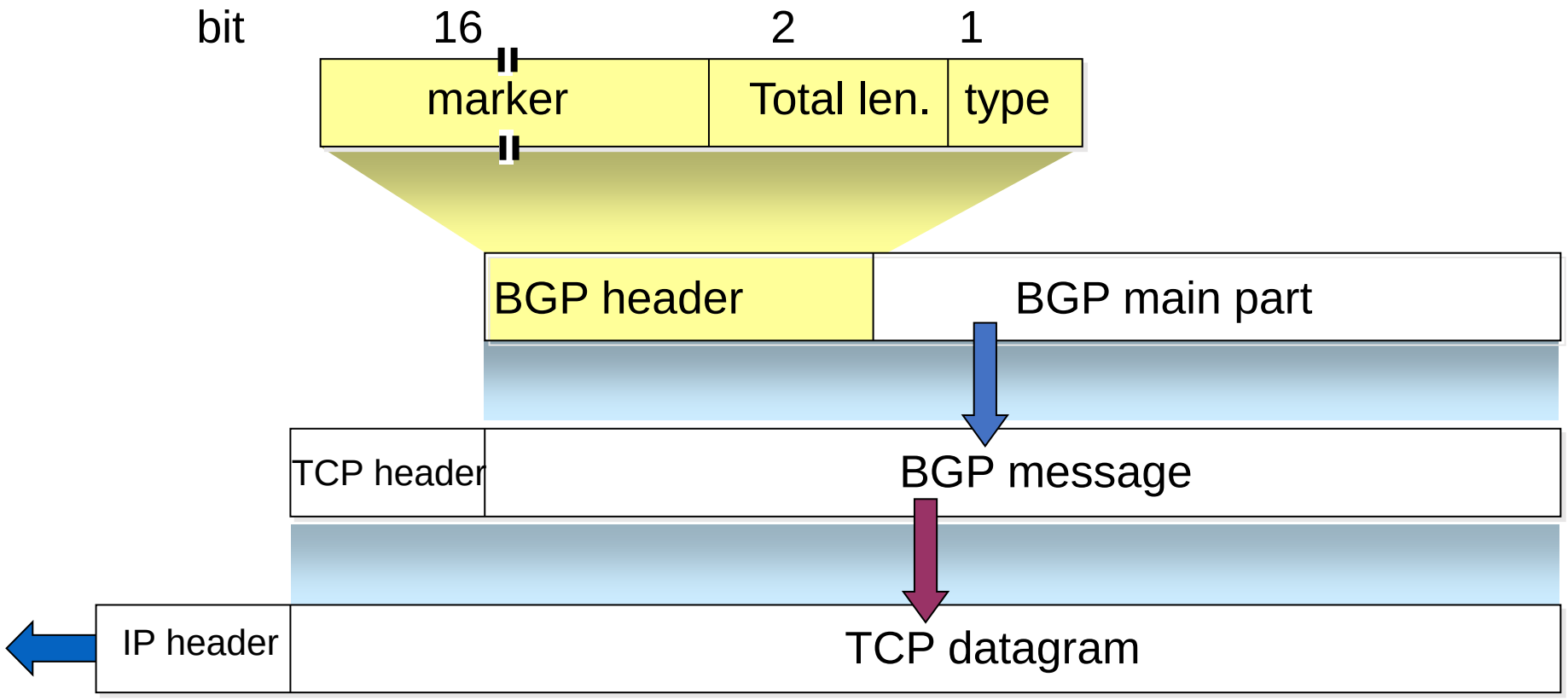
gateway router may learn about multiple paths to destination:

- AS1 gateway router 1c learns path *AS2,AS3,X* from 2a
- AS1 gateway router 1c learns path *AS3,X* from 3a
- based on *policy,* AS1 gateway router 1c chooses path *AS3,X* and advertises path within AS1 via iBGP

BGP messages

- BGP messages exchanged between peers over TCP connection
- BGP messages:
  - OPEN (Type = 1): opens TCP connection to remote BGP peer and authenticates sending BGP peer
  - KEEPALIVE (Type = 2): keeps connection alive in absence of UPDATES; also ACKs OPEN request
  - UPDATE (Type = 3): advertises new path (or withdraws old)
  - NOTIFICATION (Type = 4): reports errors in previous msg; also used to close connection

BGP message header



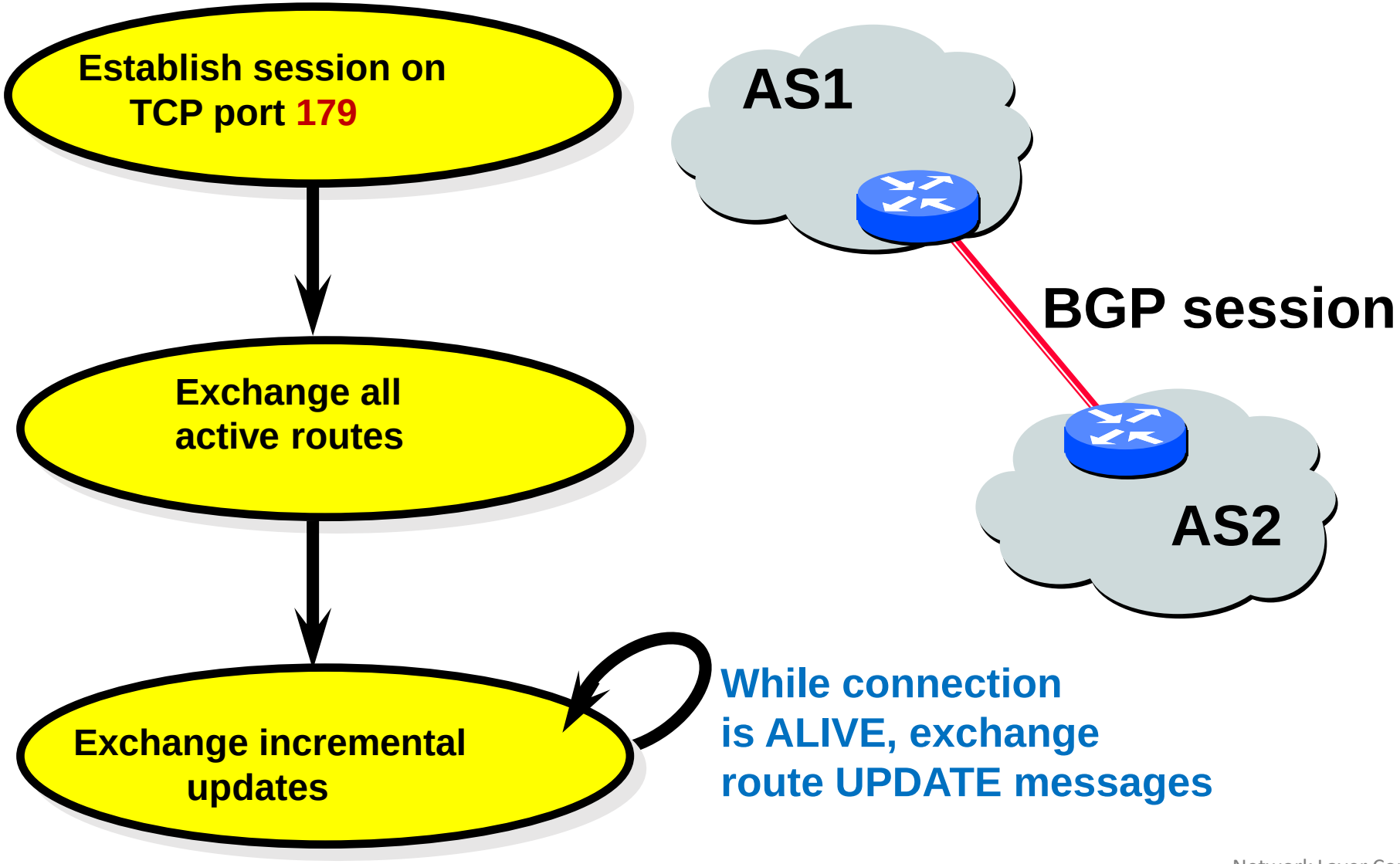
BGP runs over TCP

The corresponding TCP port number is 179

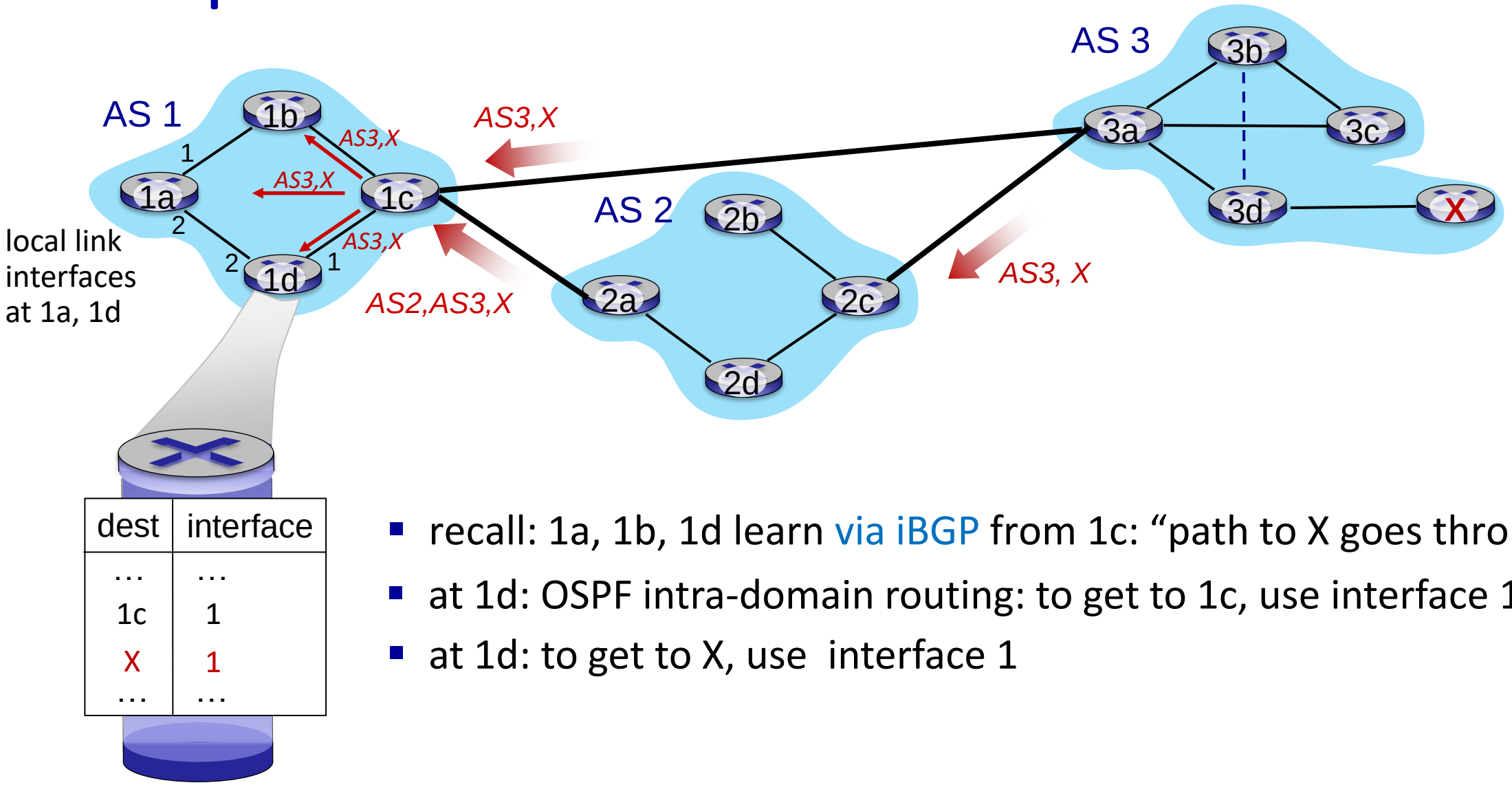
Comparison among various protocols

Feature	RIP	EIGRP	OSPF	IS-IS	BGP
Origin	Distance-vector, from Xerox (RIPv1), standardized by IETF (RIPv2)	Cisco Proprietary (now partially open)	IETF developed for IP networks	ISO developed for OSI (CLNP), later extended for IP	IETF designed for the Internet core
Scope	IGP	IGP	IGP	IGP	EGP
Protocol Layer	**Application Layer** (uses UDP port 520)	Transport/Network Layer (uses IP protocol 88)	Network Layer (uses IP protocol 89)	Above Data Link Layer (directly encapsulated, protocol 0xFEFE/0x22F3)	**Application Layer** (uses TCP port 179)
Transport Protocol	**UDP**	IP (RTP)	IP	Directly encapsulated in Data-Link frame	**TCP**
Primary Goal	Simple to use, path selection based on **hop count**	Fast convergence, efficient bandwidth use, loop avoidance	Fast convergence in large networks, path selection based on cost (bandwidth)	Stable and scalable for large networks (esp. ISP backbones)	Policy control, stability, and scalable interconnection between different ASs

BGP Operations (Simplified)

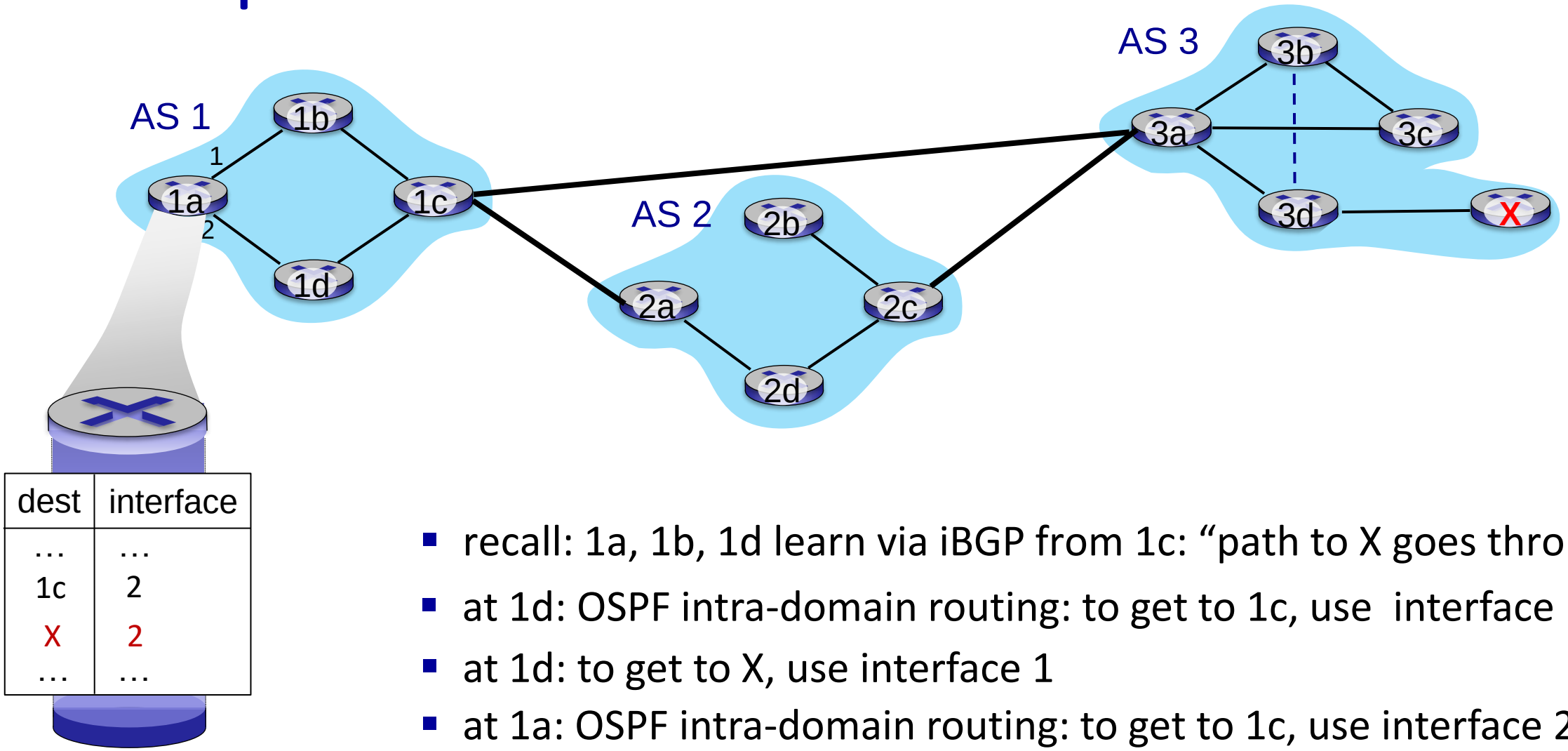


BGP path advertisement



- recall: 1a, 1b, 1d learn via iBGP from 1c: "path to X goes through 1c"
- at 1d: OSPF intra-domain routing: to get to 1c, use interface 1
- at 1d: to get to X, use interface 1

BGP path advertisement



- recall: 1a, 1b, 1d learn via iBGP from 1c: “path to X goes through 1c”
- at 1d: OSPF intra-domain routing: to get to 1c, use interface 1
- at 1d: to get to X, use interface 1
- at 1a: OSPF intra-domain routing: to get to 1c, use interface 2
- at 1a: to get to X, use interface 2

Comparison between eBGP and iBGP

Attribute	iBGP	eBGP
Scope	Within a single AS	Between different ASes
Route advertisement	Limited to iBGP peers	Broadcast to all peers
Topology	Requires full mesh (or Route Reflector)	No full mesh needed
Administrative Distance	200	20
Loop Prevention	BGP split horizon	AS Path

- In iBGP, all routers traditionally form a full mesh, directly peering with each other.
- Alternatively, a route reflector configuration can be used
  - where some routers serve as route reflectors to reduce the need for full-mesh connections.

Interdomain Loop Prevention

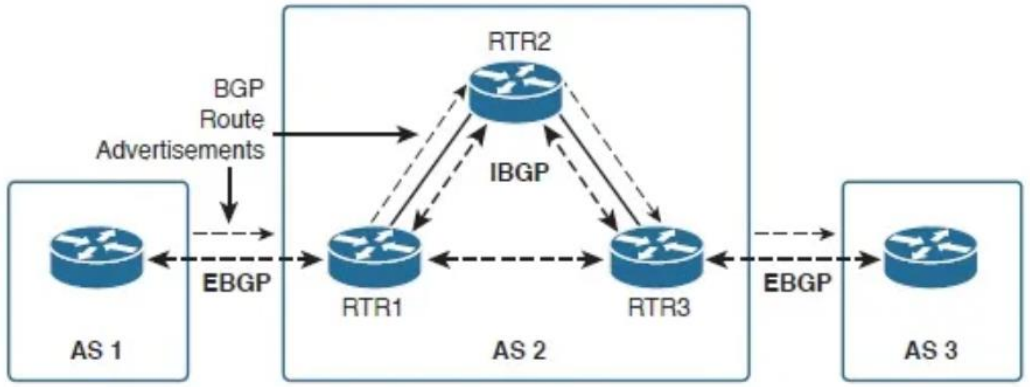


Figure 1-7 *A Route Advertisement from AS1 to AS3 Must Physically Pass Through Three Routers in AS2*

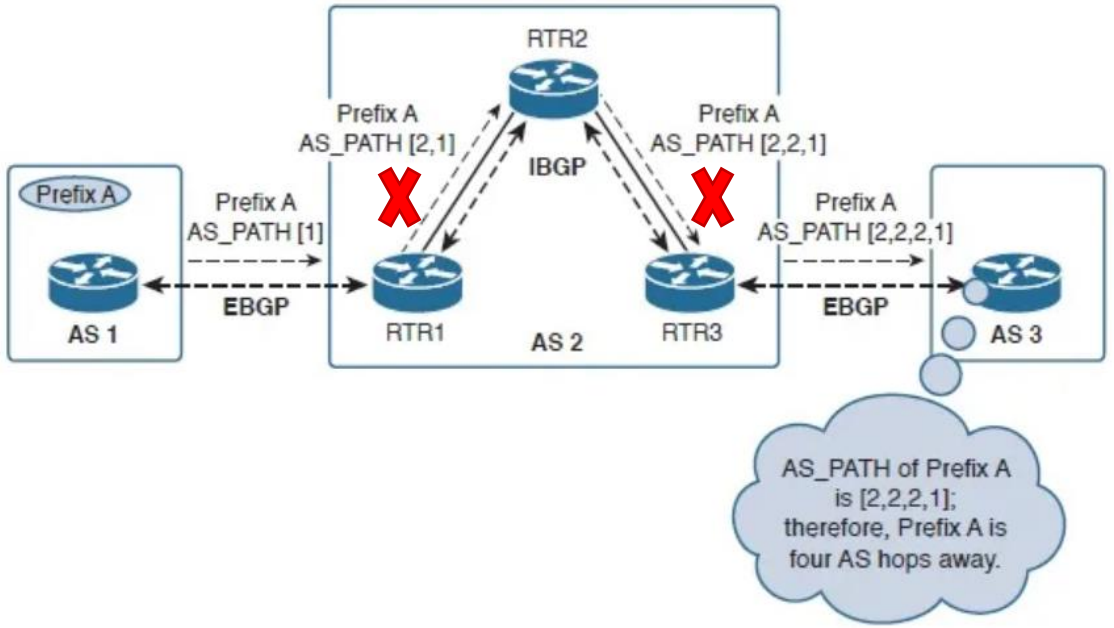


Figure 1-8 *If Each Router in AS2 Added Its AS number to the AS_PATH, the Router in AS3 Would Incorrectly Conclude That Prefix A Is Four AS Hops Away, Rather Than Two.*

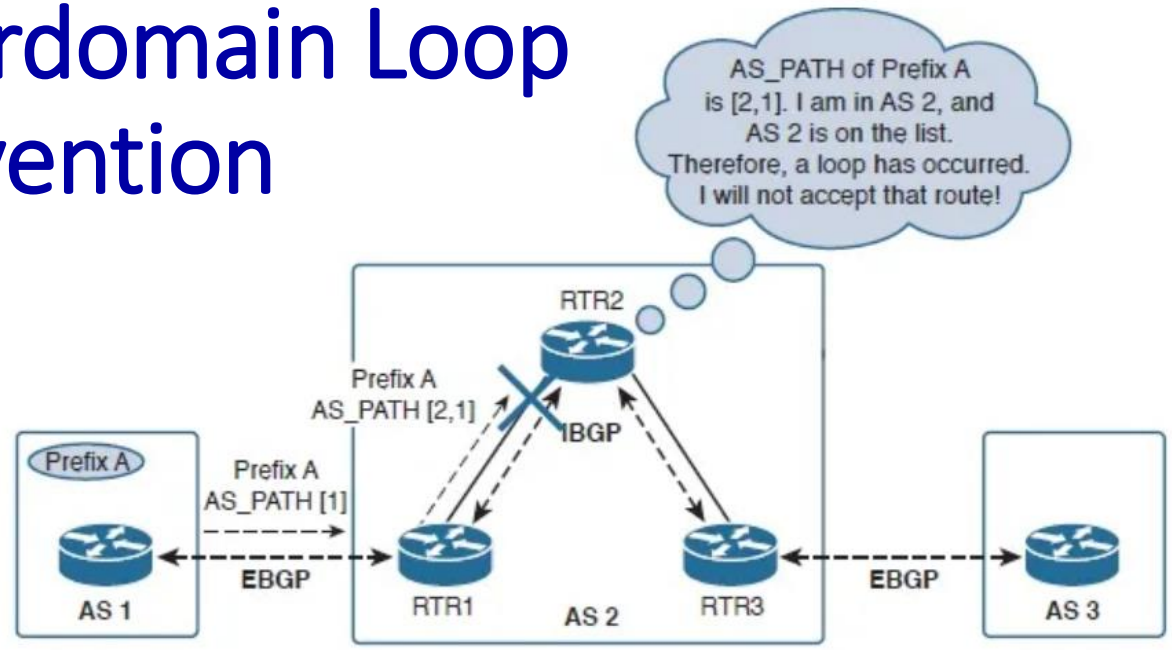


Figure 1-9 *If RTR1 Added Its AS Number to the AS_PATH, RTR2. Being in the Same AS, Would Incorrectly Conclude That a Loop Had Occurred and Drop the Route*

The solution to these problems is a special rule for IBGP:

A router adds its AS number to a route's AS_PATH *only* when the route is sent to an eBGP neighbor. The AS number is **NOT** added to routes sent to an iBGP neighbor.

Interdomain Loop Prevention

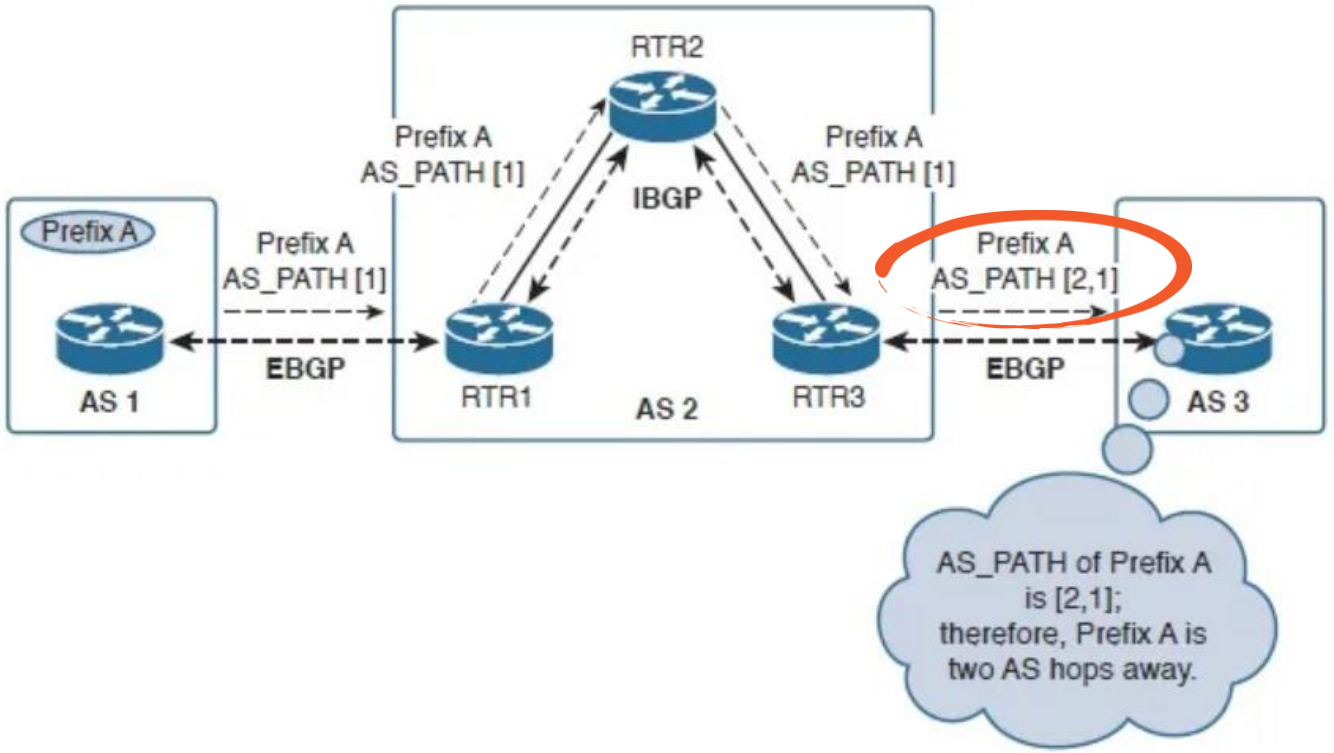


Figure 1-10 *The Problems of Figures 1-8 and 1-9 Are Prevented by Adding Only an AS Number to the AS_PATH When the Route Is Sent to an EBGP Neighbor*

- BGP's method of detecting and avoiding routing loops;
- However, the AS_PATH is meaningless within the scope of a single AS.
- What if a routing loop does exist within the AS? How to avoid it?
  - A loop-free topology is required.

Interdomain Loop Prevention

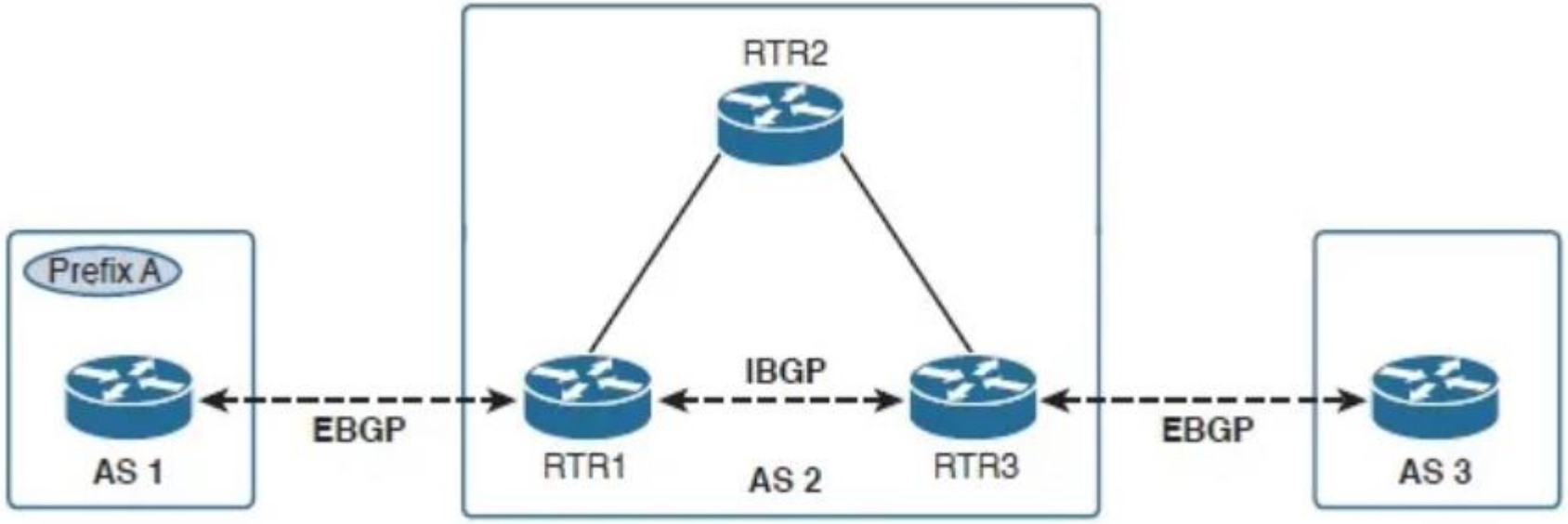


Figure 1-11 *Establishing an IBGP Session Only Between the Edge Routers Creates a Loop-Free IBGP Topology*

A **loop-free topology** is required.

- One of the keys to this solution is that BGP sessions run over TCP (unicast)
- No requirement that the two points of the session physically connect.
- The iBGP session can be established directly between the edge routers

How are BGP routing loops avoided in a complex topology?

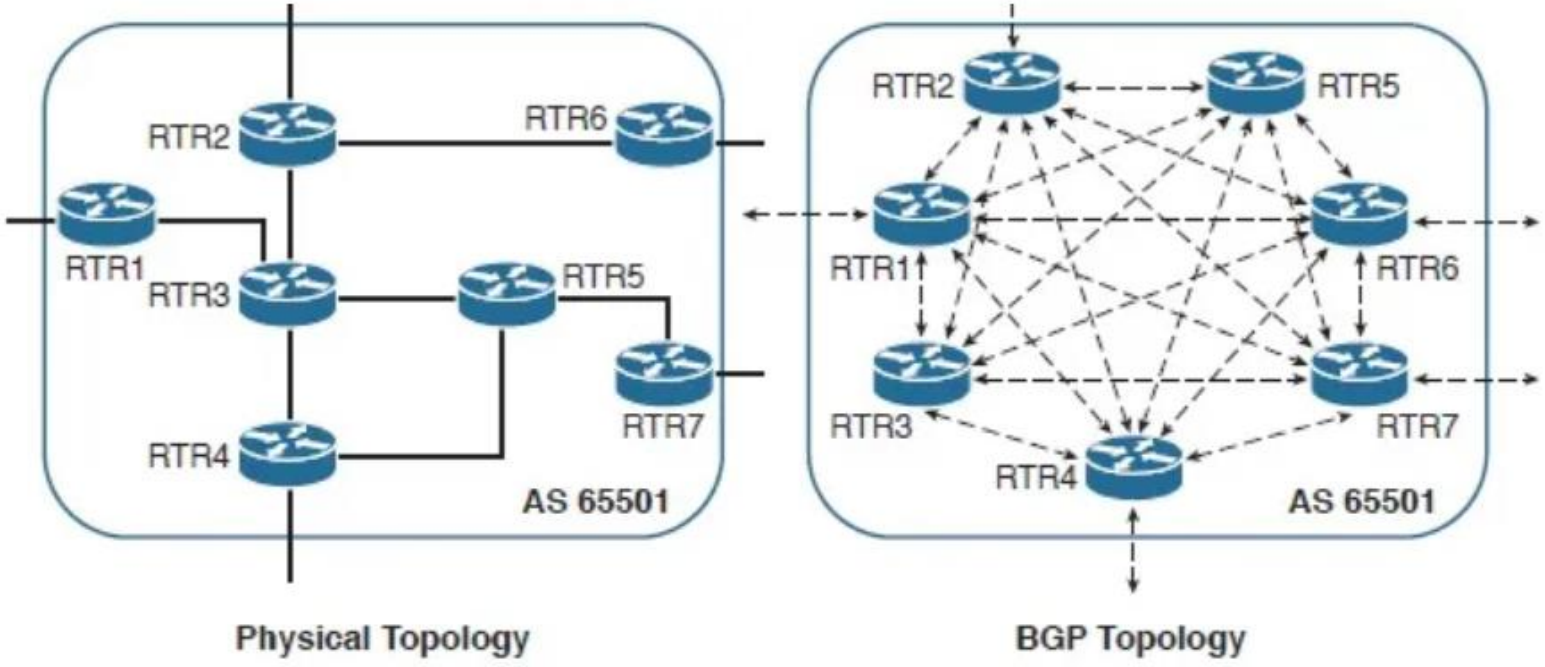
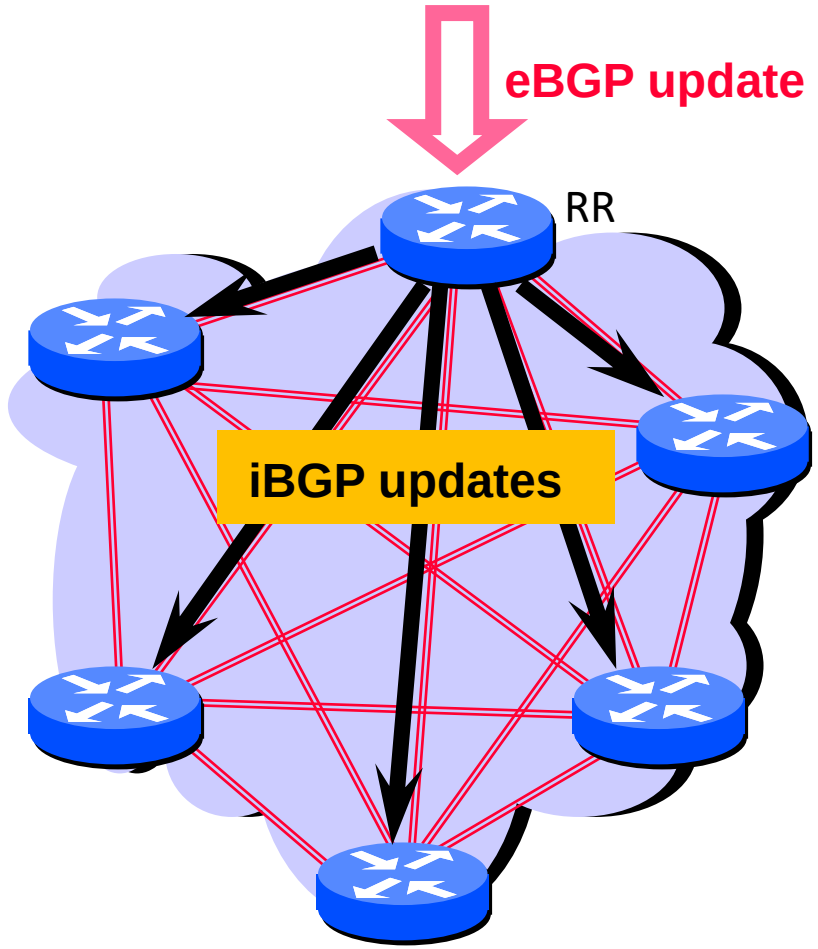


Figure 1-15 *The Requirement That IBGP Be Fully Meshed Can Result in a BGP Topology That Is Significantly Different from the Physical Topology*

The **objective** of the full IBGP mesh is to ensure that all routers within the AS have the information they need to forward packets to the correct next hop.

iBGP Peers Within an AS Must be Fully Meshed



I-BGP neighbors do not announce routes received via I-BGP to other I-BGP neighbors

- Fully mesh can be provided by Router Reflector (RR)
- iBGP is needed to avoid routing loops within an AS
- Problem: Injecting external routes into IGP (e.g., OSPF) does not scale and causes BGP policy information to be lost
- I-BGP can be used to disseminate BGP routes to all routers in AS
- BGP route + IGP route suffice to create forwarding table
- BGP does not provide “shortest path” routing
- Is iBGP an IGP? NO!

Why different Intra-, Inter-AS routing ?

metrics	Inter-AS routing among ASs	Intra-AS routing within an AS
Policy	dominant issue ★	less of an issue △
Scale	a critical issue ★	less of a concern △
performance	of secondary concern △	important ★

policy:

- inter-AS: admin wants control over how its traffic routed, who routes through its network
- intra-AS: single admin, so policy less of an issue

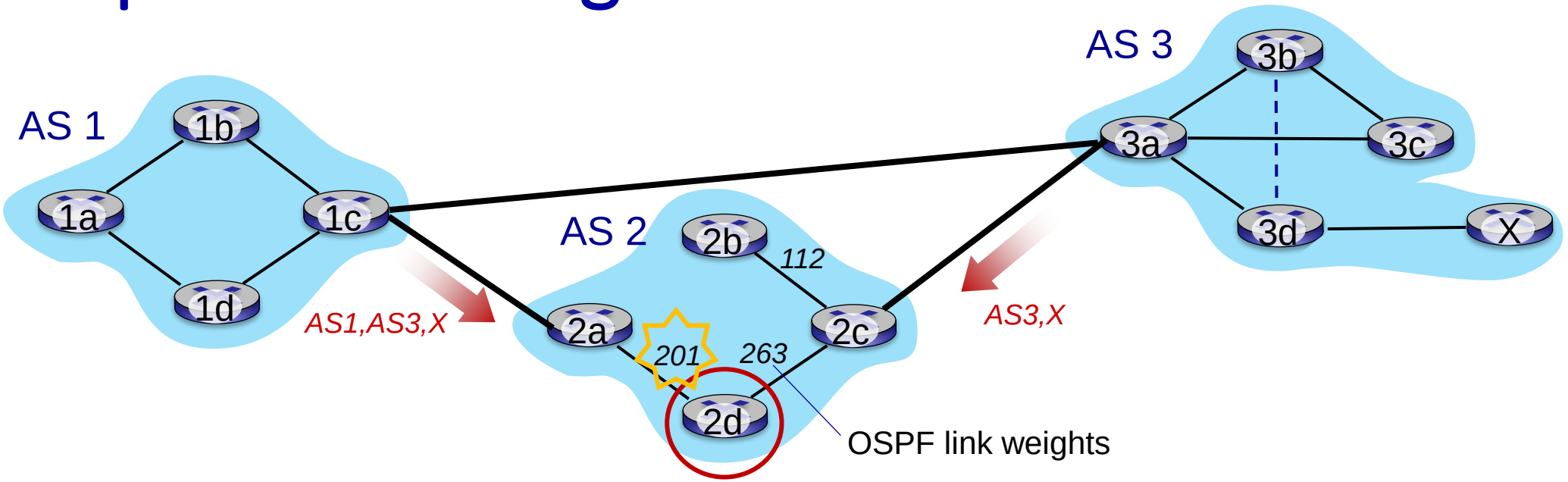
scale:

- hierarchical routing saves table size, reduced update traffic
  - If a single ISP becomes too large, it is always possible to divide it into two ASs and perform inter-AS routing between the two new ASs.

performance:

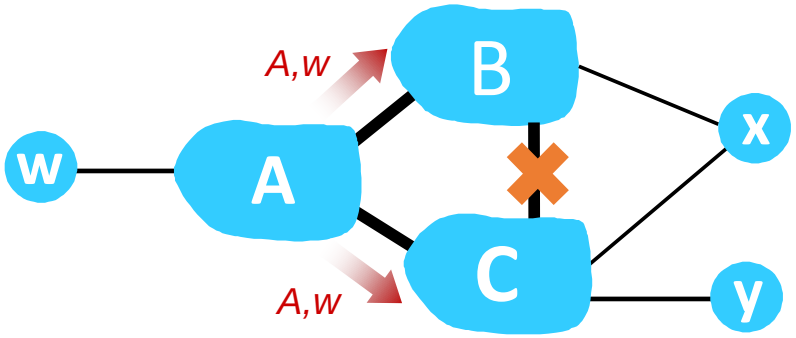
- intra-AS: can focus on performance
- inter-AS: policy dominates over performance

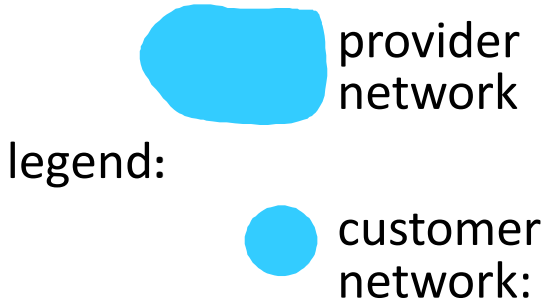
Hot potato routing



- 2d learns (via iBGP) it can route to X via 2a or 2c
- hot potato routing: choose local gateway that has least *intra-domain* cost (e.g., 2d chooses 2a, even though more AS hops to *X*): don't worry about inter-domain cost!

BGP: achieving policy via advertisements



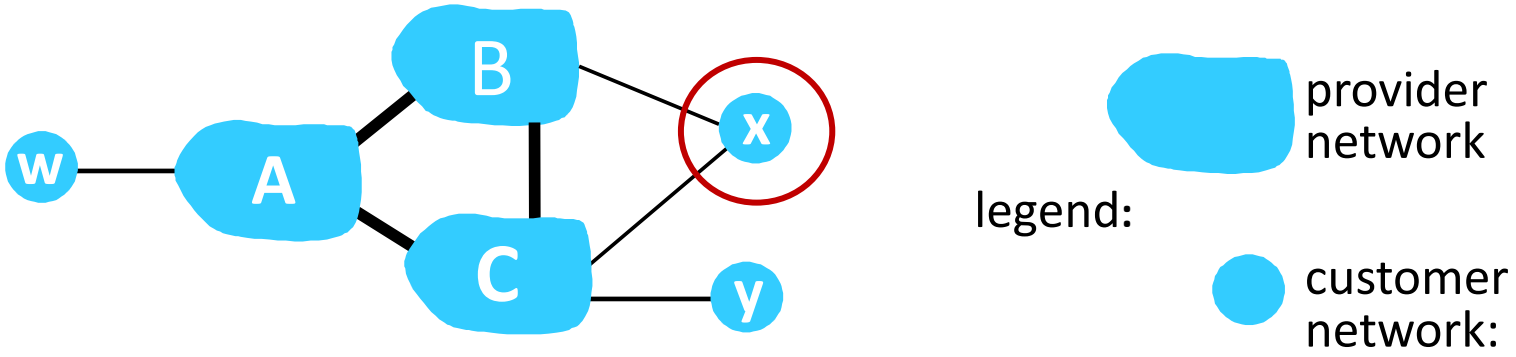


- A,B,C,x,w,y are ASes, not routers
- A,B,C are backbone provider networks
- x,w,y are stub networks and customers (of provider networks)
- Stub networks are typically no need for BGP

ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical "real world" policy)

- A advertises path Aw to B and to C
- B *chooses not to advertise* BAw to C!
  - B gets no "revenue" for routing CBAw, since none of C, A, w are B's customers
  - C does *not* learn about CBAw path
- C will route CAw (not using B) to get to w

BGP: achieving policy via advertisements (more)



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical "real world" policy)

- A,B,C are provider networks
- x,w,y are customer (of provider networks)
- x is dual-homed: attached to two networks
- *policy to enforce:* x does not want to route from B to C via x
  - .. so x will not advertise to B a route to C

BGP route selection

- router may learn about more than one route to destination AS, selects route based on:
  1. local preference value attribute: policy decision
  2. shortest AS-PATH
     - Distance vector algorithm
     - Distance metric: # AS hops, NOT # router hops
  3. closest NEXT-HOP router: hot potato routing
     - Least-cost path determined by intra-AS algorithm
  4. additional criteria

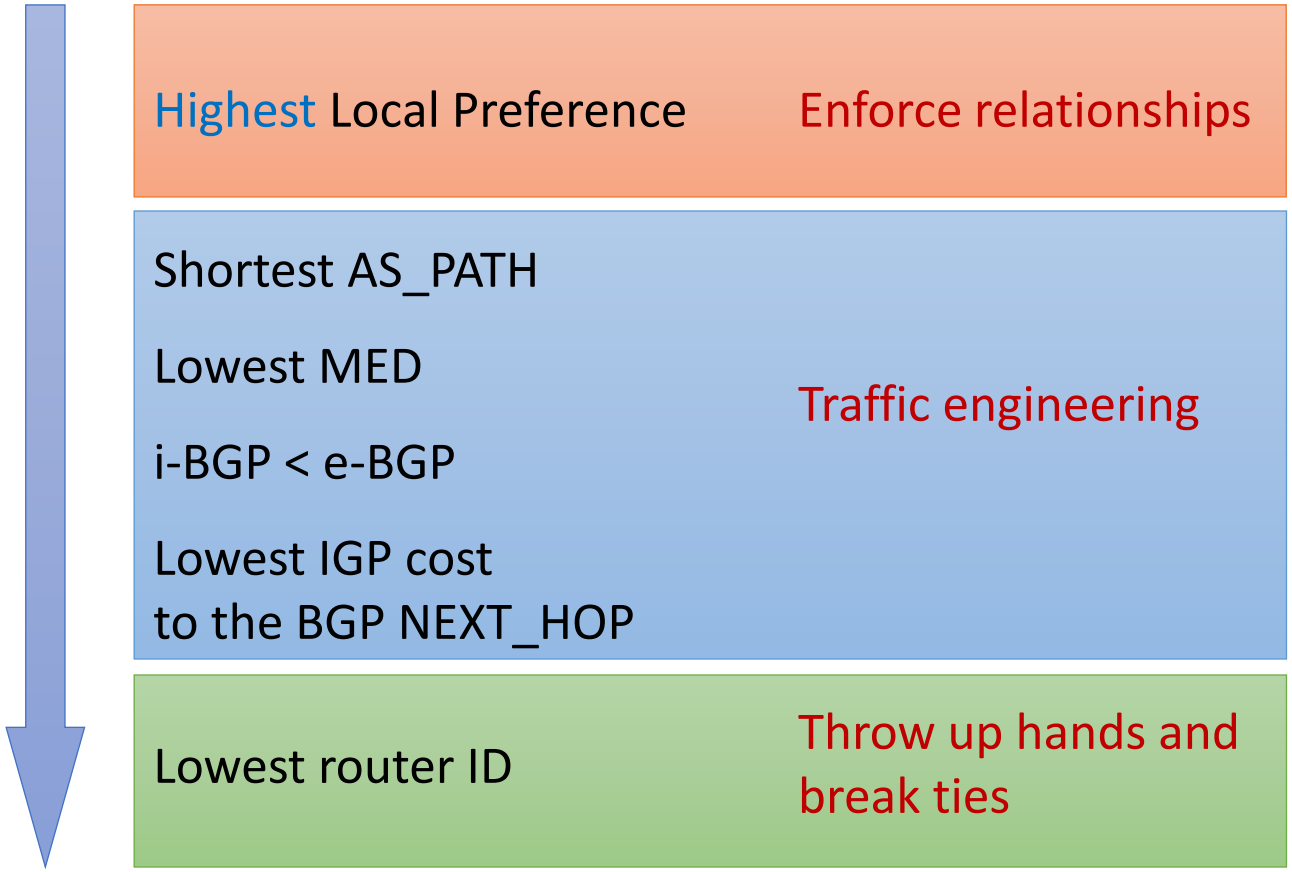
BGP Attributes

Value	Code	Reference
1	ORIGIN	[RFC1771]
2	AS_PATH	[RFC1771]
3	NEXT_HOP	[RFC1771]
4	MULTI_EXIT_DISC	[RFC1771]
5	LOCAL_PREF	[RFC1771]
6	ATOMIC_AGGREGATE	[RFC1771]
7	AGGREGATOR	[RFC1771]
8	COMMUNITY	[RFC1997]
9	ORIGINATOR_ID	[RFC2796]
10	CLUSTER_LIST	[RFC2796]
11	DPA	[Chen]
12	ADVERTISER	[RFC1863]
13	RCID_PATH / CLUSTER_ID	[RFC1863]
14	MP_REACH_NLRI	[RFC2283]
15	MP_UNREACH_NLRI	[RFC2283]
16	EXTENDED COMMUNITIES	[Rosen]
...		
255	reserved for development	

From IANA: http://www.iana.org/assignments/bgp-parameters

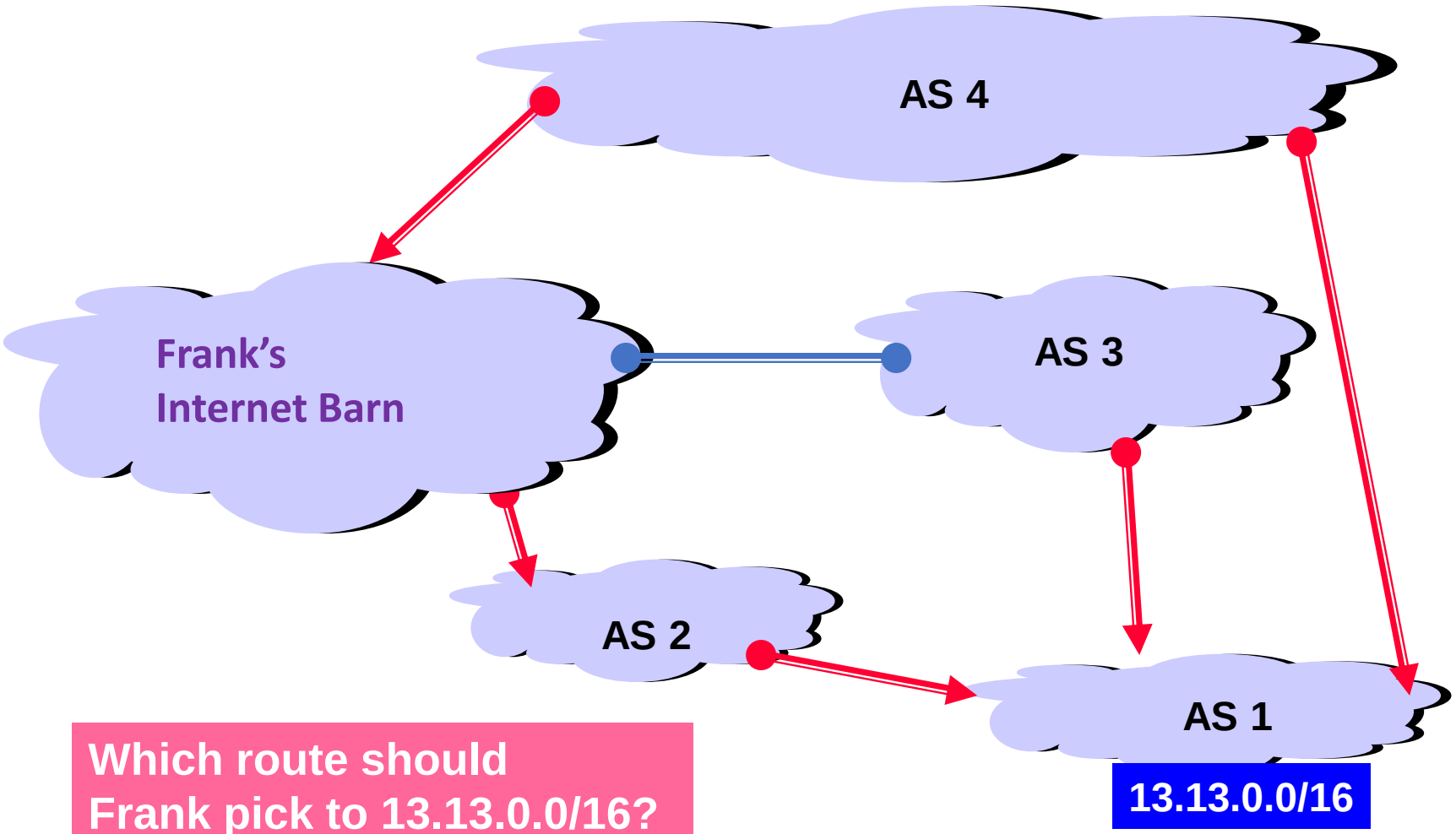
Not all attributes need to be present in every announcement

Route Selection Summary

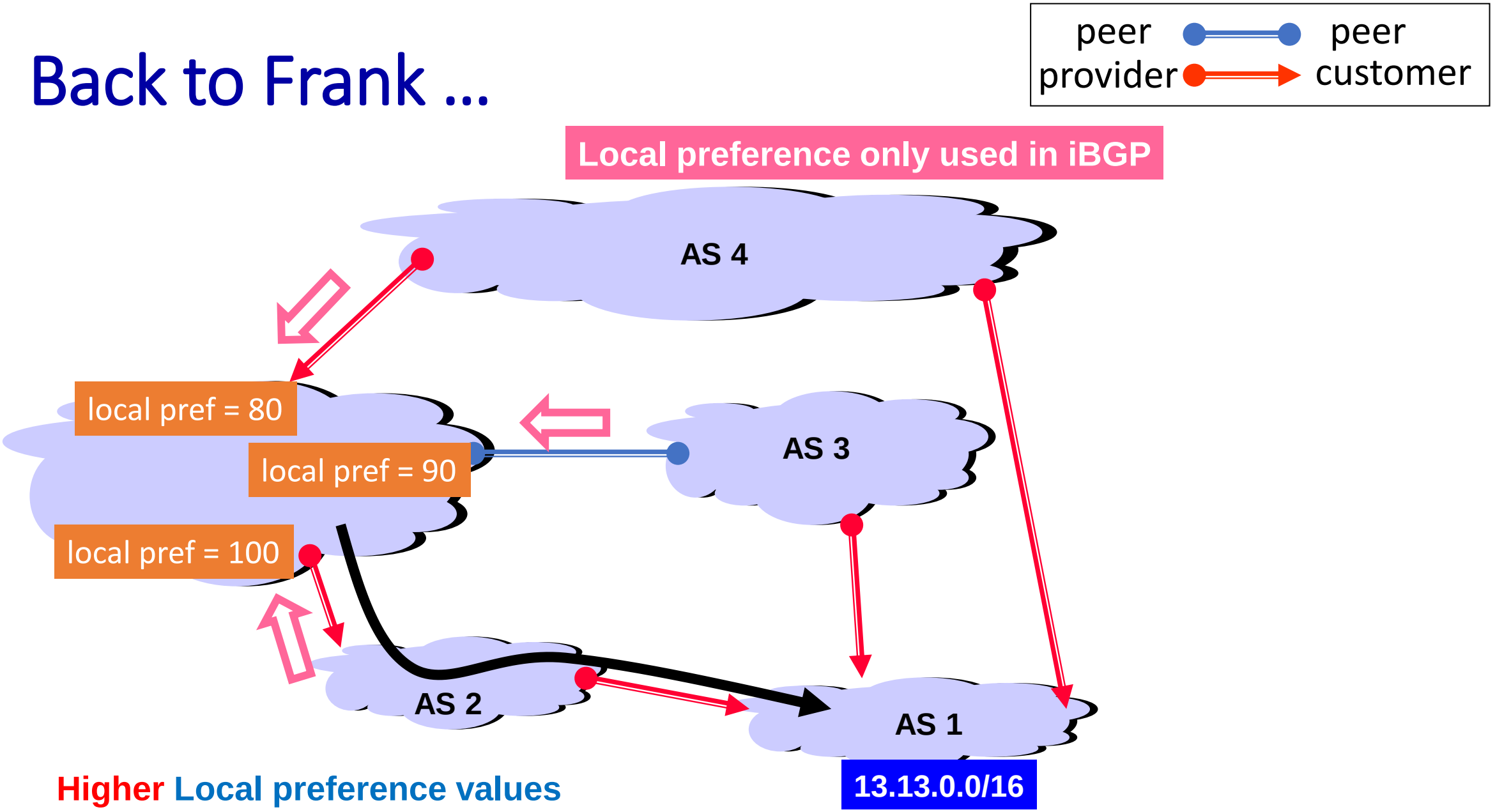


Cisco BGP Best Path Selection
Highest Weight
Highest LOCAL_PREF
Prefer locally originated
Shortest AS_PATH
Lowest origin type
Lowest MED
Prefer eBGP over iBGP
Lowest IGP metric to the BGP NEXT_HOP
Oldest path
Lowest Router ID source
Minimum cluster list length
Lowest neighbor address

So Many Choices

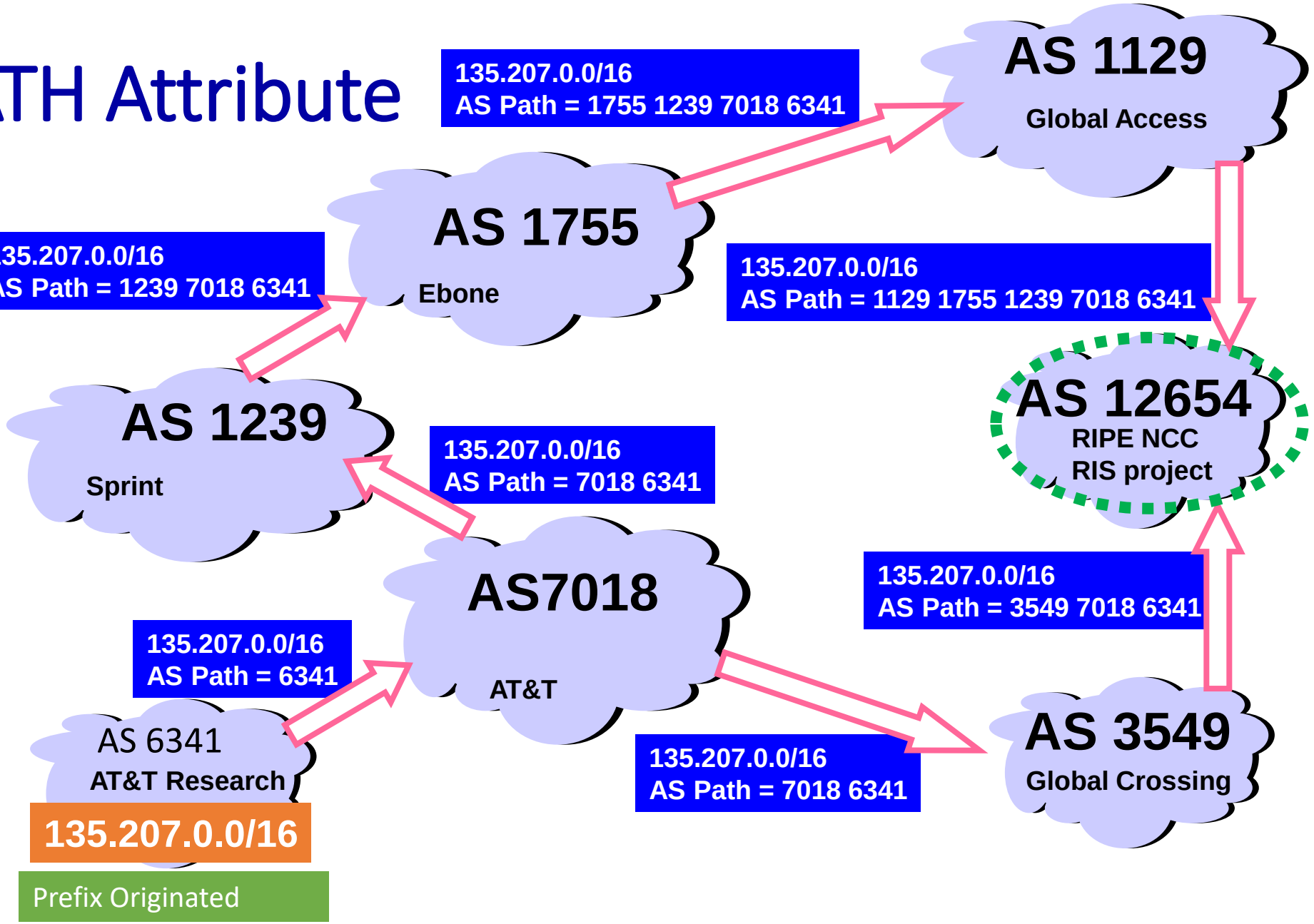


Back to Frank …



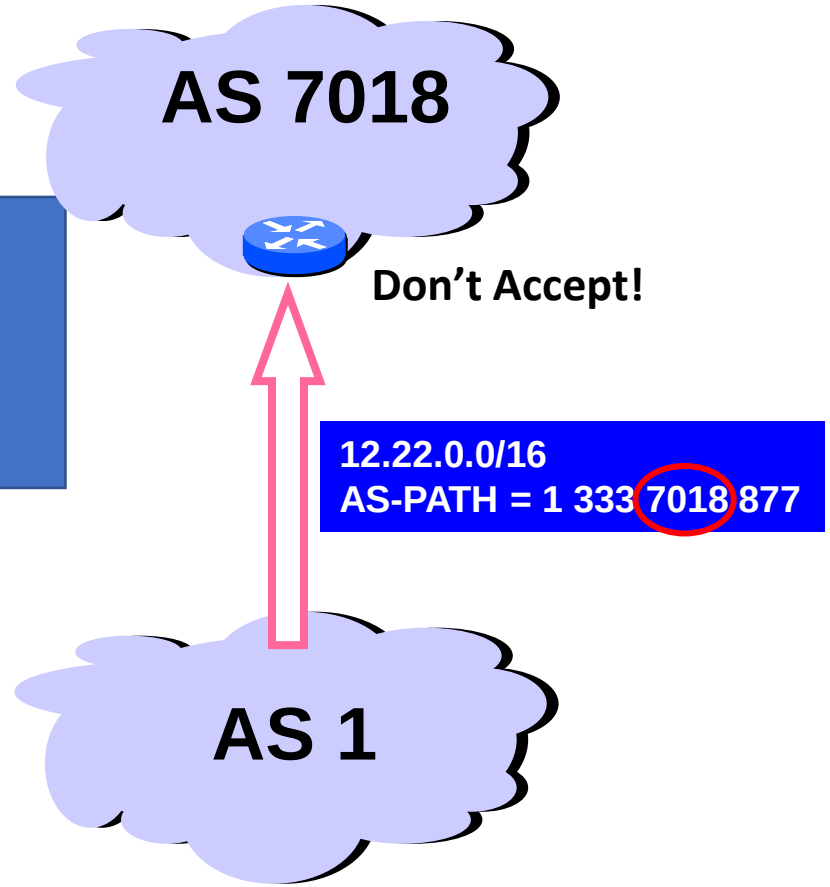
Higher Local preference values
are more preferred

AS-PATH Attribute



Interdomain Loop Prevention

BGP at AS YYY will never accept a route with AS PATH containing YYY.



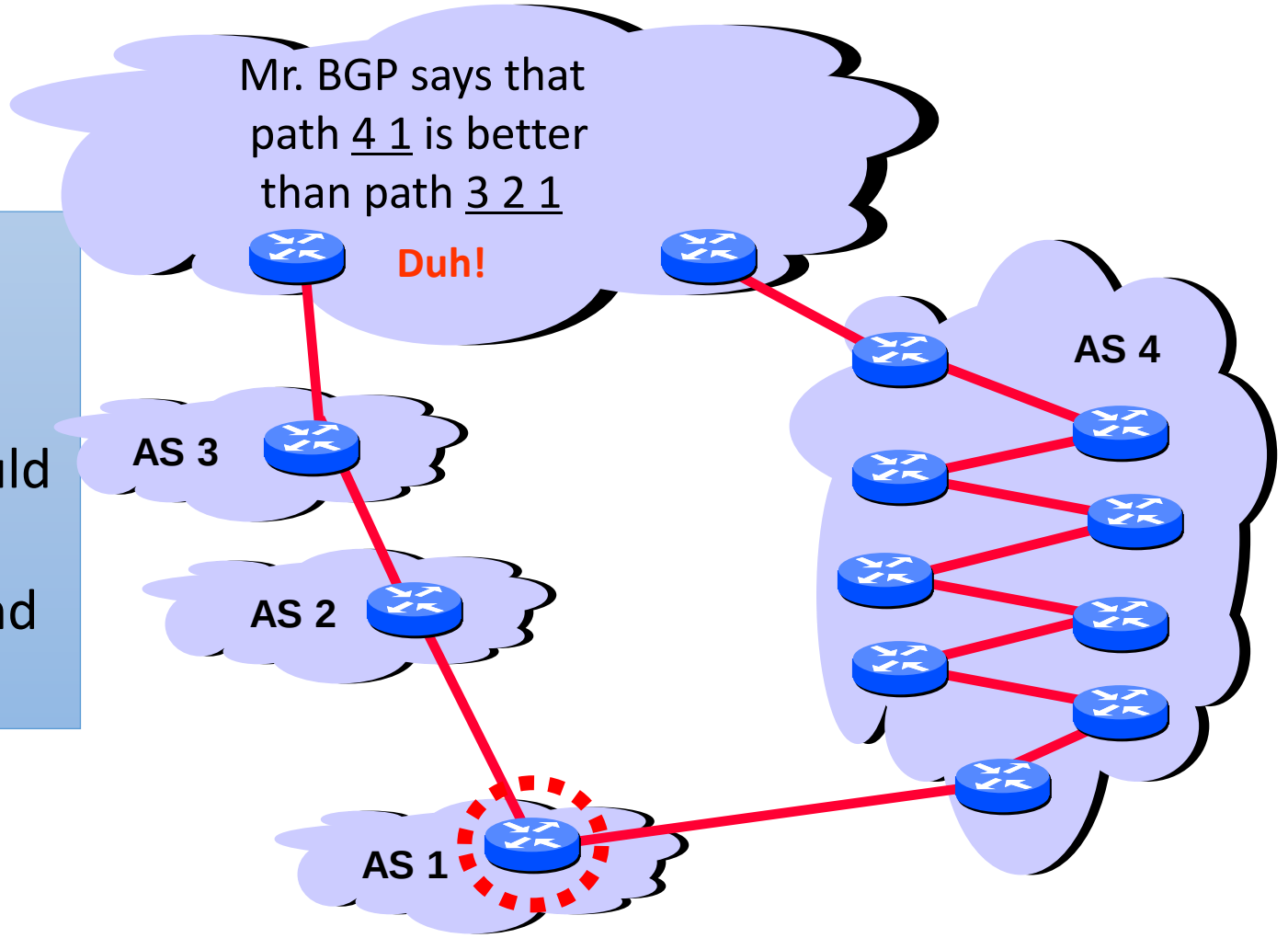
BGP routers use the AS_PATH as

- an AS hop count metric
- a loop avoidance device:

Shorter Doesn't Always Mean Shorter

In fairness: could you do this "right" and still scale?

Exporting internal state would dramatically increase global instability and amount of routing state



“hot potato” and “cold potato” routing



- There’s a general concept in routing/traffic engineering called “hot potato” and “cold potato”.
  - They refer to how quickly (or not) you want to get the traffic off your network.
- With hot potato routing, you want the traffic going to a particular destination off your network ASAP.
- Cold potato routing is the opposite.
  - You want to keep that traffic on your network as long as possible
  - transport it to a point as close as possible to the final destination before handing it over to a peer/provider (if necessary).
- OSPF allows traffic engineering using both hot and cold potato routing.

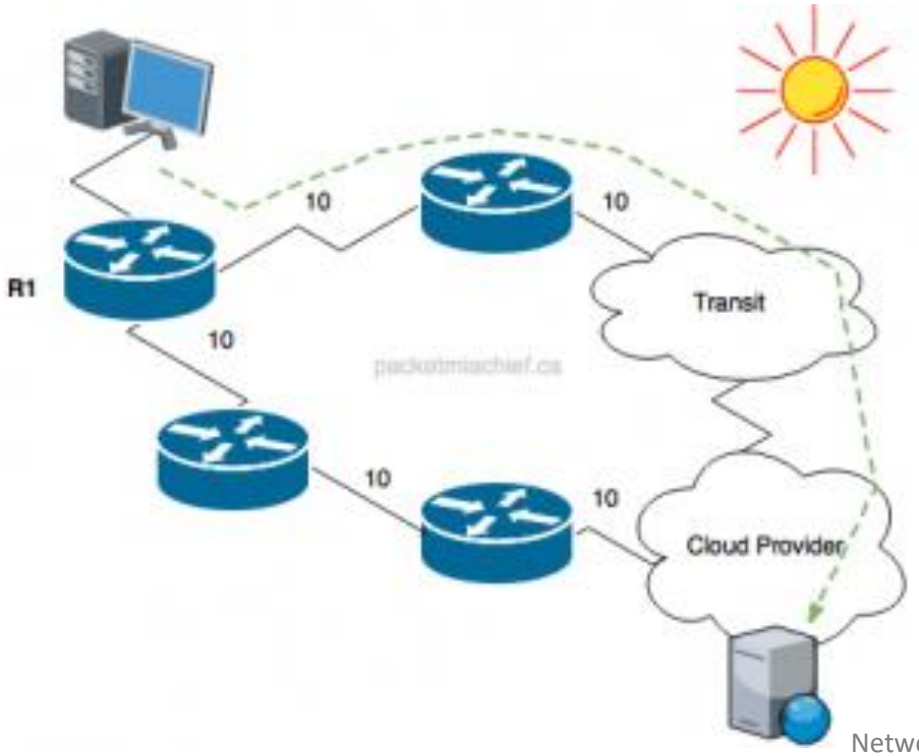
“hot potato” routing





- Hot potato routing is the practice of handing over traffic at the earliest convenience (hot, hot! Here, you take it!)
  - Hot-potato routing is the normal behavior of most settlement-free peering agreements

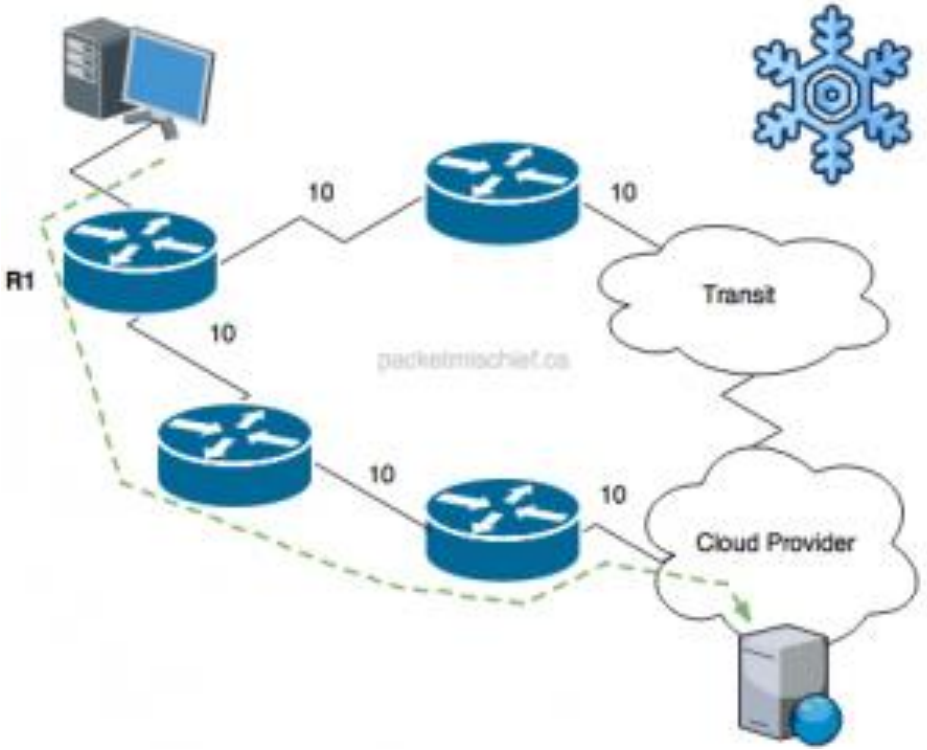




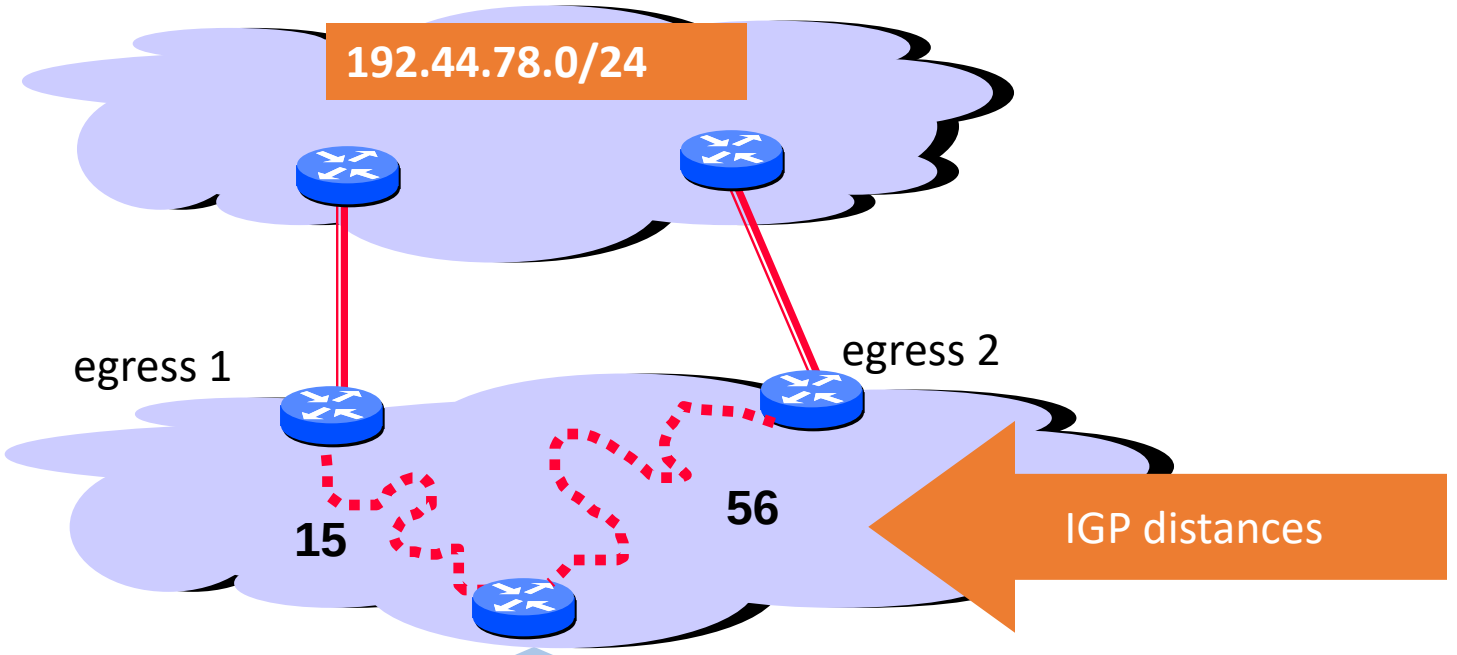
“cold potato” routing



- Cold potato routing is where you hold onto traffic as long as you can before handing it over to another network
  - more expensive to do, allowing operators of well-provisioned networks to offer a higher QoS to their customers
  - Favored by some content networks (Multi Exit Discriminator exchange/honoring) in order to deliver content from replicated server farms closer to the end-user.



Hot Potato Routing: Go for the Closest Egress Point

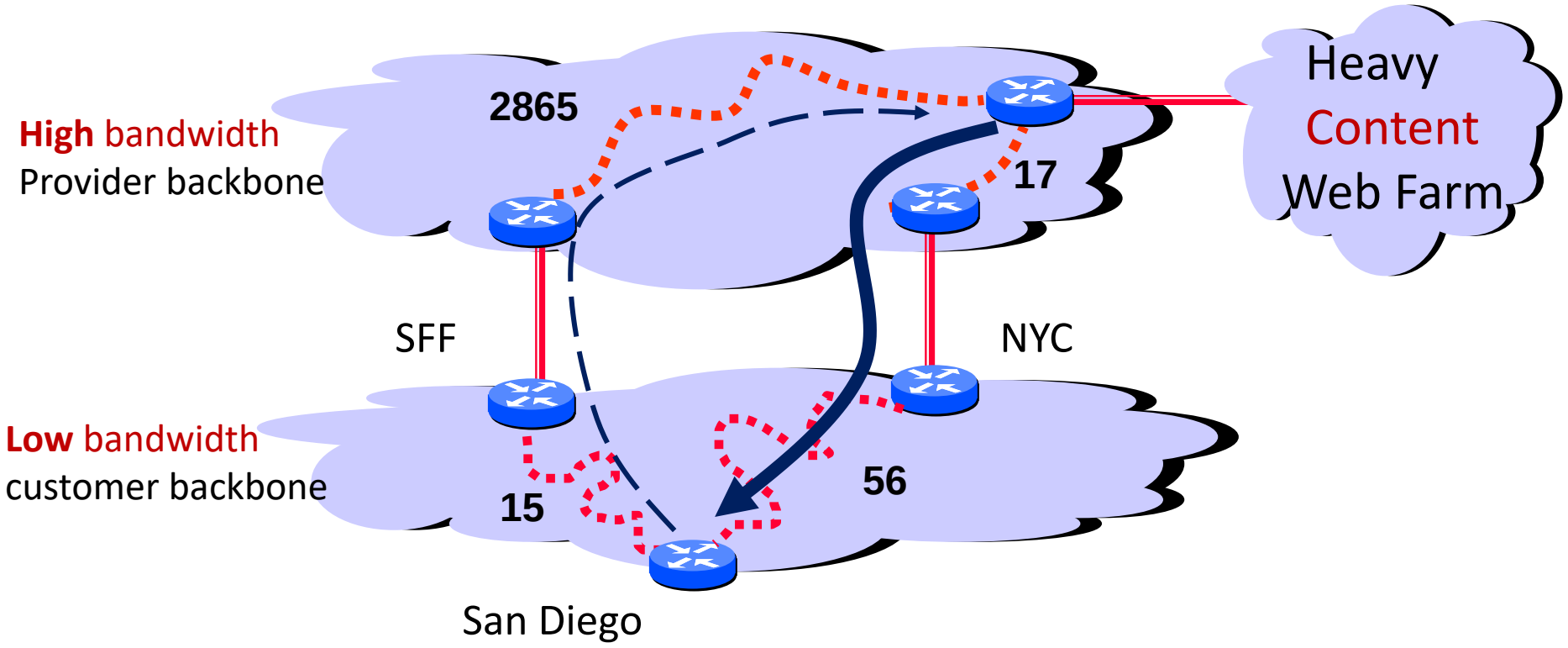


This Router has two BGP routes to 192.44.78.0/24.

Hot potato: get traffic off your network as soon as possible.

Go for egress 1!

Getting Burned by the Hot Potato

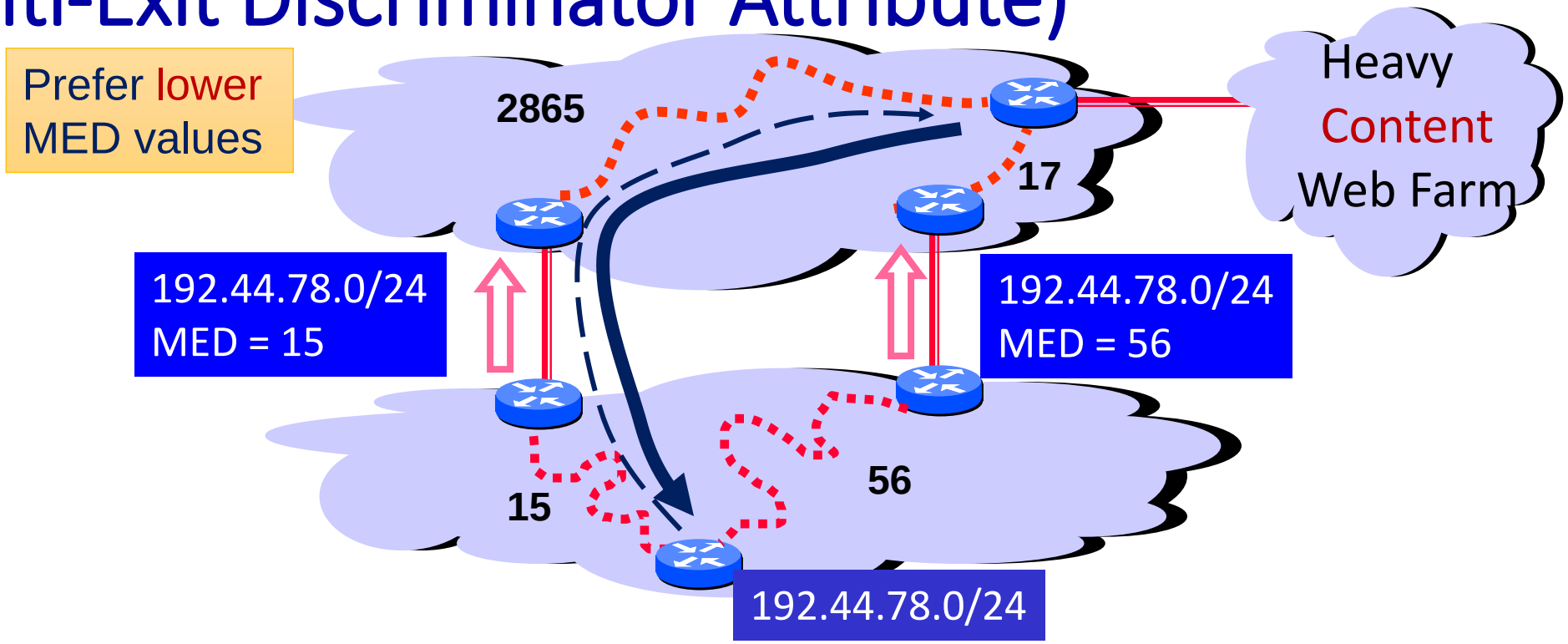


Many customers want their provider to carry the bits!

tiny http request

huge http reply (content)

Cold Potato Routing with MEDs (Multi-Exit Discriminator Attribute)

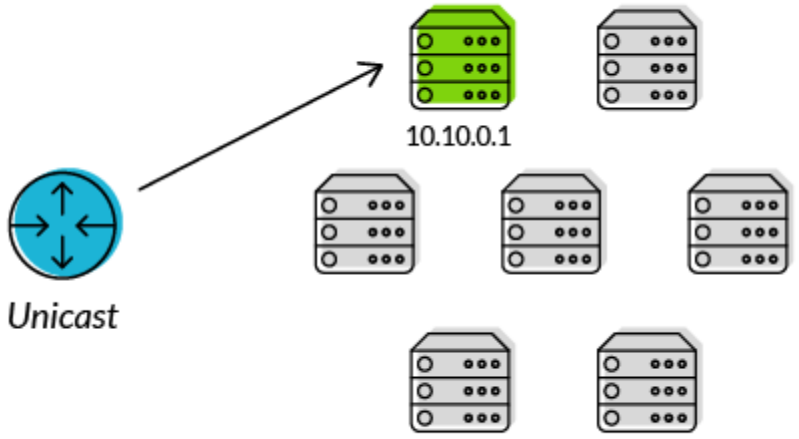


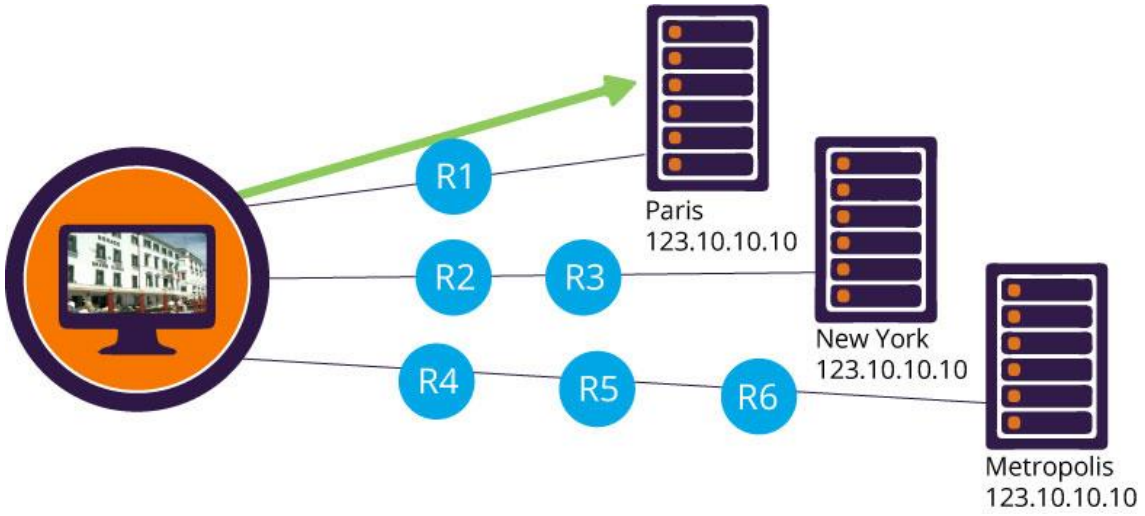
This means that MEDs must be considered BEFORE IGP distance!

Note1 : Some providers will not listen to MEDs
Note2 : MEDs need not be tied to IGP distance

IP Anycast Service

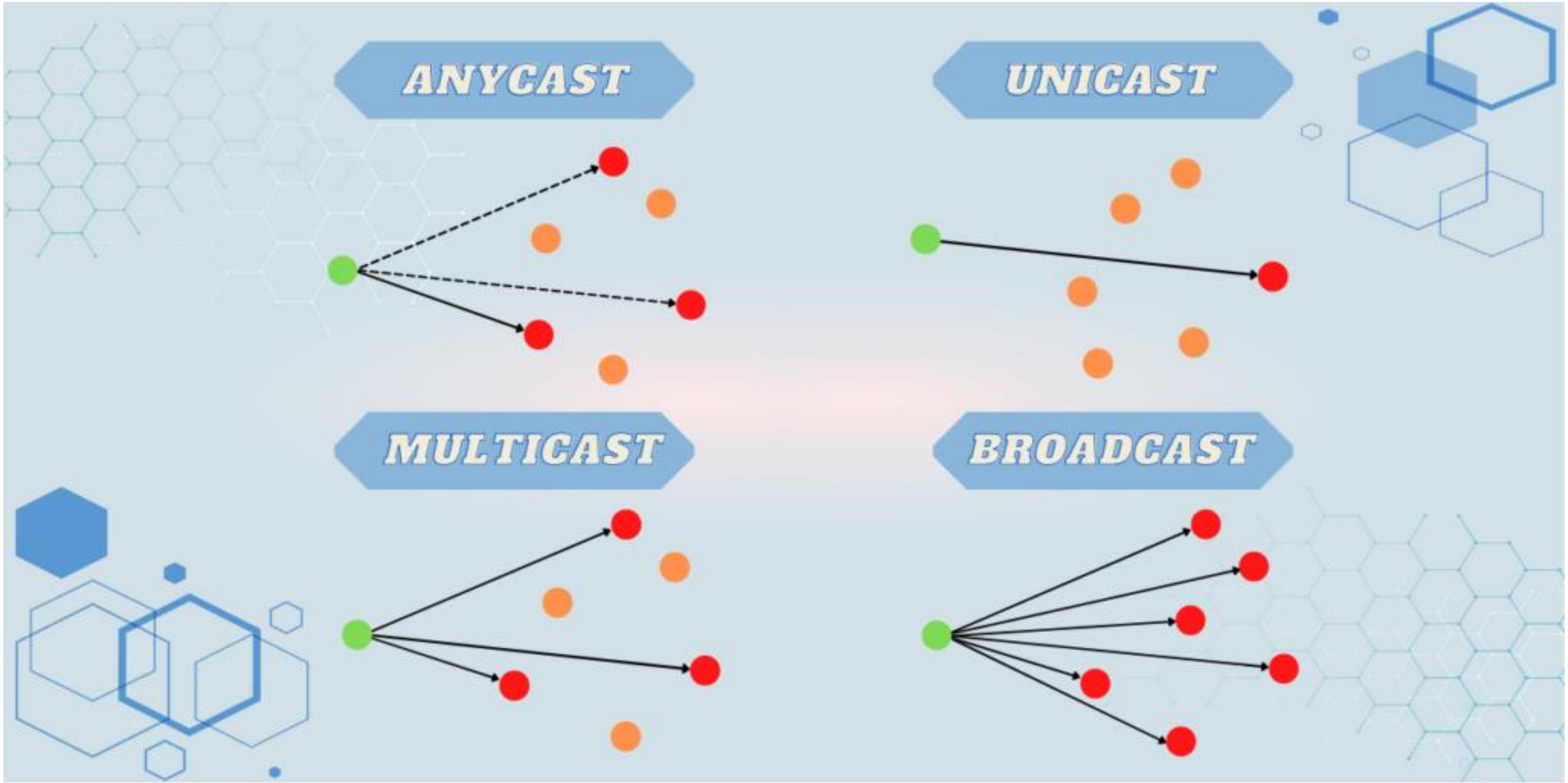
- In addition to being the Internet's inter-AS routing protocol, BGP is often used to implement the IP anycast service, which is commonly used in DNS.
- BGP's route-selection algorithm provides an easy and natural mechanism
  - replicating the same content on different servers in many different dispersed geographical locations (e.g., CDN, DNS)
  - **Anycast IP addresses** enhance network performance by routing traffic to the nearest server, reducing latency and improving reliability.





Features of Unicast, Anycast, Multicast, Broadcast

Feature	Unicast	Anycast	Multicast	Broadcast
Communication	One-to-One	One-to-Nearest	One-to-Many	One-to-All
Destination	Single Unique Address	A Group of Identical Addresses	A Group Address	Broadcast Address
Network Efficiency	Low (Data sent separately to each receiver)	High (Routes request to optimal node)	High (Data replicated once on links)	Low (Data sent to all nodes)
Common Use Cases	Web Browsing, FTP, Email	**DNS** Root Servers, CDN Nodes	Video Conferencing, Live Streaming	ARP Requests, DHCP Discovery
Scope	Routable across subnets	Routable across subnets	Routable (with protocol support)	Usually confined to local broadcast domain


ANYCAST
UNICAST
MULTICAST
BROADCAST

Network layer: "control plane" roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
  - SNMP
  - NETCONF/YANG

Software defined networking (SDN)

- Internet network layer: historically implemented via distributed, per-router control approach:
  - *monolithic* router contains switching hardware, runs proprietary implementation of Internet standard protocols (IP, RIP, IS-IS, OSPF, BGP) in proprietary router OS (e.g., Cisco IOS)
  - different "middleboxes" for different network layer functions: firewalls, load balancers, NAT boxes, ..
- ~2005: renewed interest in rethinking network control plane

Software defined networking (SDN)

Why a *logically centralized* control plane?

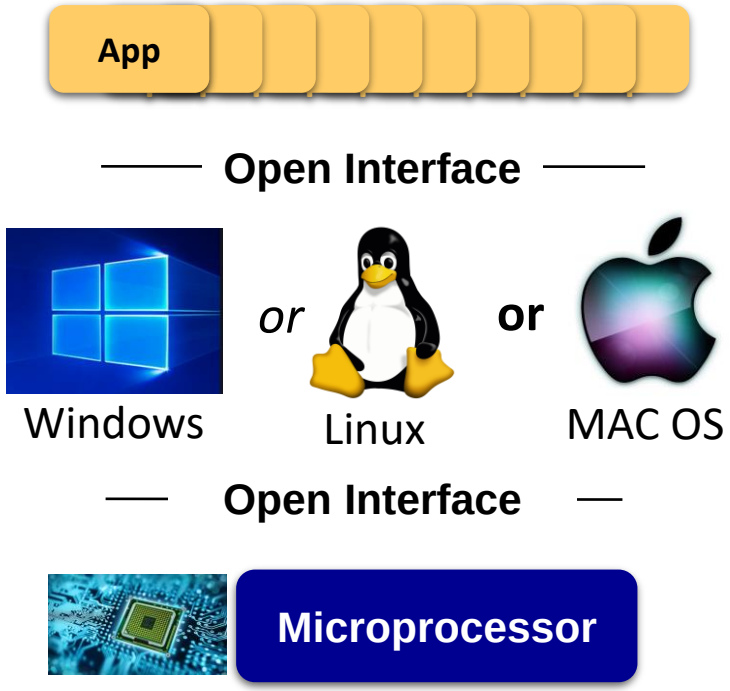
- easier network management: avoid router misconfigurations, greater flexibility of traffic flows
- table-based forwarding (recall OpenFlow API) allows "programming" routers
  - centralized "programming" easier: compute tables centrally and distribute
  - distributed "programming" more difficult: compute tables as result of distributed algorithm (protocol) implemented in each-and-every router
- open (non-proprietary) implementation of control plane
  - foster innovation: let 1000 flowers bloom



SDN analogy: mainframe to PC revolution



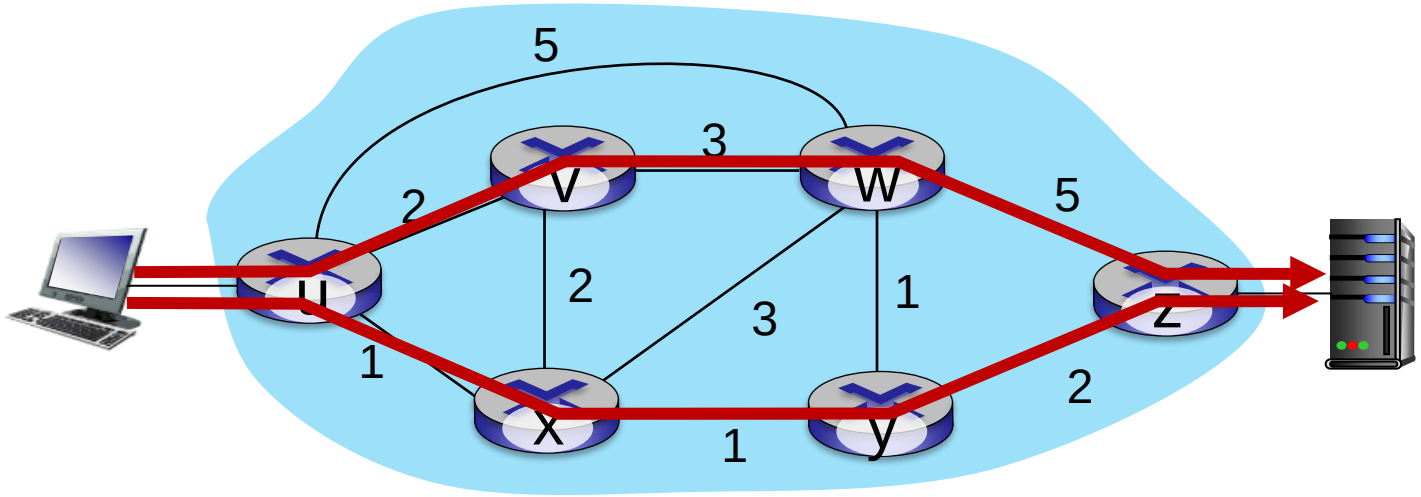
Vertically integrated
Closed, proprietary
Slow innovation
Small industry



Horizontal
Open interfaces
Rapid innovation
Huge industry

* Slide courtesy: N. McKeown

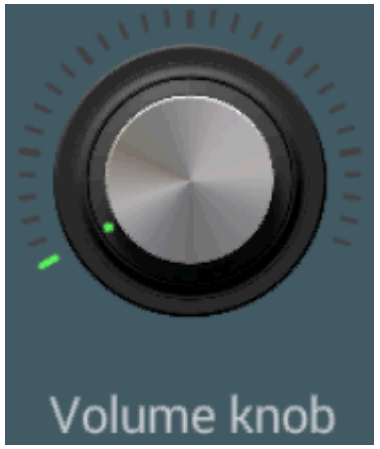
Traffic engineering: difficult with traditional routing



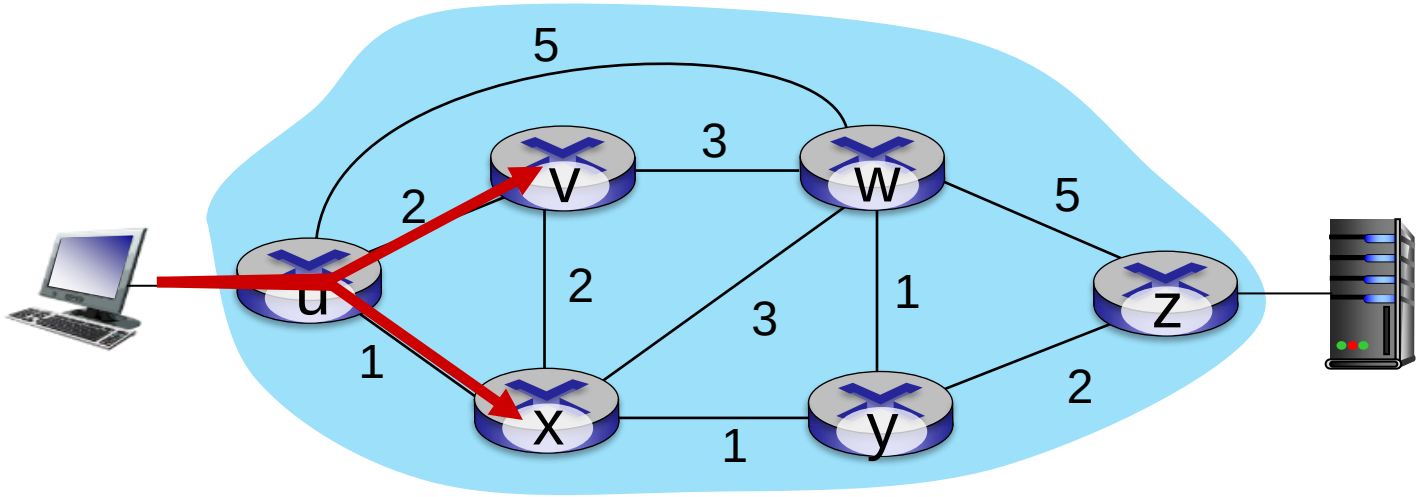
Q: what if network operator wants u-to-z traffic to flow along *uvwz*, rather than *uxyz*?

A: need to re-define link weights so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

link weights are only control "knobs": not much control!



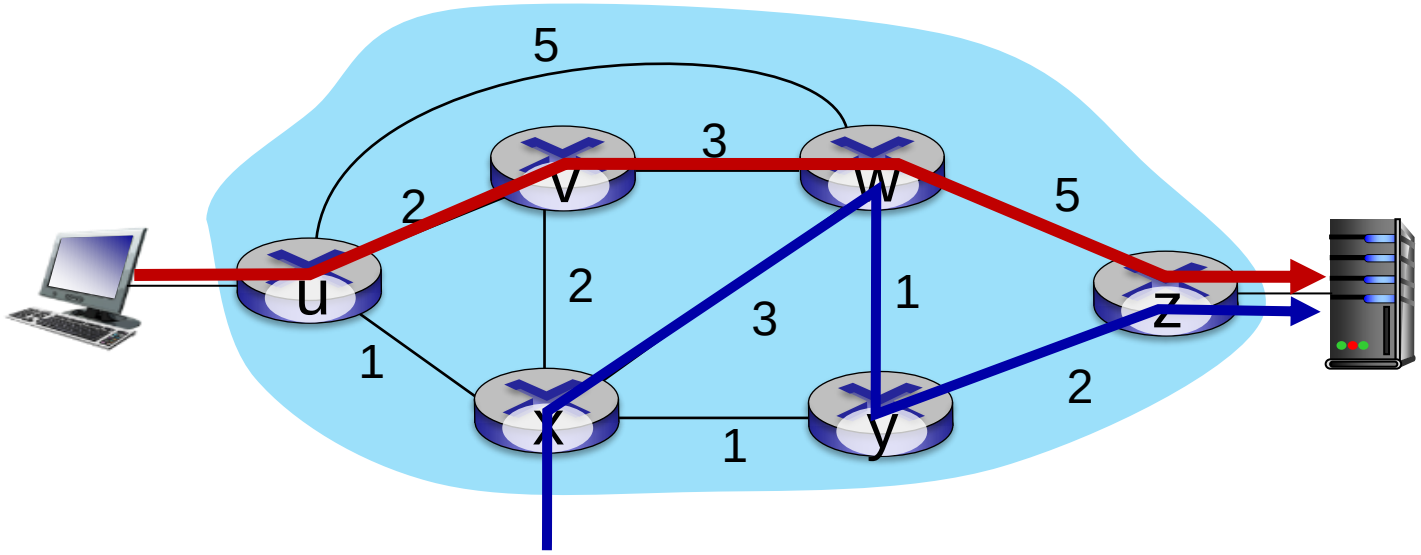
Traffic engineering: difficult with traditional routing



Q: what if network operator wants to split u-to-z traffic along uvwz *and* uxyz (load balancing)?

A: can't do it (or need a new routing algorithm)

Traffic engineering: difficult with traditional routing



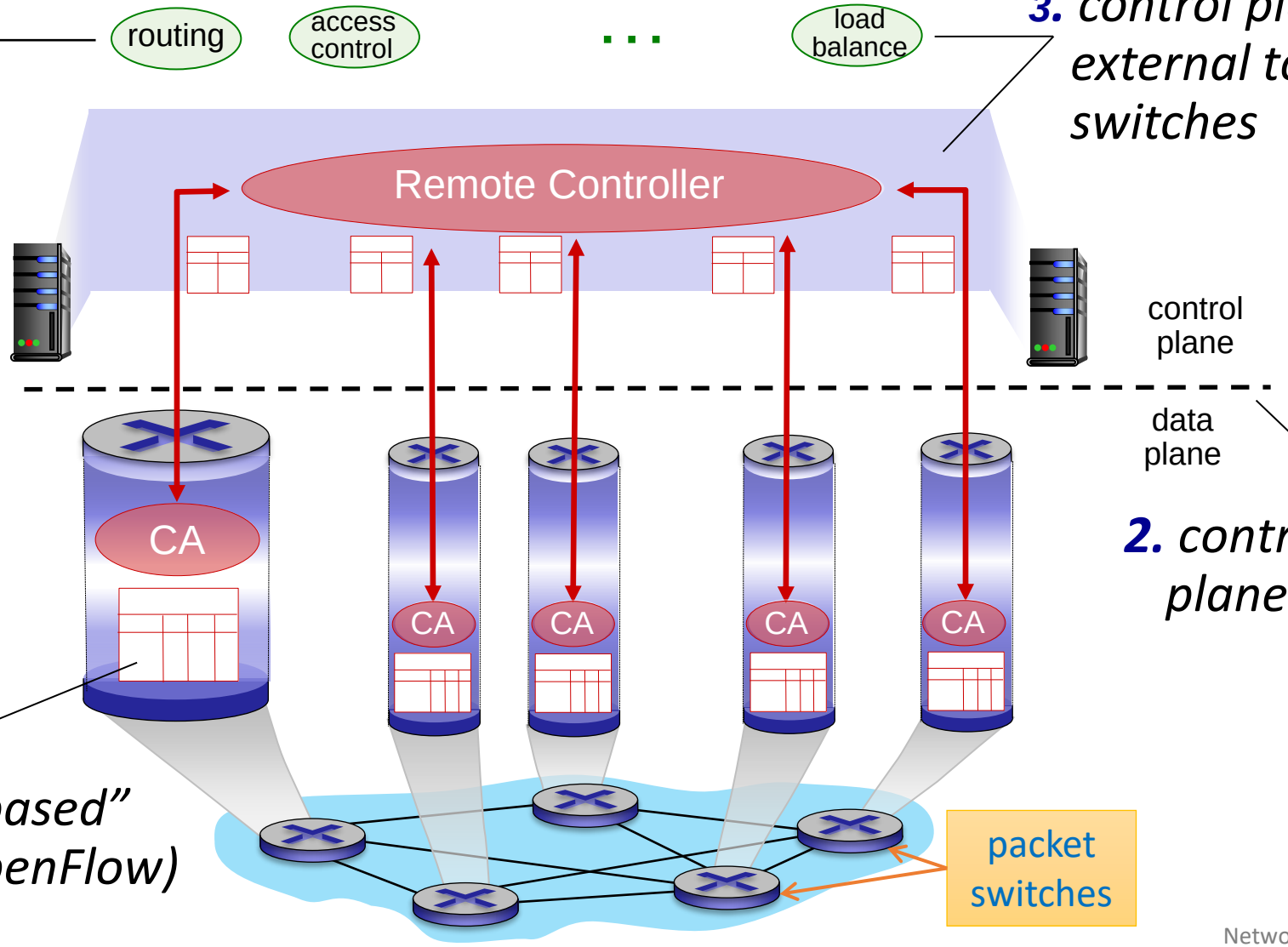
Q: what if w wants to route blue and red traffic differently from w to z?

A: can't do it (with destination-based forwarding, and LS, DV routing)

We learned that generalized forwarding and SDN can be used to achieve *any* routing desired

Software defined networking (SDN)

4. *programmable control applications*



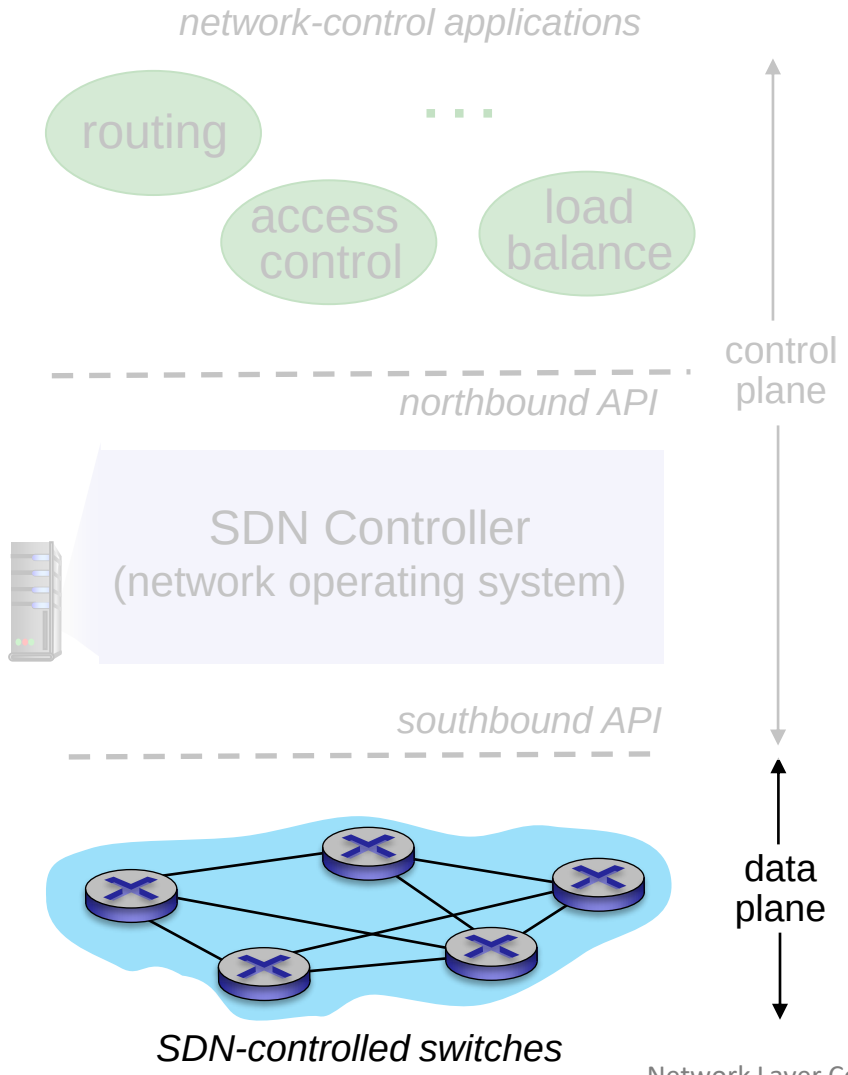
3. *control plane functions external to data-plane switches*

2. *control, data plane separation*

1: *generalized "flow-based" forwarding (e.g., OpenFlow)*

SDN Data-plane switches

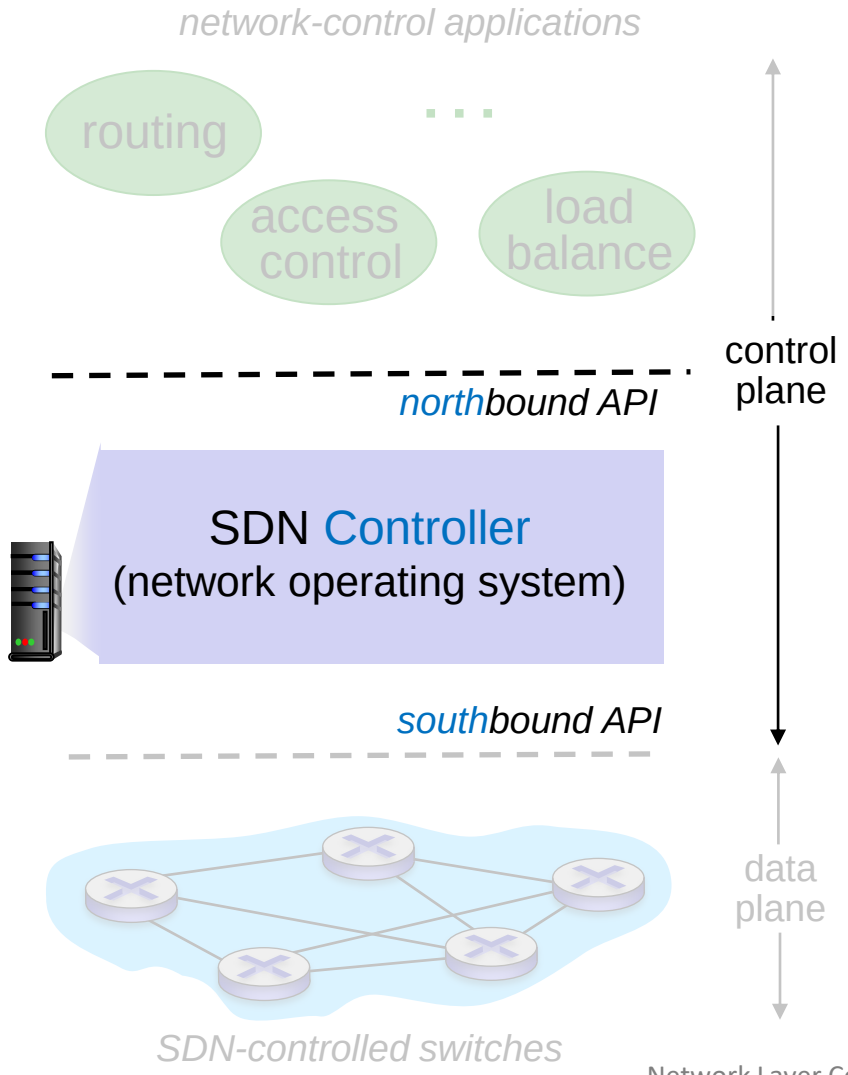
- fast, simple, commodity switches implementing generalized data-plane forwarding in hardware
- flow (forwarding) table computed, installed under controller supervision
- API for table-based switch control (e.g., OpenFlow)
  - defines what is controllable, what is not
- protocol for communicating with controller (e.g., OpenFlow)



SDN controller (network OS)

"logically centralized", but implemented as distributed system for performance, scalability, fault-tolerance, robustness

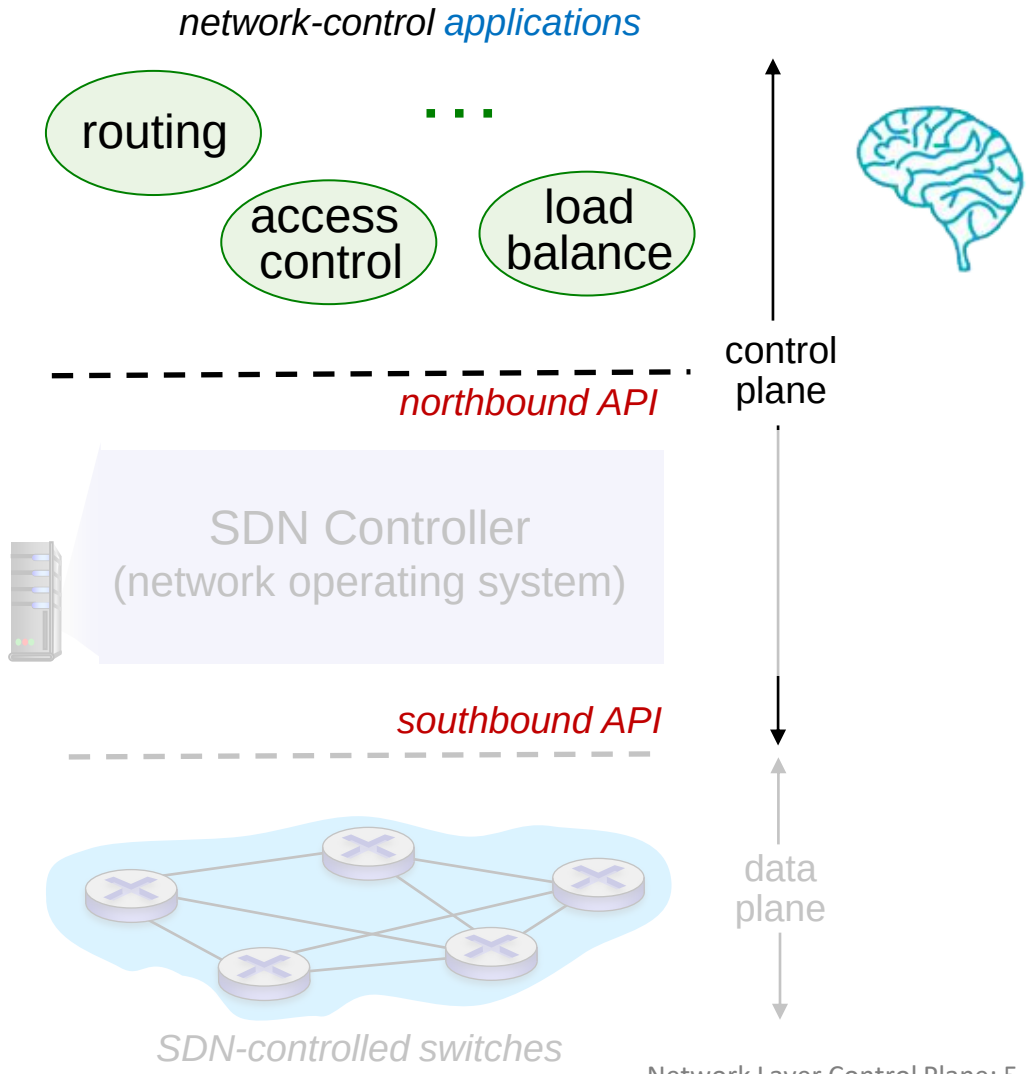
- **The interface to the network-control application layer:** interacts with network control applications "above" via northbound API
- **A network-wide state-management layer:** maintain network state information (e.g., the state of remote links, switches, and hosts)
- **A communication layer:** interacts with network switches "below" via southbound API



Software defined networking (SDN)

network-control apps:

- "brains" of control: implement control functions using lower-level services, API provided by SDN controller
- *unbundled:* can be provided by 3rd party: distinct from routing vendor, or SDN controller

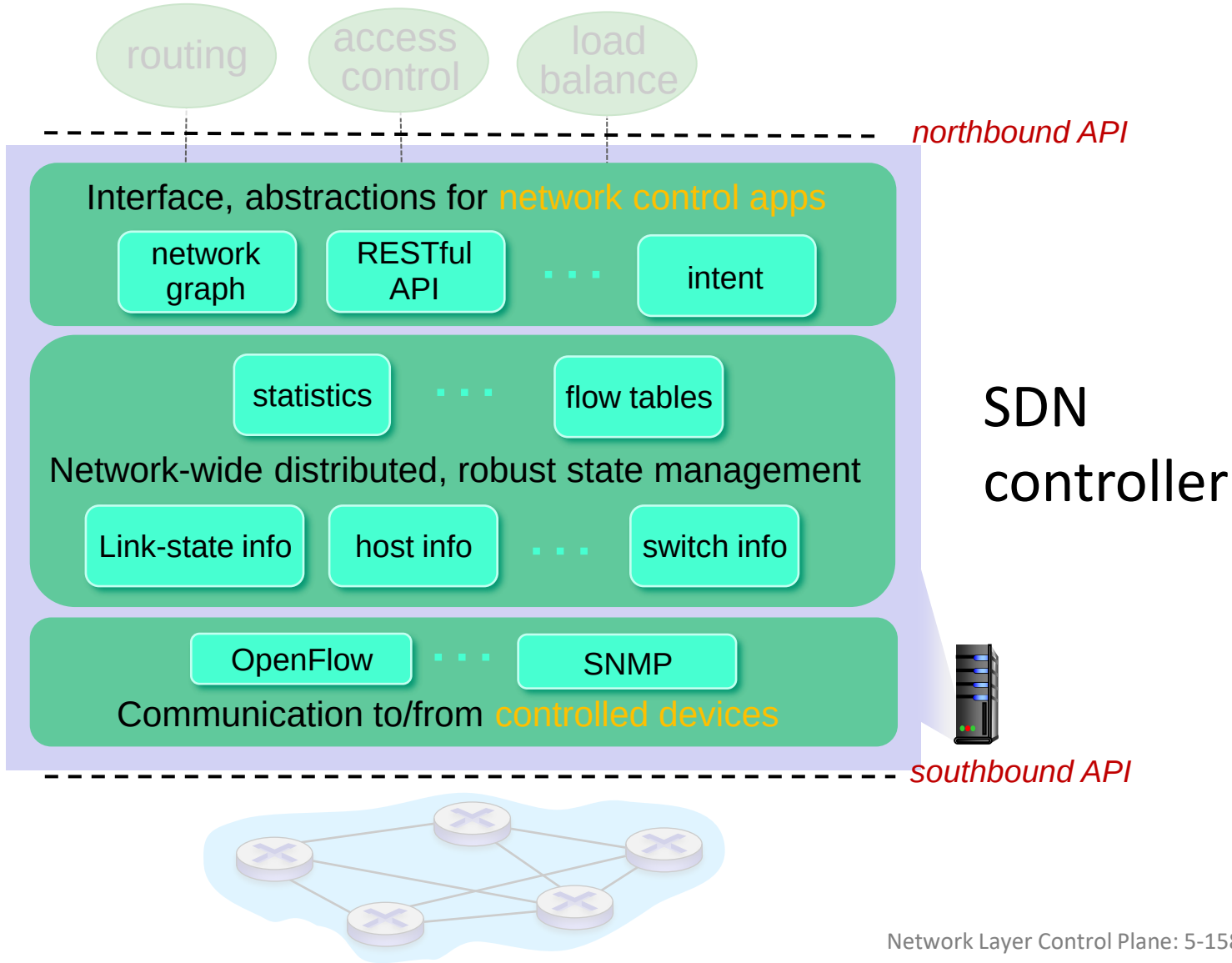


Components of SDN controller

interface layer to network control apps: abstractions API

network-wide state management : state of networks links, switches, services: a *distributed database*

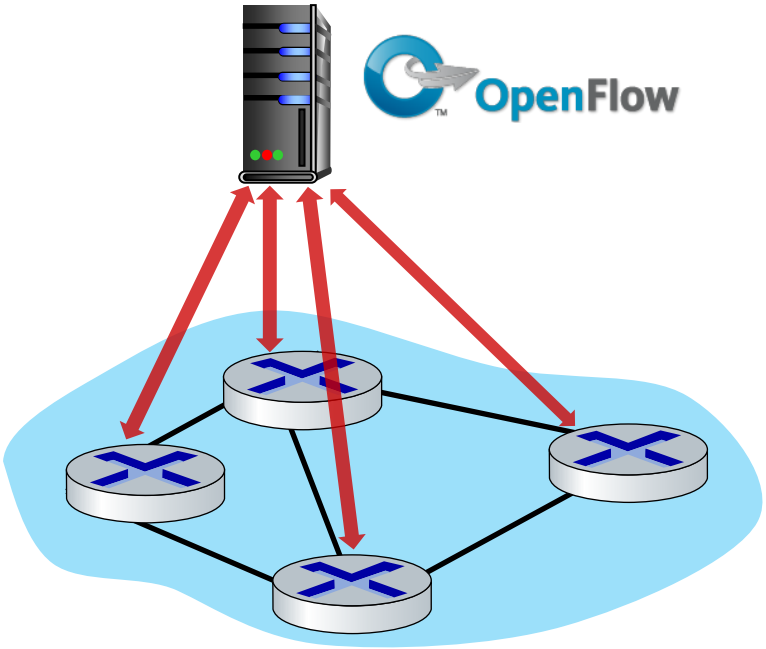
communication: communicate between SDN controller and controlled switches (a protocol is needed.)



OpenFlow protocol

- operates between SDN controller and SDN-controlled switch
- TCP used to exchange messages
  - optional encryption
- three classes of OpenFlow messages:
  - controller-to-switch
  - asynchronous (switch to controller)
  - symmetric (misc.)
- distinct from OpenFlow API
  - API used to specify generalized forwarding actions

OpenFlow Controller

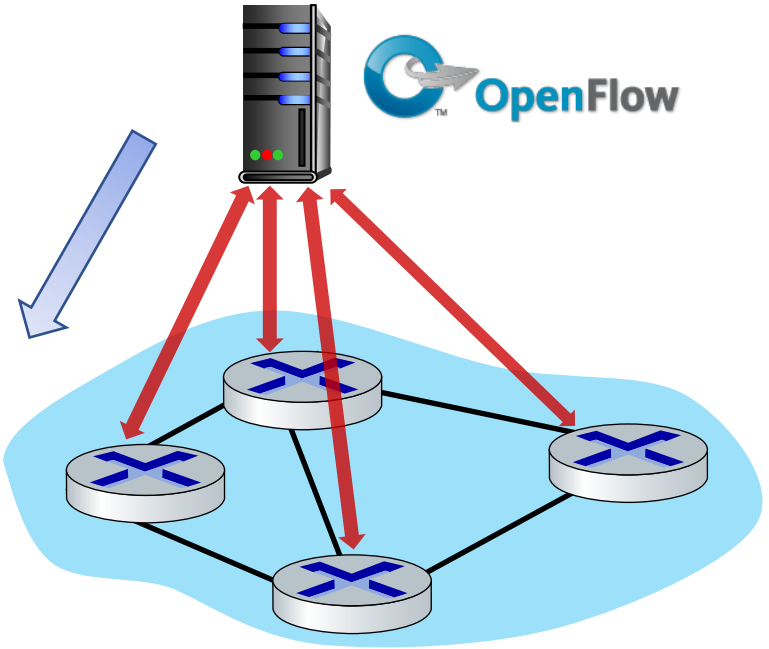


OpenFlow: controller-to-switch messages

Key controller-to-switch messages

- *Read-State:* controller queries switch features (statistics and counter values), switch replies
- *configure:* controller queries/sets switch configuration parameters
- *modify-state:* add, delete, modify flow entries in the OpenFlow tables
- *packet-out:* controller can send this packet out of specific switch port

OpenFlow Controller

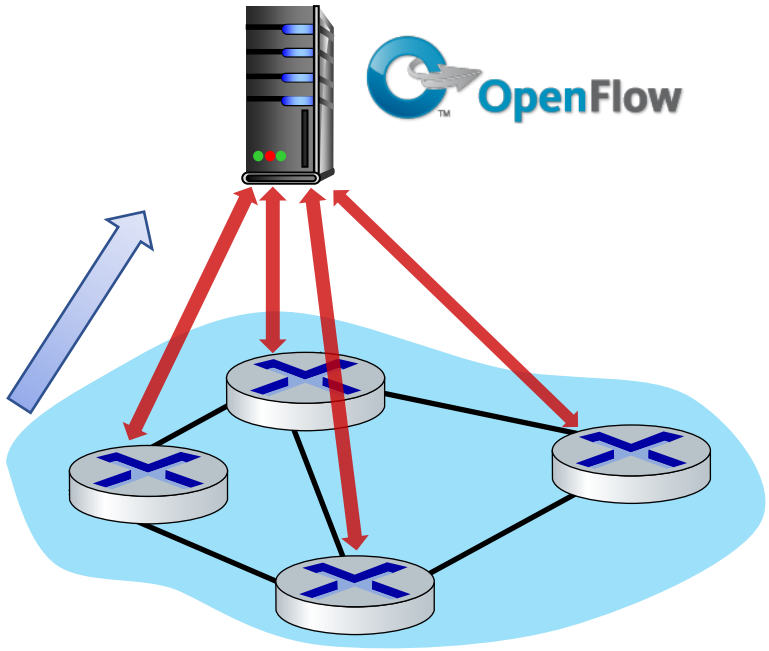


OpenFlow: switch-to-controller messages

Key switch-to-controller messages

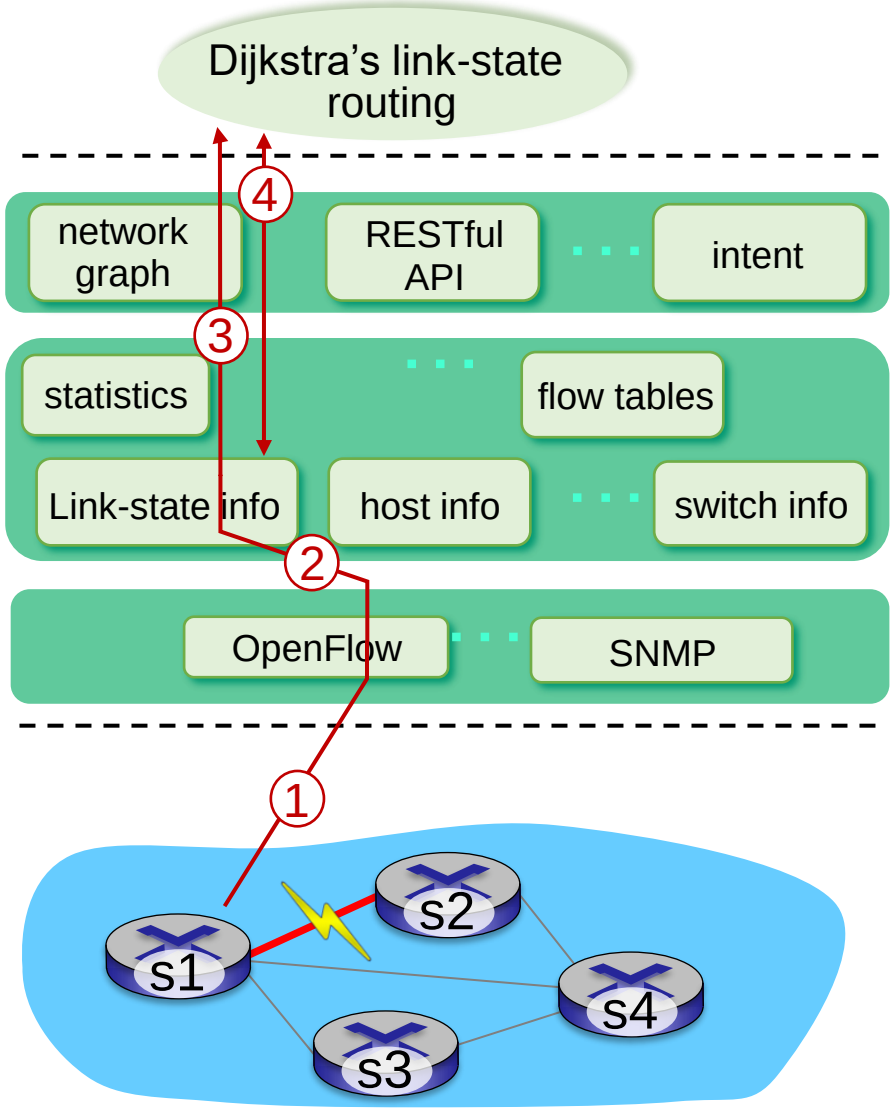
- *packet-in:* transfer packet (and its control) to controller. See packet-out message from controller
- *flow-removed:* flow table entry deleted at switch
- *port status:* inform controller of a change on a port.

OpenFlow Controller



Fortunately, network operators don't "program" switches by creating/sending OpenFlow messages directly. Instead use higher-level abstraction at controller

SDN: control/data plane interaction example



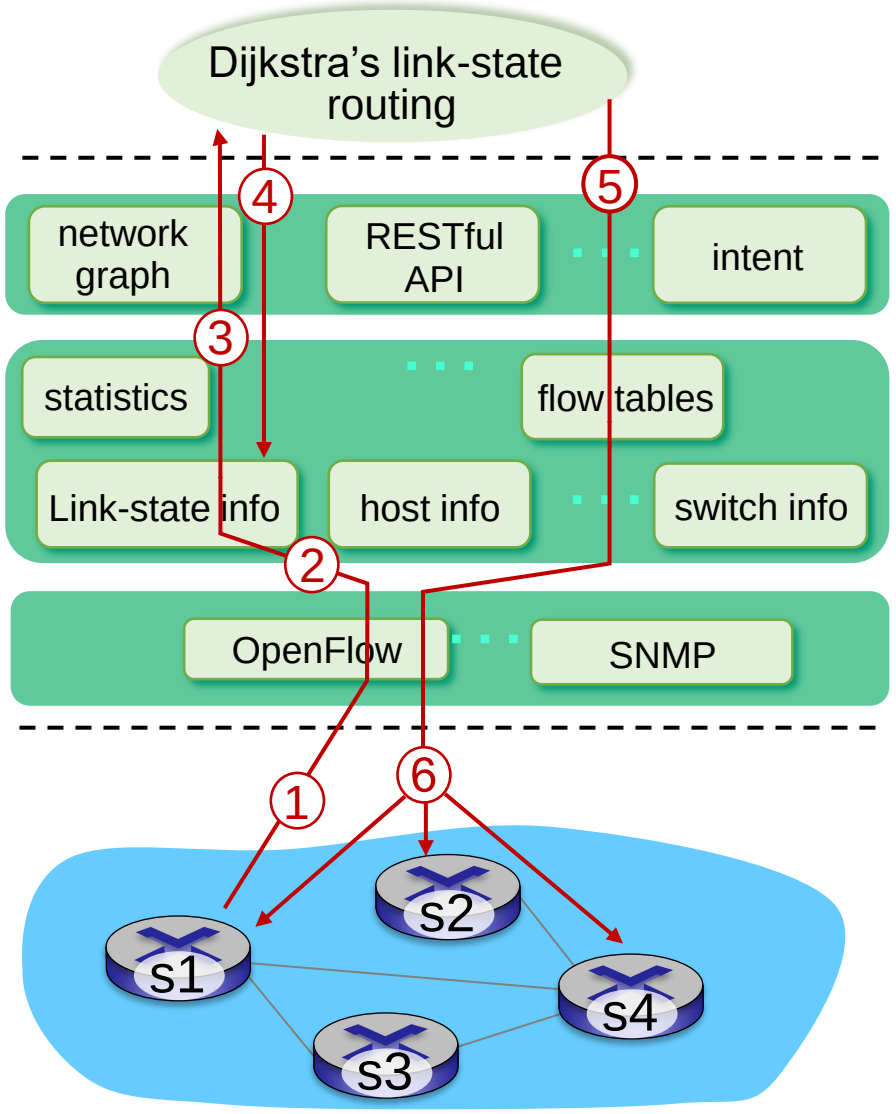
① S1, experiencing link failure uses OpenFlow *port status* message to notify controller

② SDN controller receives OpenFlow message, updates link status info

③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.

④ Dijkstra's routing algorithm access network graph info, link state info in controller, computes new routes

SDN: control/data plane interaction example



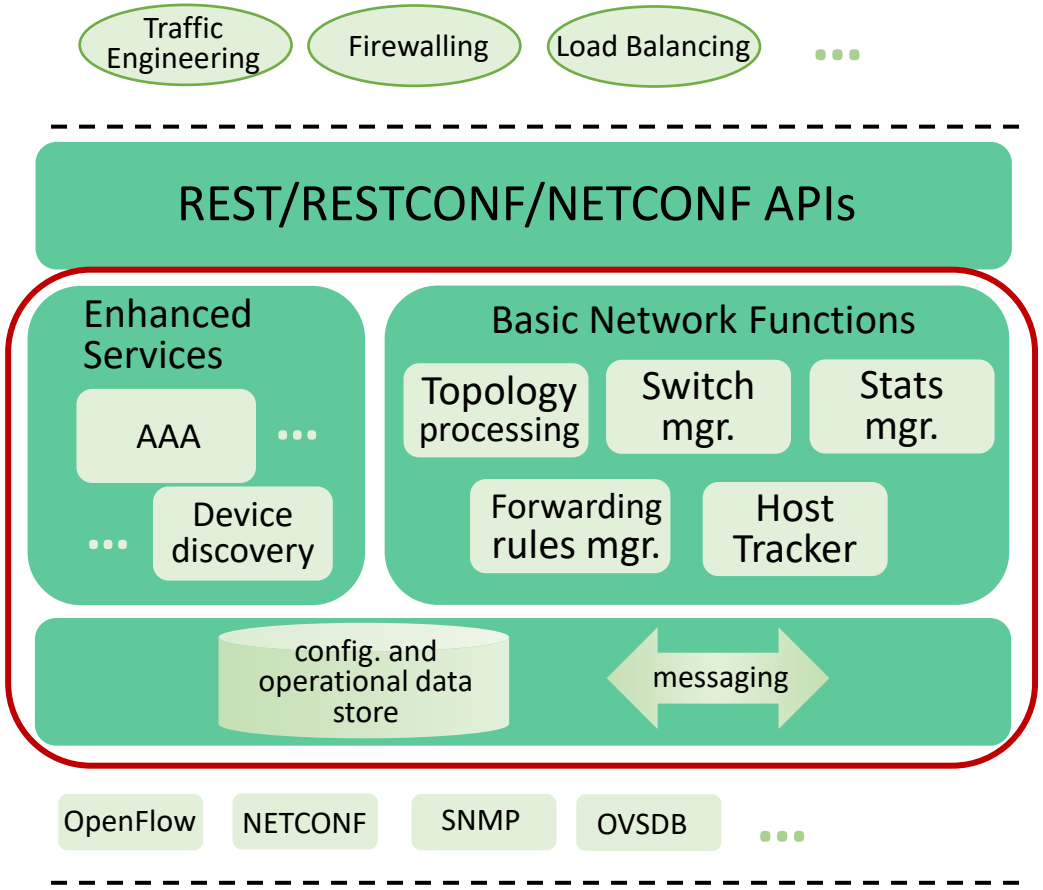
⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed

⑥ controller uses OpenFlow to install new tables in switches that need updating

Network-layer routing	SDN	Traditional per-router control
Operation	change its application-control software	Changes occur in each and every network router
Feature	Simple and easy	Complex (software in all routers might be provided by multiple vendors)

OpenDaylight (ODL) controller

open-source and are being developed in partnership with the Linux Foundation.

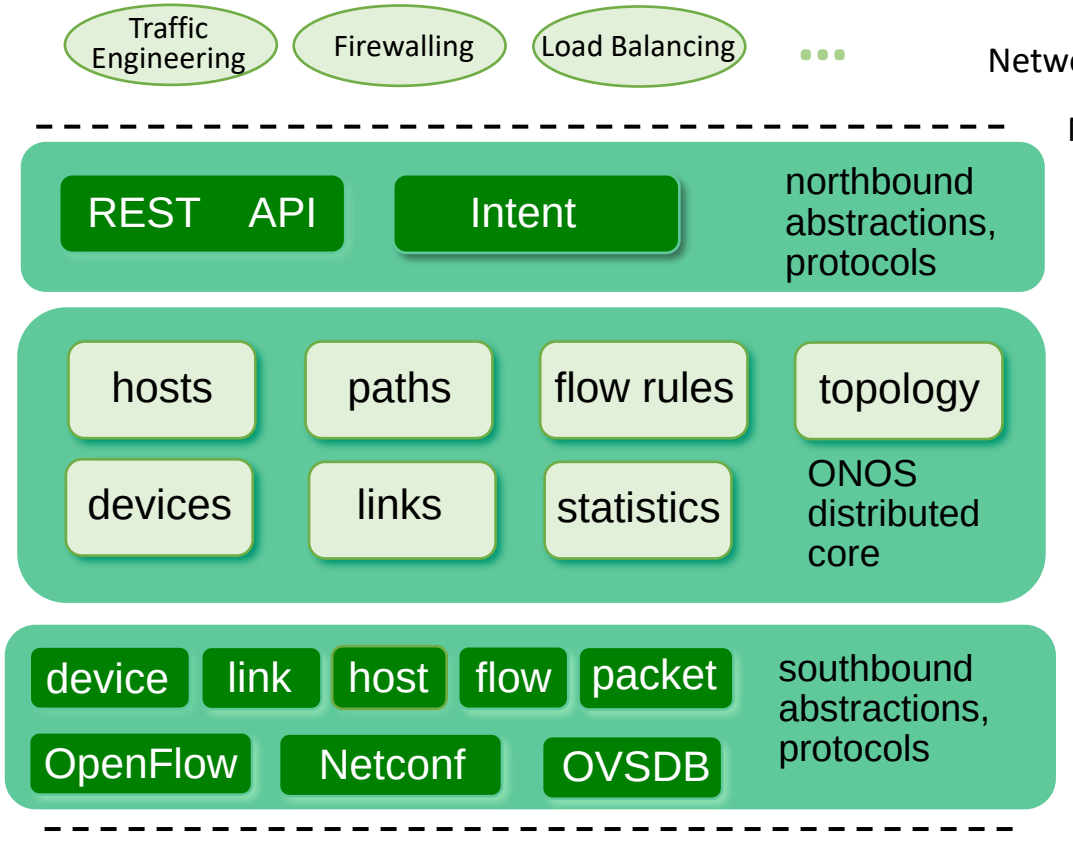


Service Abstraction Layer (SAL):

- interconnects internal, external applications and services

ONOS controller (Open Network Operating System)

open-source and are being developed in partnership with the Linux Foundation.



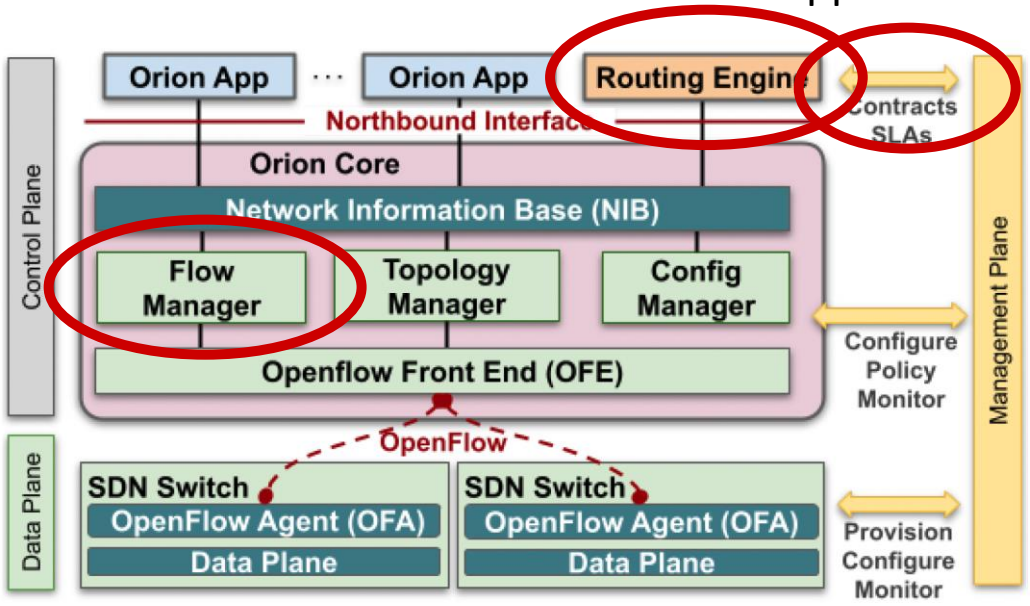
- control apps separate from controller
- intent framework: high-level specification of service: what rather than how
- considerable emphasis on distributed core: service reliability, replication performance scaling

Google ORION SDN control plane

ORION: Google's SDN control plane (*NSDI'21*): control plane for Google's datacenter (Jupiter) and wide area (B4) networks

- routing (intradomain, iBGP), traffic engineering: implemented in *applications* on top of ORION core
- edge-edge flow-based controls (e.g., CoFlow scheduling) to meet contract SLAs
- management: pub-sub distributed microservices in Orion core, OpenFlow for switch signaling/monitoring

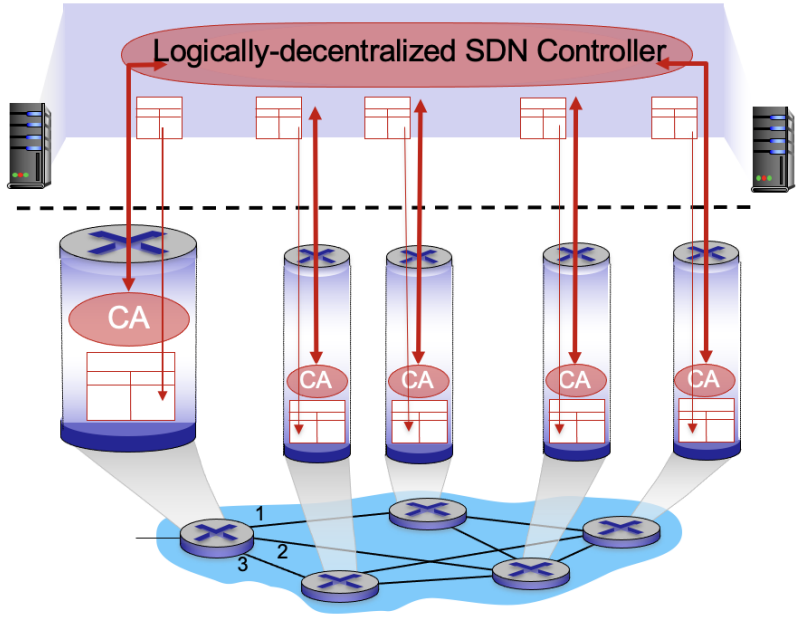
Orion SDN architecture and core apps



Note: ORION provides *intradomain* services within Google's network

Google, SDN: the value of a logically-centralized abstraction

- new opportunities to more formally, intentionally manage configuration



" ... in a single building or even across a private global WAN such that we[Google] were designing, the location and role of every network element is known ahead of time. So - rather than allowing a node to boot, exchange messages with neighbors, and learn its role in the global network, we knew we could configure the role of all network elements into a logically centralized controller that could then inform all relevant elements of new and changing configurations. This shifted the problem of network management from managing the configuration of individual network elements to managing the configuration of a logical network, dramatically simplifying the problem both from a technical perspective but also from the perspective of the cognitive load on network managers."∂

https://www.youtube.com/watch?v=qn7d_iK92TI



Amin Vahdat, Engineering Fellow, VP for the Machine Leaning, Systems, and Cloud AI team

SDN: selected challenges

- hardening the control plane: dependable, reliable, performance-scalable, secure distributed system
  - robustness to failures: leverage strong theory of reliable distributed system for control plane
  - dependability, security: "baked in" from day one?
- networks, protocols meeting mission-specific requirements
  - e.g., real-time, ultra-reliable, ultra-secure
- Internet-scaling: beyond a single AS (intra-AS → inter-AS)
- SDN critical in 5G cellular networks



SDN and the future of traditional network protocols

- SDN-computed vs router-computed forwarding tables:
  - just one example of logically-centralized-computed versus protocol computed
- One could imagine SDN-computed congestion control:
  - controller sets sender rates based on router-reported (to controller) congestion levels



How will implementation of network functionality (SDN versus protocols) evolve?



Network layer: "control plane" roadmap

- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



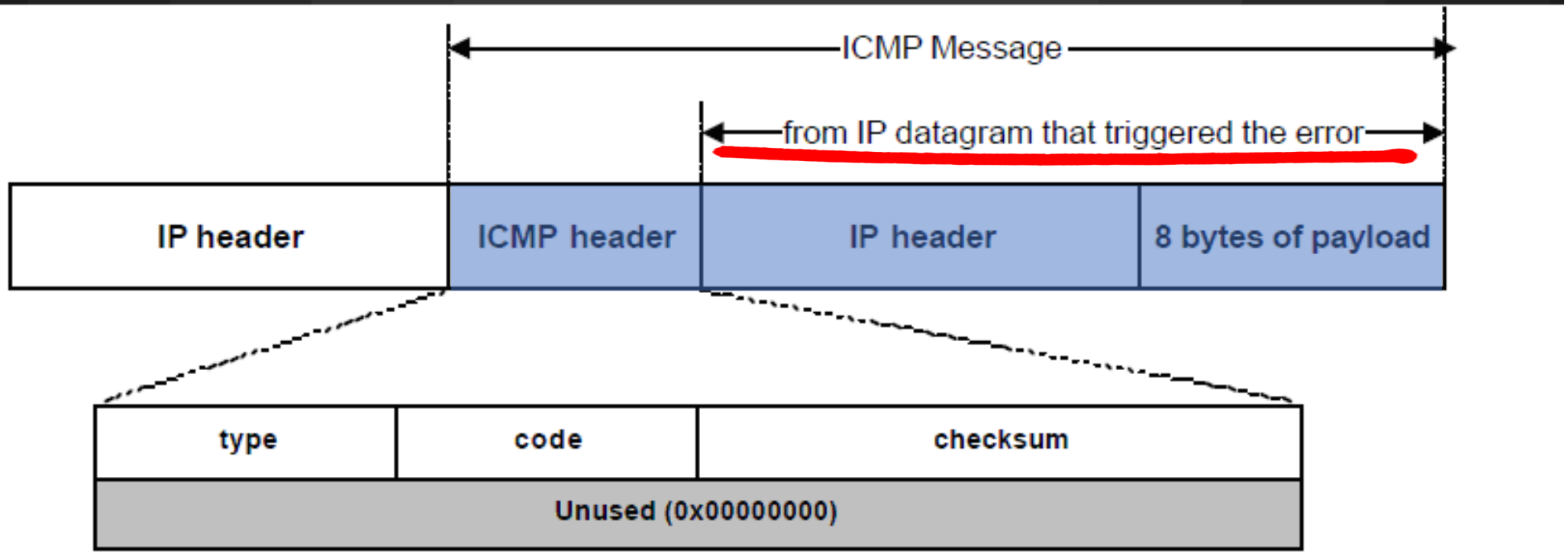
- network management, configuration
  - SNMP
  - NETCONF/YANG

ICMP: Internet control message protocol

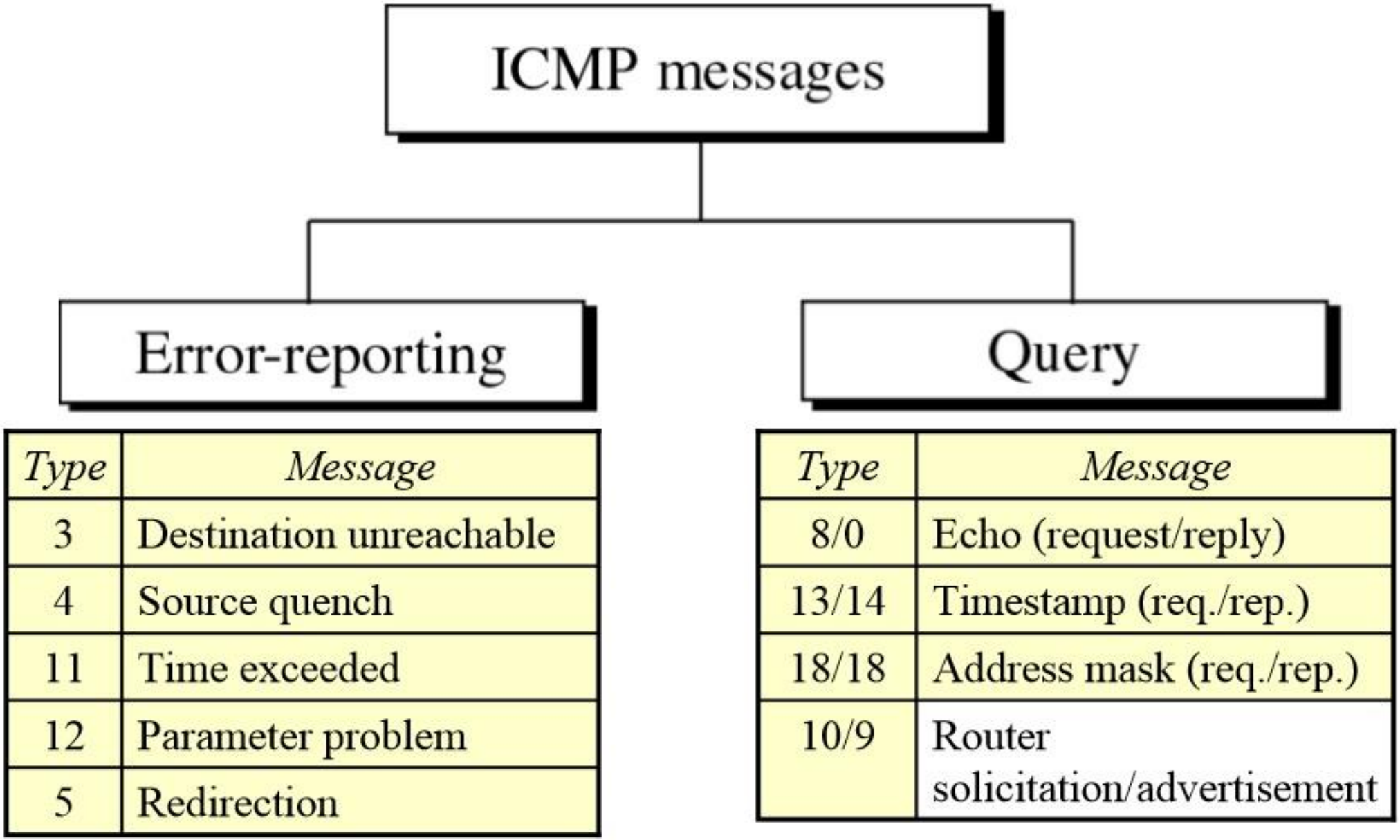
- used by hosts and routers to communicate network-level information
  - error reporting: unreachable host, network, port, protocol
  - echo request/reply (used by ping)
- network-layer "above" IP:
  - ICMP messages carried in IP datagrams
- *ICMP message:* type, code plus first 8 bytes of IP datagram causing error

Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

ICMP Error message



- ICMP error messages include the complete IP header and the first 8 bytes of the payload (typically: UDP, TCP)

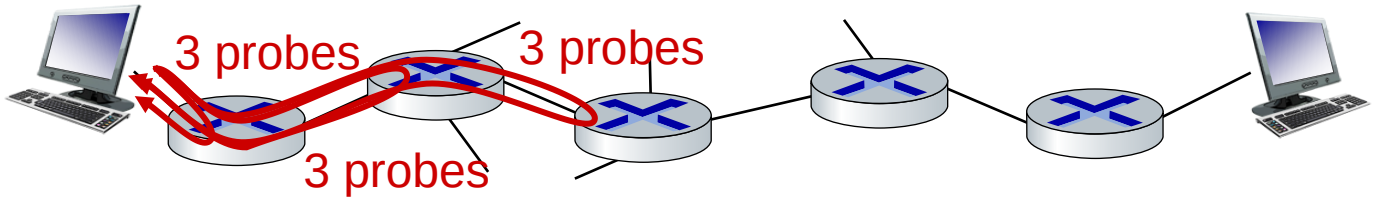


Type	*Message*
3	Destination unreachable
4	Source quench
11	Time exceeded
12	Parameter problem
5	Redirection

Type	*Message*
8/0	Echo (request/reply)
13/14	Timestamp (req./rep.)
18/18	Address mask (req./rep.)
10/9	Router solicitation/advertisement

Traceroute and ICMP

Traceroute is implemented with ICMP messages.



- source sends sets of UDP segments to destination
  - 1st set has TTL =1, 2nd set has TTL=2, etc.
- datagram in *n*th set arrives to *n*th router:
  - router discards datagram and sends source ICMP message (type 11, code 0, TTL expired)
  - ICMP message possibly includes name of router & IP address
- when ICMP message arrives at source: record RTTs

stopping criteria:

- UDP segment eventually arrives at destination host
- destination returns ICMP "port unreachable" message (type 3, code 3)
- source stops

Tracert example

命令提示符

```
Microsoft Windows [版本 10.0.18363.1556]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\Users\Administrator.DESKTOP-EEN44LR>tracert www.unibe.ch

通过最多 30 个跃点跟踪
到 cmsextrass1.unibe.ch [2001:620:400:2500::6] 的路由:

  1     6 ms     9 ms     3 ms  240e:3a1:d11:8175:d2d7:83ff:feb9:613d
  2     8 ms    18 ms    16 ms  240e:1a:2000::54
  3     6 ms    16 ms    11 ms  240e:1a:2000:5154::2
  4    11 ms    10 ms    17 ms  240e:1a:2000:f117::2
  5    11 ms    17 ms    21 ms  240e::1:21:61:6a02
  6     *        *        *     请求超时。
  7    16 ms    15 ms    25 ms  240e:0:a::c9:533d
  8     *        *        *     请求超时。
  9   221 ms   211 ms   214 ms  2001:7f8:1::a502:1320:1
 10   255 ms   256 ms   268 ms  ae7.mx1.fra.de.geant.net [2001:798:cc::1a]
 11   221 ms   228 ms   227 ms  ae3.mx1.gen.ch.geant.net [2001:798:cc::e]
 12   343 ms   270 ms   271 ms  switch-ias-switch-gw.gen.ch.geant.net [2001:798:1::be]
 13   281 ms   285 ms   272 ms  swiCE4-B4.switch.ch [2001:620:0:c00c::2]
 14   283 ms   286 ms   290 ms  swiLS1-100GE-0-0-0-1.switch.ch [2001:620:0:c04f::2]
 15   264 ms   266 ms   255 ms  swiBE3-100GE-0-1-0-1.switch.ch [2001:620:0:c005::1]
 16   262 ms   255 ms   256 ms  castor-40GE-0-0-0-0.unibe.ch [2001:620:0:fff0::2]
 17   253 ms   258 ms   259 ms  2001:620:400:d8ff::3
 18     *        *        *     请求超时。
 19     *        *        *     请求超时。
 20     *        *        *     请求超时。
 21     *        *        *     请求超时。
 22     *        *        *     请求超时。
 23     *        *        *     请求超时。
 24     *        *        *     请求超时。
 25     *        *        *     请求超时。
 26     *        *        *     请求超时。
 27     *        *        *     请求超时。
 28     *        *        *     请求超时。
 29     *        *        *     请求超时。
 30     *        *        *     请求超时。

跟踪完成。

C:\Users\Administrator.DESKTOP-EEN44LR>
```

Network layer: "control plane" roadmap

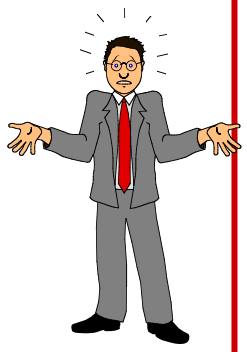
- introduction
- routing protocols
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



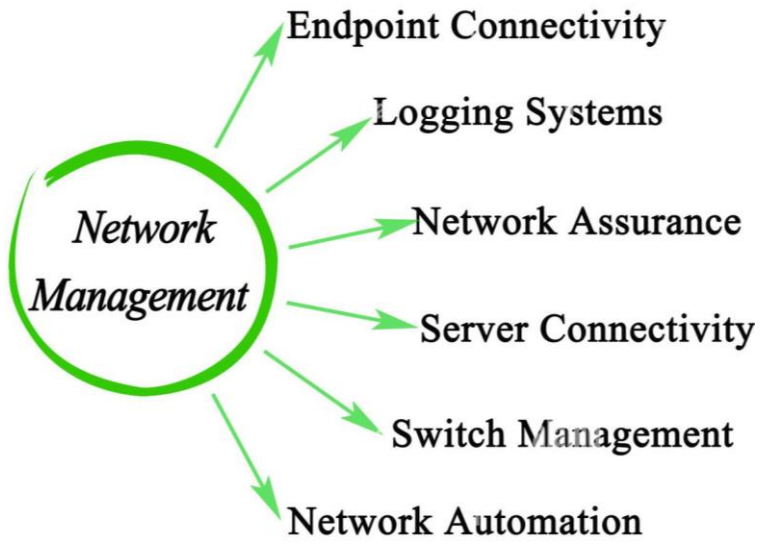
- network management, configuration
  - SNMP
  - NETCONF/YANG

What is network management?

- autonomous systems (aka “network”): 1000s of interacting hardware/software components
- other complex systems requiring monitoring, configuration, control:
  - jet airplane, nuclear power plant, others?



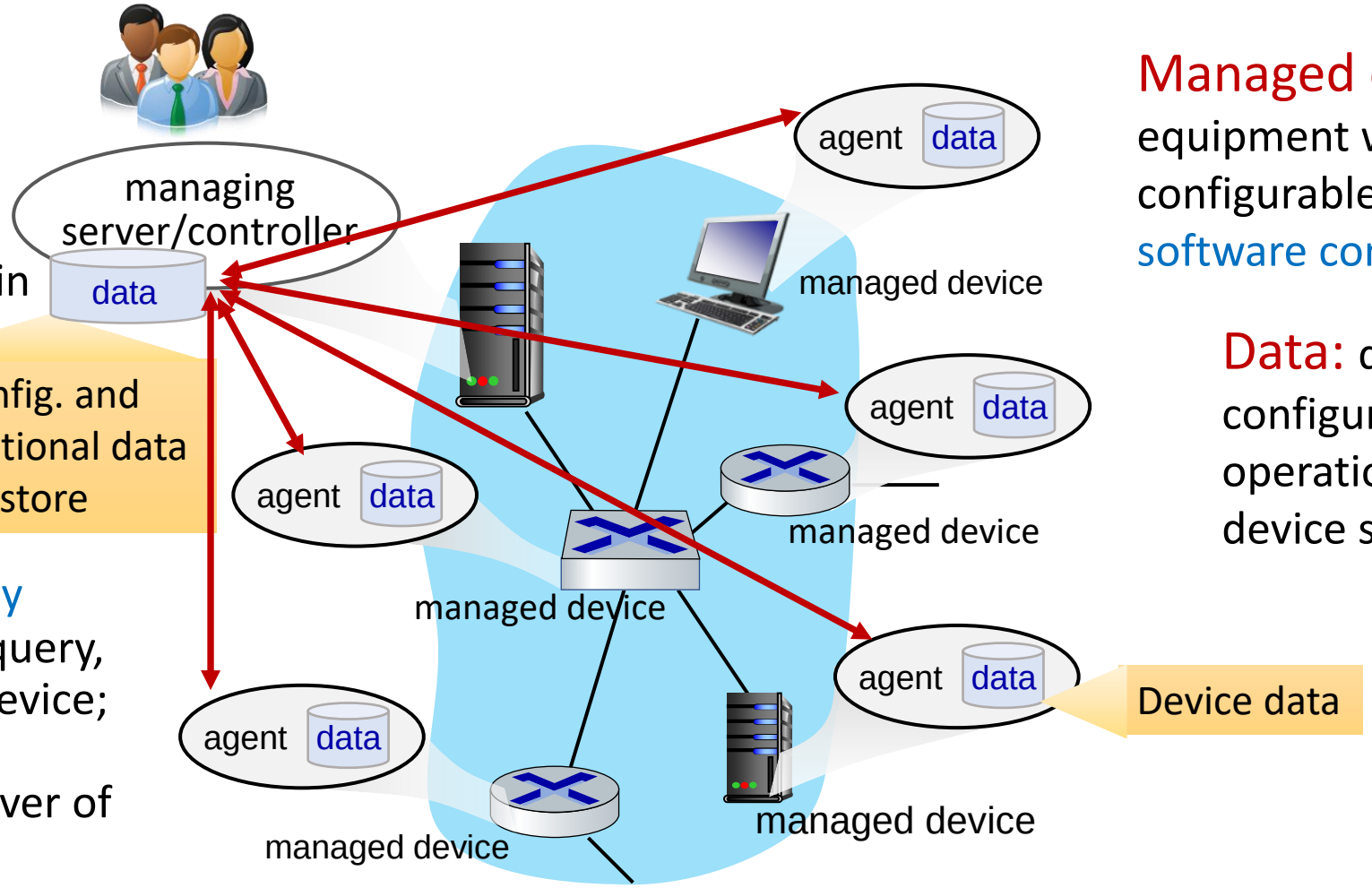
"Network management includes the deployment, integration and coordination of the hardware, software, and human elements to monitor, test, poll, configure, analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost."



Components of network management

Managing server: application, typically with network managers (humans) in the loop in NOC

Network management protocol: ❶used by managing server to query, configure, manage device; ❷used by devices to inform managing server of data, events.



Managed device: equipment with manageable, configurable hardware, software components

Data: device "state" configuration data, operational data, device statistics

Network operator approaches to management

prone to errors

CLI (Command Line Interface)

- operator issues (types, scripts) direct to individual devices (e.g., vis ssh)

Not good at large-scale network management

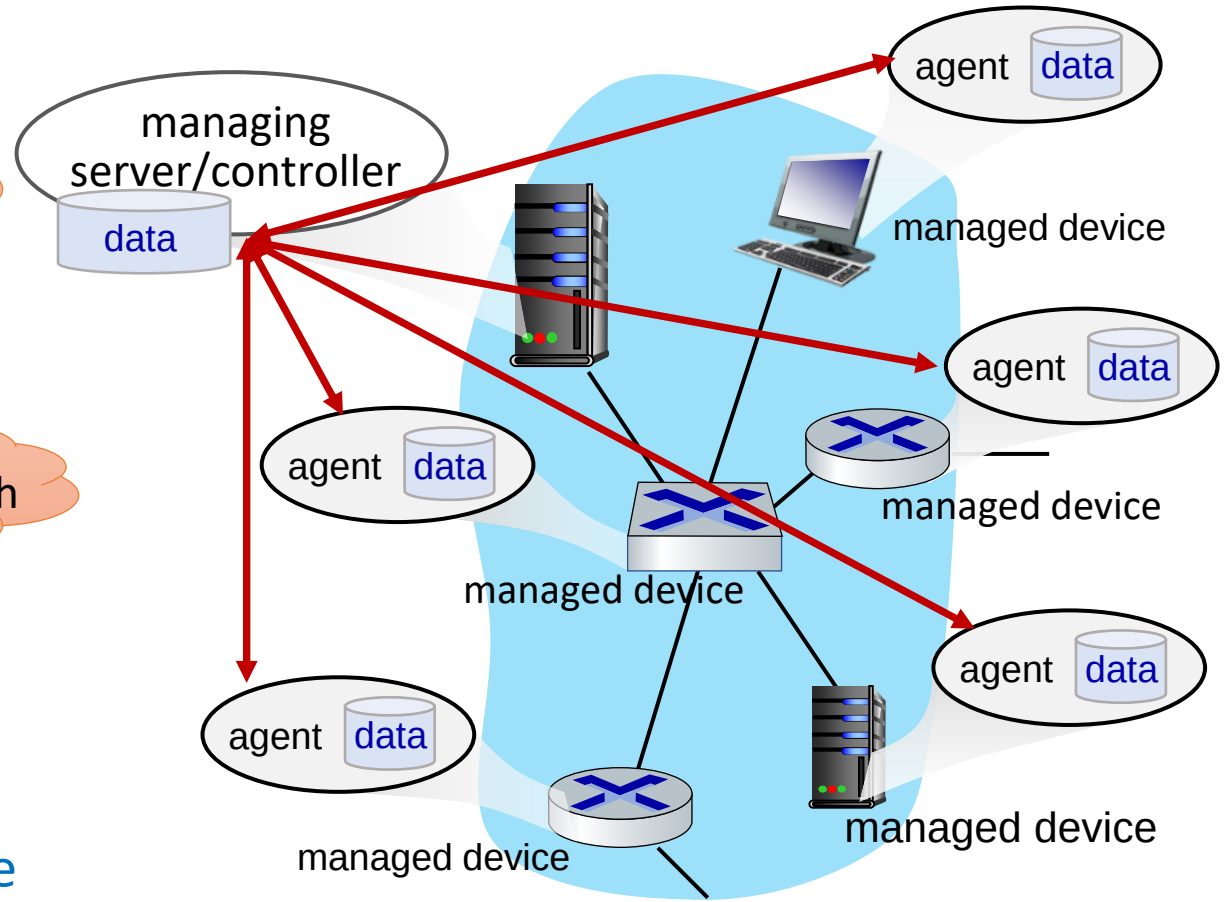
SNMP/MIB

- operator queries/sets devices data (MIB) using Simple Network Management Protocol (SNMP)

most recent approach

NETCONF/YANG

- more abstract, network-wide, holistic
- emphasis on multi-device configuration management.
- YANG: data modeling language
- NETCONF: communicate YANG-compatible actions/data to/from/among remote devices



Network operator using CLI for management

- Commands can be typed directly on a managed device's console or over a Telnet or secure shell (SSH) connection, possibly via scripting.
- CLI commands are vendor and device-specific and can be rather arcane.
- CLI use is prone to errors, and it is difficult to automate or efficiently scale for large networks.



Wireless home router setup





Wireless Setup

While this approach may work well for single, simple devices and is less error-prone than CLI, it also doesn't scale to larger-sized networks.

SNMP/MIB (Management Information Base)

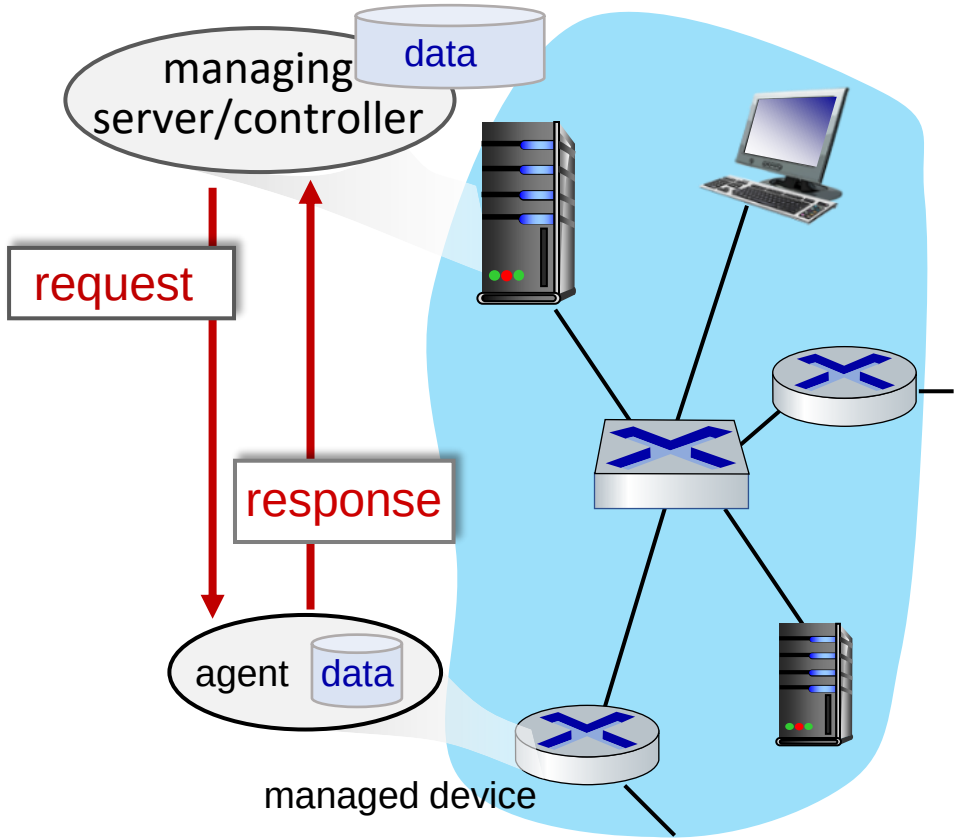
have been in use since the late 1980s

- A network operator would most typically use SNMP to query and monitor operational state and device statistics, and then use CLI to actively control/configure the device.
- Both approaches (SNMP/MIB and CLI) manage devices *individually*.
- SNMP's shortcomings: particularly for device configuration and network management at scale.
  - This gave rise to the most recent approach for network management, using NETCONF and YANG.
- The **Simple Network Management Protocol** version 3 (SNMPv3) [RFC 3410] is an application-layer protocol used to convey network-management control and information messages between a managing server and an agent executing on behalf of that managing server.

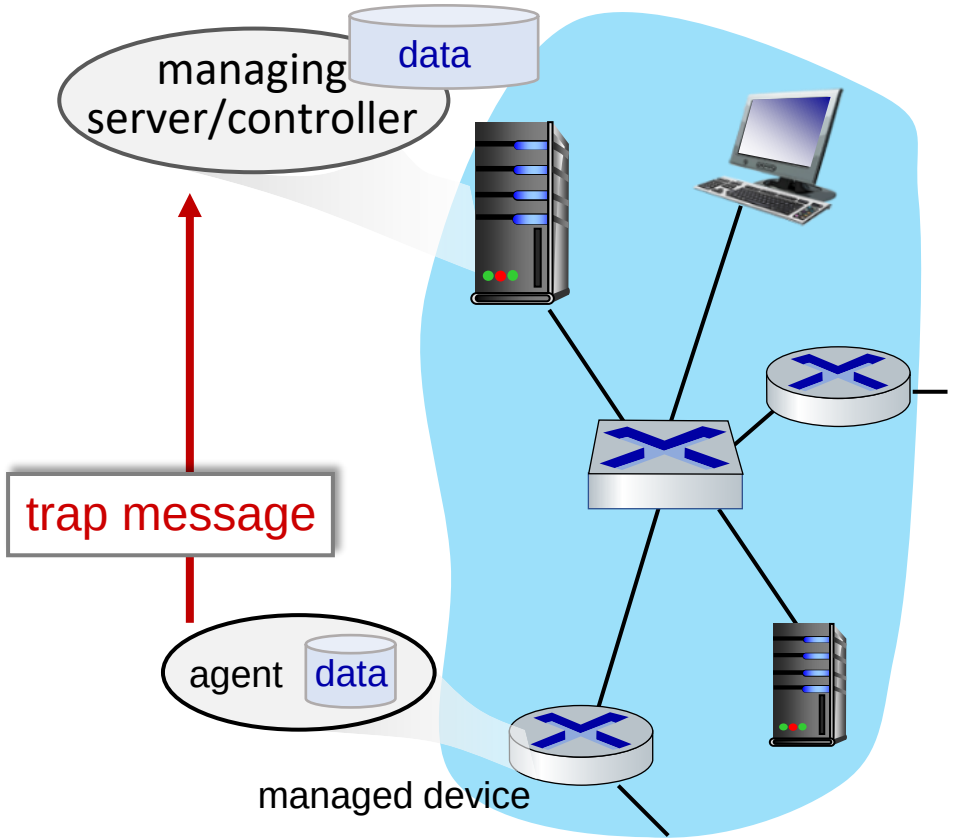
SNMP protocol

Trap messages are used to notify a managing server of an exceptional situation that has resulted in changes to MIB object values.

Two ways to convey MIB info, commands:



request/response mode



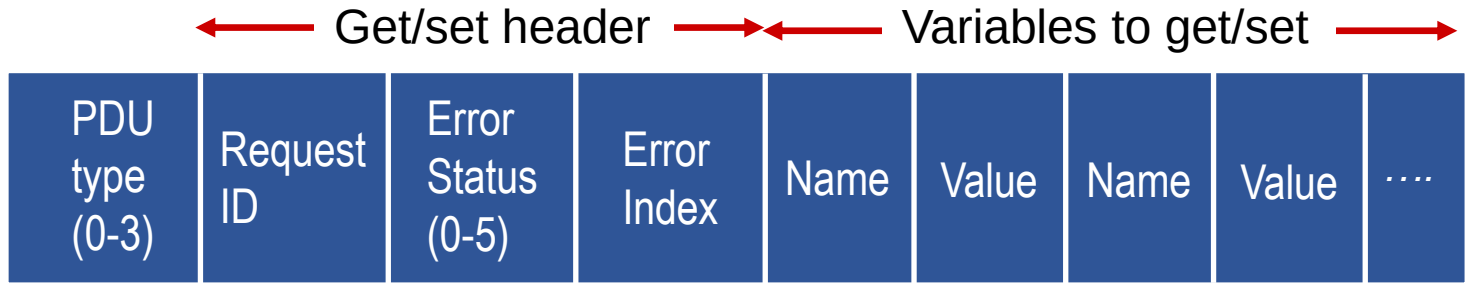
trap mode

SNMP protocol: message types

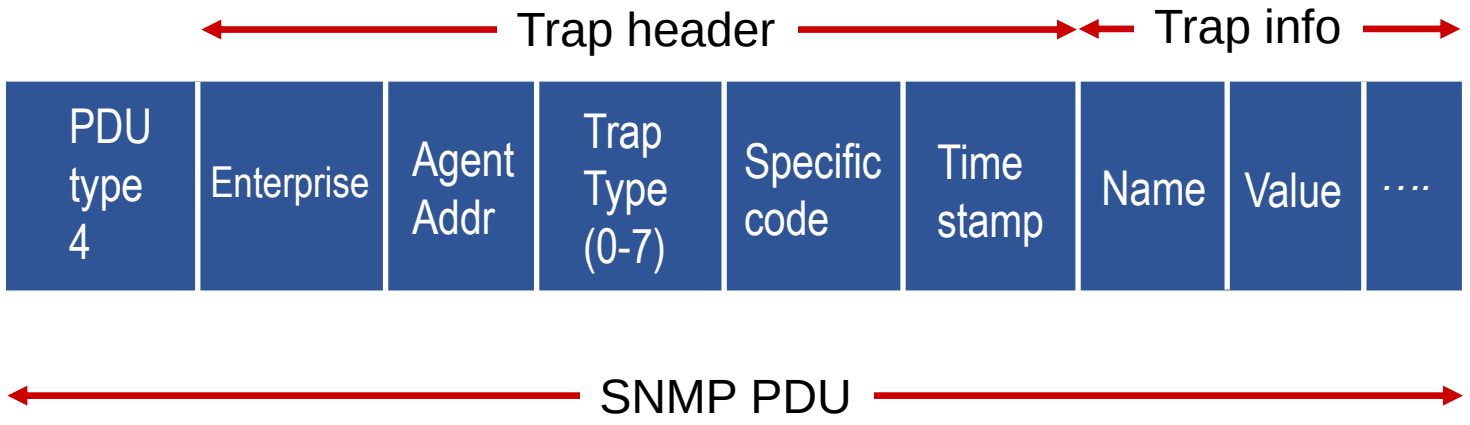
Message type	Function
GetRequest GetNextRequest GetBulkRequest	manager-to-agent: "get me data" (data instance, next data in list, block of data).
SetRequest	manager-to-agent: set MIB value
Response	Agent-to-manager: value, response to Request
Trap	Agent-to-manager: inform manager of exceptional event

SNMP protocol: message formats

message types 0-3



message type 4



SNMP: Management Information Base (MIB)



- managed device's operational (and some configuration) data
- gathered into device MIB module
  - 400 MIB modules defined in RFC's; many more vendor-specific MIBs
- Structure of Management Information (SMI): data definition language
- example MIB variables for UDP protocol:

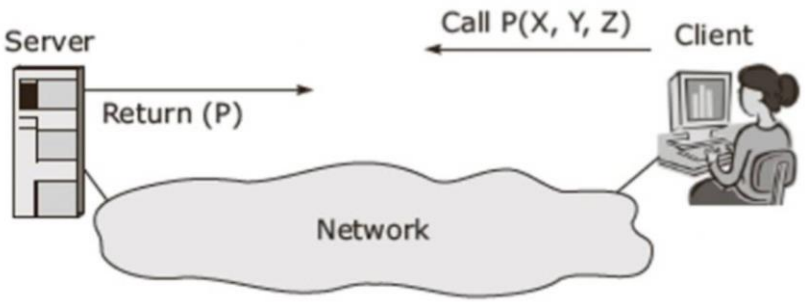
Object ID	Name	Type	Comments
1.3.6.1.2.1.7.1	UDPInDatagrams	32-bit counter	total # datagrams delivered
1.3.6.1.2.1.7.2	UDPNoPorts	32-bit counter	# undeliverable datagrams (no application at port)
1.3.6.1.2.1.7.3	UDInErrors	32-bit counter	# undeliverable datagrams (all other reasons)
1.3.6.1.2.1.7.4	UDPOutDatagrams	32-bit counter	total # datagrams sent
1.3.6.1.2.1.7.5	udpTable	SEQUENCE	one entry for each port currently in use

NETCONF overview

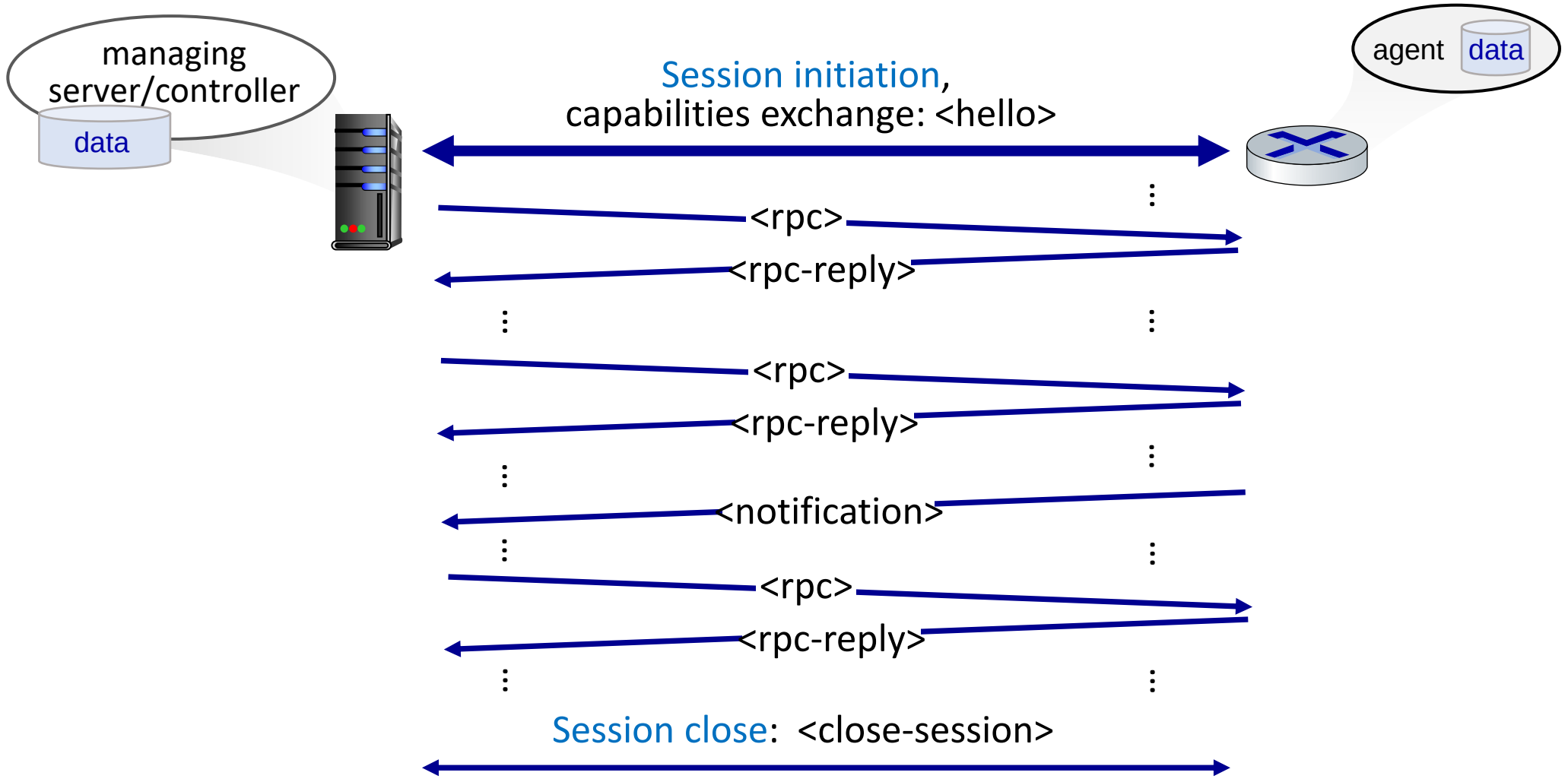
- GOAL: actively manage/configure devices network-wide
- operates between managing server and managed network devices
  - actions: retrieve, set, modify, activate configurations
  - atomic-commit actions over multiple devices
  - query operational data and statistics
  - subscribe to notifications from devices
- Remote Procedure Call (RPC) paradigm
  - NETCONF protocol messages encoded in XML
  - exchanged over secure, reliable transport (e.g., TLS) protocol over TCP

Remote Procedure Call

Basic RPC operation



NETCONF initialization, exchange, close



Selected NETCONF Operations

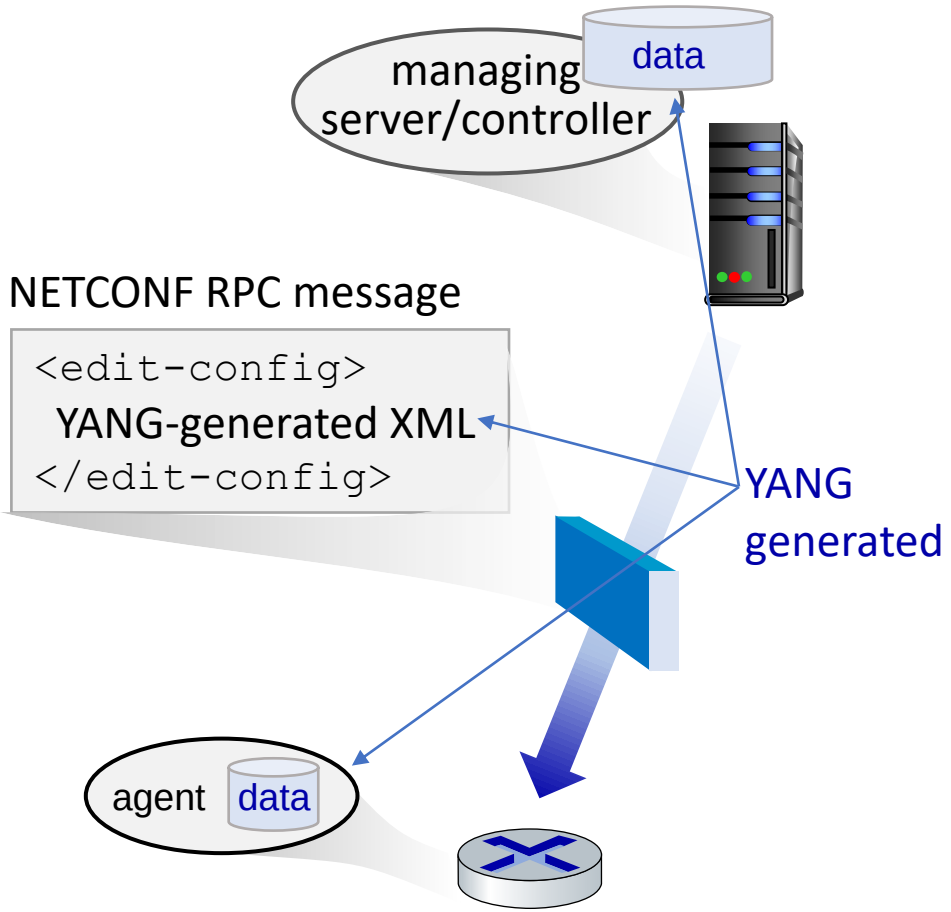
NETCONF	Operation Description
<get-config>	Retrieve all or part of a given configuration. A device may have multiple configurations.
<get>	Retrieve all or part of both configuration state and operational state data.
<edit-config>	Change specified (possibly running) configuration at managed device. Managed device <rpc-reply> contains <ok> or <rpcerror> with rollback.
<lock>, <unlock>	Lock (unlock) configuration datastore at managed device (to lock out NETCONF, SNMP, or CLIs commands from other sources).
<create-subscription>, <notification>	Enable event notification subscription from managed device

Sample NETCONF RPC message

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <rpc message-id="101"                    note message id
03   xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
04   <edit-config>                         change a configuration
05     <target>
06       <running/>                        change the running configuration
07     </target>
08     <config>
09       <top xmlns="http://example.com/schema/
         1.2/config">
10         <interface>
11           <name>Ethernet0/0</name>      change MTU of Ethernet 0/0 interface to 1500
12           <mtu>1500</mtu>
13         </interface>
14       </top>
15     </config>
16   </edit-config>
17 </rpc>
```

YANG

- data modeling language used to specify structure, syntax, semantics of NETCONF network management data
  - built-in data types, like SMI
- XML document describing device, capabilities can be generated from YANG description
- can express constraints among data that must be satisfied by a valid NETCONF configuration
  - ensure NETCONF configurations satisfy correctness, consistency constraints



Network layer: Summary

we've learned a lot!

- approaches to network control plane
  - per-router control (traditional)
  - logically centralized control (software defined networking)
- traditional routing algorithms
  - implementation in Internet: OSPF , BGP
- SDN controllers
  - implementation in practice: ODL, ONOS
- Internet Control Message Protocol
- network management

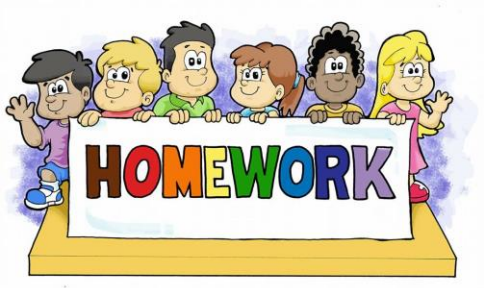
Network layer, control plane: Done!

- introduction
- routing protocols
  - link state
  - distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol



- network management, configuration
  - SNMP
  - NETCONF/YANG

Homework



1. Compare OSPF and IS-IS
2. Today's common Internet routing protocols (RIP, OSPF, BGP) are largely **load-insensitive**. Is that true or false? How to solve the resulting insufficient problem?
3. Give an example about BGP incidents and explain the reasons.
4. BGP is often used to implement the IP-anycast service, which is commonly used in DNS. Give more relevant information.
5. Introduce ICMP
6. Which kind of networks is SDN generally deployed in? Introduce the development of the **future** SDN architectures and capabilities.

WireShark LAB

Internet Control Message Protocol

